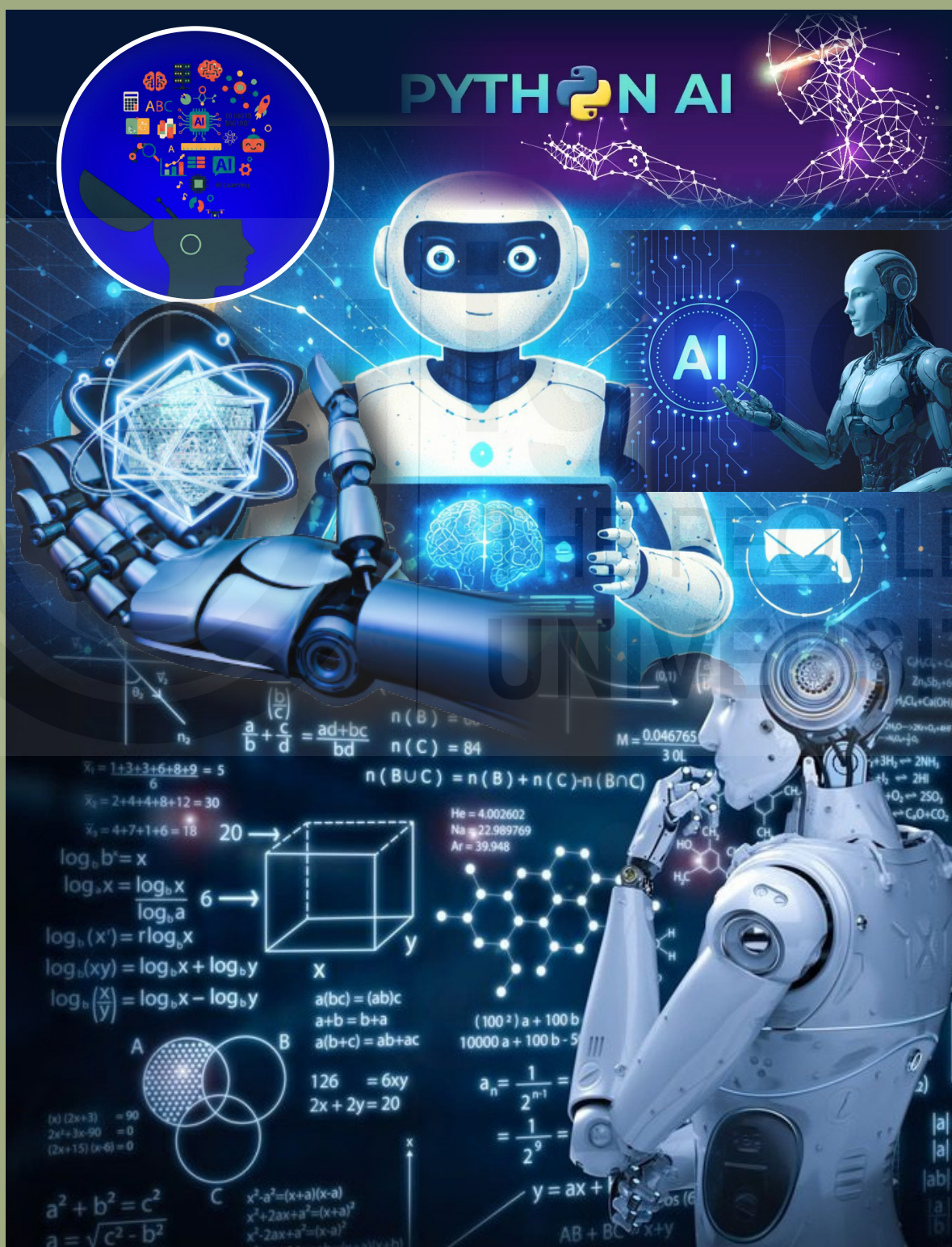




PROGRAMME DESIGN COMMITTEE

Prof. Vivek Kumar Singh, University of Delhi, Delhi
Prof. Dinesh Kumar Vishwakarma, DTU, New Delhi
Prof. Seeja K.R., IGDTUW, Delhi
Prof. R.K. Agarwal, JNU, New Delhi

Prof. Sandeep Singh Rawat, SOCIS, IGNOU
Prof. P.V. Suresh, SOCIS, IGNOU
Prof. V.V. Subrahmanyam, SOCIS, IGNOU
Prof. Divakar Yadav, SOCIS, IGNOU
Prof. Akshay Kumar, SOCIS IGNOU
Prof. M.P. Mishra, SOCIS IGNOU
Dr. Sudhansh Sharma, Associate Professor, SOCIS, IGNOU

COURSE DESIGN COMMITTEE

Prof. Vivek Kumar Singh, University of Delhi, Delhi
Prof. Dinesh Kumar Vishwakarma, DTU, New Delhi
Prof. Seeja K.R., IGDTUW, Delhi
Prof. R.K. Agarwal, JNU, New Delhi

Prof. Sandeep Singh Rawat, SOCIS, IGNOU
Prof. P.V. Suresh, SOCIS, IGNOU
Prof. V.V. Subrahmanyam, SOCIS, IGNOU
Prof. Divakar Yadav, SOCIS, IGNOU
Prof. Akshay Kumar, SOCIS IGNOU
Prof. M.P. Mishra, SOCIS IGNOU
Dr. Sudhansh Sharma, Associate Professor, SOCIS, IGNOU

SOCIS FACULTY

Prof. Sandeep Singh Rawat, Director, SOCIS, IGNOU
Prof. V.V. Subrahmanyam, SOCIS, IGNOU
Prof. Akshay Kumar, SOCIS, IGNOU
Dr. Sudhansh Sharma, Associate Professor, SOCIS, IGNOU

Prof. P.V. Suresh, IGNOU, SOCIS, IGNOU
Prof. Divakar Yadav, SOCIS, IGNOU
Prof. M.P. Mishra, SOCIS, IGNOU
Dr. Adwitiya Sinha, Associate Professor, SOCIS, IGNOU

COURSE PREPARATION TEAM

Content Writer (Unit-1) (*Adopted from MCS 224*)
Dr Sudhansh Sharma, Associate professor, SOCIS, IGNOU

Content Writer (Unit-2 & 3) (*Adopted from MCS 224*)
Dr Anant Kumar Jayswal, Asst. Professor, Amity School of Engineering & technology

Content Writer (Unit-4) (*Adopted from MCS 224*)
Prof. Arvind, Department of Mathematics, Hansraj College, University of Delhi

Content Writer (Unit-5 & 6) (*Adopted from MCS 224*)
Dr Sudhansh Sharma, Assoc. Professor, SOCIS, IGNOU

Content Writer (Unit-7) (*Adopted from MCS 224*)
Dr Manish Kumar, Asst Professor, CIC University of Delhi

Content Writer (Unit-8)
Prof. Manoj Kumar Gupta, School of Computer Science and Engineering, Shri Mata Vaishno Devi University, Katra, Jammu

Content Writer (Unit-9 & 10) (*Adopted from MCSE-003*)
Prof. Manohar Lal, SOCIS, IGNOU, New Delhi
Sh. Neeraj Sharma, Lecturer, Department of Computer Science, Ram Lal Anand College, New Delhi

Content Writer (Unit-11 & 12)
Dr Shikha Jain, Department of Computer Science & Engineering, IIIT, Noida

Content Editor(s):
Dr Mohit Kumar, NIT Hamirpur (HP)
Prof. Divakar Yadav, SOCIS, IGNOU, New Delhi

Language Editor:
Prof. Santoshi Sengupta, Department of Management, Graphics Era Hill University, Bhimtal, Uttarakhand

COURSE COORDINATOR

Prof. Divakar Yadav, Professor, School of Computer and Information Sciences, IGNOU (divakaryadav@ignou.ac.in)

PRINT PRODUCTION

Mr. Sanjay Aggrawal, Assistant Registrar (Publication)
July, 2026

©Indira Gandhi National Open University, July 2026

ISBN –

All rights reserved. No part of this work may be reproduced in any form, by mimeography or any other means, without permission in writing from the Indira Gandhi National Open University.

Further information on the Indira Gandhi National Open University courses may be obtained from the University's office at Maidan Garhi, New Delhi 110068.

CRC prepared in-house at SOCIS by Mr Vishal Singh (Skilled Worker)

Printed at:

Printed and published on behalf of the Indira Gandhi National Open University, New Delhi by MPDD, IGNOU.

Table of Contents

Topics	Page No.
Block-1 Artificial Intelligence -Introduction:	
UNIT 1 INTRODUCTION TO ARTIFICIAL INTELLIGENCE	
1.0 Introduction	13
1.1 Objectives	14
1.2 Basics of Artificial Intelligence (AI)?	15
1.3 Brief history of Artificial Intelligence	18
1.4 Components of Intelligence	20
1.5 Approaches to Artificial Intelligence	24
1.6 Comparison between Artificial Intelligence (AI) Machine Learning (ML) and Deep Learning (DL)	27
1.7 Application Areas of Artificial Intelligence Systems	29
1.8 Intelligent Agents	32
1.9 Summary	41
1.10 Solutions/Answers	41
1.11 Further Readings	42
UNIT 2 PROBLEM SOLVING USING SEARCH	
2.0 Introduction	43
2.1 Objectives	46
2.2 Introduction to State Space Search	46
2.2.1 Problem Formulation	47
2.2.2 Structure of a State Space	47
2.2.3 Problem solution of State space	47
2.3.4 Searching for solution in state spaces formulated	48
2.3 Formulation of 8 puzzle problem from AI Perspective	50
2.4 N-queen's problem- Formulation and Solution	52
2.4.1 Formulation of 8 Queen's problem	55
2.4.2 State space tree for 4-Queen's problem	55
2.4.3 Backtracking approach to solve N Queen's problem	57
2.5 Two agent search: Adversarial search	60
2.5.1 Elements of Game Playing Search	60
2.5.2 Types of algorithms in Adversarial search	62
2.6 Minimax search strategy	63
2.6.1 Minimax algorithm	63
2.6.2 Working of Minimax algorithm	64

2.6.3 Properties of Minimax Algorithm	67
2.6.4 Advantages and Disadvantages of Minimax Search	67
2.7 Alpha-Beta Pruning Algorithm	68
2.7.1 Working of Alpha-Beta Pruning	68
2.7.2 Move Ordering of Alpha-Beta Pruning	77
2.8 Summary	80
2.9 Solutions/Answers	82
2.10 Further readings	90

UNIT 3 UNINFORMED & INFORMED SEARCH

3.0 Introduction	92
3.1 Objectives	94
3.2 Formulating search in state space	95
3.2.1 Evaluation of the Search Algorithm	97
3.3 Uninformed Search	97
3.3.1 Breath-First search (BFS)	98
3.3.2 Time and space complexity of BFS	101
3.3.3 Advantages & disadvantages of BFS	101
3.3.4 Depth First Search (DFS)	104
3.3.5 Performance of DFS algorithm	104
3.3.6 Advantages and disadvantages of DFS	105
3.3.7 Comparison of BFS and DFS	105
3.4 Iterative Deepening Depth First search (IDDFS)	106
3.4.1 Time and space complexity of IDDFS	108
3.4.2 Advantages and Disadvantages of IDDFS	108
3.5 Bidirectional search	109
3.6 Comparison of Uninformed search strategies	111
3.7 Informed (heuristic) search	112
3.7.1 Strategies for providing heuristics information	113
3.7.2 Formulation of Informed (heuristic) search problem as a state space	114
3.7.3 Best-First search	115
3.7.4 Greedy Best first search	115
3.8 A* Algorithm	120
3.8.1 Working of A* algorithm	122
3.8.2 Advantages and disadvantages of A* algorithm	124
3.8.3 Admissibility properties of A* algorithm	124
3.8.4 Properties of heuristic algorithm	125
3.8.5 Results on A* algorithm	126
3.9 Problem reduction search	126
3.9.1 Problem definition in AND-OR graph	127
3.9.2 AO* algorithm	130

3.9.3 Advantages of AO* algorithm	131
3.10 Memory Bound heuristic search	131
3.10.1 Iterative Deepening A* (IDA*)	131
3.10.2 Working of IDA*	132
3.10.3 Analysis of IDA*	133
3.10.4 Comparison of A* and IDA* algorithm	134
3.11 Recursive Best First search (RBFS)	134
3.11.1 Advantages and disadvantages of RBFS	135
3.12 Summary	137
3.13 Solutions/Answers	140
3.14 Further readings	144

Block-2 Artificial Intelligence - Knowledge Representation:

UNIT 4 PREDICATE AND PROPOSITIONAL LOGIC

4.1	Introduction	147
4.2	Objectives	148
4.3	Introduction to Propositional Logic	148
4.4	Syntax of Propositional Logic	150
	4.4.1 Atomic Propositions	150
	4.4.2 Compound Propositions	150
4.5	Logical Connectives	152
	4.5.1 Conjunction	152
	4.5.2 Disjunction	152
	4.5.3 Negation	152
	4.5.4 Implication	153
	4.5.5 Bi-Conditional	153
4.6	Semantics	154
	4.6.1 Negation Truth Table	156
	4.6.2 Conjunction/Disjunction/Implication/ Biconditional Truth Table	156
	4.6.3 Truth Table with three variables	157
4.7	Propositional Rules of Inference	159
	4.7.1 Modus Ponens (MP)	159
	4.7.2 Modus Tollens (MT)	159
	4.7.3 Disjunctive Syllogism (DS)	159
	4.7.4 Addition	159
	4.7.5 Simplification	160
	4.7.6 Conjunction	160
	4.7.7 Hypothetical Syllogism (HS)	160
	4.7.8 Absorption	160

4.7.9	Constructive Dilemma (CD)	160
4.8	Propositional Rules of Replacement	160
4.9	Validity and Satisfiability	161
4.10	Introduction to Predicate Logic	162
4.11	Inferencing in Predicate Logic	164
4.12	Proof Systems	166
4.13	Natural Deduction	167
4.14	Propositional Resolution	171
	4.14.1 Clausal Form	171
	4.14.2 Determining Unsatisfiability	171
4.15	Answers/Solutions	178
4.16	Further Readings	180

UNIT 5 FIRST ORDER PREDICATE LOGIC

5.0	Introduction	181
5.1	Objectives	182
5.2	Syntax of First Order Predicate Logic(FOPL)	182
5.3	Interpretations in FOPL	183
5.4	Semantics of Quantifiers	186
5.5	Inference & Entailment in FOPL	192
5.6	Conversion to clausal form	199
5.7	Resolution & Unification	204
5.8	Summary	209
5.9	Solutions/Answers	210
5.10	Further/Readings	212

UNIT 6 RULE BASED SYSTEMS AND OTHER FORMALISMS

6.0	Introduction	213
6.1	Objectives	214
6.2	Rule-Based Systems	215
	6.2.1 Forward chaining	217
	6.2.2 Backward chaining	220
	6.2.3 Conflict resolution	224
6.3	Semantic nets	226
6.4	Frames	228
6.5	Scripts	229
6.6	Summary	231
6.7	Solutions/Answers	231
6.8	Further/Readings	232

Block 3: Artificial Intelligence – Decision Making

UNIT 7 PROBABILISTIC REASONING

7.0	Introduction	235
7.1	Objectives	235
7.2	Reasoning with uncertain information	236
7.3	Review of Probability Theory	236
7.4	Introduction to Bayesian Theory	241
7.5	Bayes' Networks	242
7.6	Probabilistic Inference	242
7.7	Basic idea of Inferencing with Bayes Networks	243
7.8	Other Paradigms of Uncertain Reasoning	244
7.9	Dempster-Shafer Theory	244
7.10	Summary	249
7.11	Solutions/ Answers	250
7.12	Further Readings	251

UNIT 8 DECISION MAKING

8.0	Introduction	252
8.1	Objectives	252
8.2	Basics of utility theory	253
8.3	Decision theory	257
8.4	Sequential decision problems	262
	8.4.1 Markov decision process	262
	8.4.2 Sequential decision problems solving with Value Iteration	264
8.5	Elementary game theory	267
	8.5.1 Introduction to game theory	267
	8.5.2 Key Concepts in Game Theory	268
	8.5.3 Nash Equilibrium	269
8.6	Sample applications	271
8.7	Summary	272
8.8	Solutions/Answers	273

UNIT 9 EXPERT SYSTEMS

9.0	Introduction	277
9.1	Objectives	277
9.2	An Introduction to Expert Systems	278
9.3	Concept of Planning, Representing, and Using Domain Knowledge	279
9.4	Knowledge Representation Schemes	279
9.4	Examples of Expert Systems: MYCIN, COMPASS	286
9.6	Expert System Building Tools	287

9.6.1	Expert System Shells	288
9.6.1.1	Knowledge Base	289
9.6.1.2	Knowledge Acquisition Subsystem (Example: COMPASS)	289
9.6.1.3	Inference Engine	290
9.6.1.4	Explanation Sub-system (Example: MYCIN)	290
9.6.1.5	User Interface	293
9.6.1.6	An Expert System shell: EMYCIN	293
9.7	Some Application Areas of Expert Systems	294
9.8	Summary	294
9.9	Solutions/Answers	295
9.10	Further Readings	295

Block 4: Implementation and Applications of AI

UNIT 10 INTELLIGENT AGENTS

10.0	Introduction	299
10.1	Objectives	300
10.2	Definitions	300
10.3	Agents and Rationality	302
10.3.1	Rationality vs. Omniscience	302
10.3.2	Autonomy and learning capability of the agent	303
10.3.3	Example: A boundary following robot	303
10.4	Task Environment of Agents	307
10.4.1	PEAS (Performance, Environment, Actuators, Sensors)	307
10.4.2	Example: An Automated Public Road Transport Driver	307
10.4.3	Different Types of Task Environments	308
10.4.3.1	Fully Observable vs. Partially Observable Environment	308
10.4.3.2	Static vs. Dynamic Environment	309
10.4.3.3	Deterministic vs. Stochastic Environment	309
10.4.3.4	Episodic vs. Sequential Environment	309
10.4.3.5	Single agent vs. Multi-agent Environment	310
10.4.3.6	Discrete Vs. Continuous Environment	310
10.4.4	Some Examples of Task Environments	311
10.4.4.1	Crossword Puzzle	311
10.4.4.2	Medical Diagnosis	311
10.4.4.3	Playing Tic-tac-toe	311
10.4.4.4	Playing Chess	312
10.4.4.5	Automobile Driver Agent	312
10.5	The Structure of Agents	312
10.5.1	SR (Simple Reflex) Agents	313
10.5.2	Model Based reflex Agents	314

10.5.3	Goal-based Agents	314
10.5.4	Utility-based Agents	316
10.5.5	Learning Agents	317
10.6	Different Forms of Learning in Agents	319
10.7	Summary	321
10.8	Solutions/Answers	321
10.9	Further Readings	321

Unit 11: AI PROGRAMMING WITH PYTHON

11.1	Introduction to Python for AI	322
11.2	Objectives	323
11.3	Python Libraries for AI	323
11.4	Hands-on Implementations of:	349
11.4.1	N-Queen Problem	349
11.4.2	Min-Max Algorithm and Alpha-Beta Pruning	351
11.4.3	Breadth First Search	357
11.4.4	Depth First Search	359
11.4.5	Best First Search	360
11.4.6	A* Algorithm	361
11.4.7	AO* Algorithm	364
11.4.8	Iterative Deepening A*	366
11.5	Model Evaluation and Hyperparameter Tuning in Python	368
11.6	Summary	370
11.7	Solutions/Answers	370
11.8	Further Readings	371

Unit 12: Applications of AI

12.0	Introduction	372
12.1	Objective	373
12.2	AI in healthcare	373
12.3	AI in finance	376
12.4	AI in transportation	379
12.5	AI in customer service and chatbots	382
12.6	AI in education	384
12.7	Summary	386
12.8	Solutions/ Answers	387
12.9	Further Reading	389

COURSE INTRODUCTION

The course **Artificial Intelligence** (MCS-081) is designed to provide learners with a comprehensive understanding of the fundamental concepts, techniques, and real-world applications. It introduces Artificial Intelligence (AI) as a discipline concerned with building systems capable of performing tasks that typically require human intelligence, such as reasoning, learning, decision-making, and problem-solving. The course begins by exploring the meaning, historical evolution, and various approaches to AI, along with a clear distinction between AI, Machine Learning, and Deep Learning. It also introduces the concept of intelligent agents and the essential components that contribute to intelligent behavior.

A central theme of the course is **problem-solving through search**, where learners examine how complex problems can be represented as state spaces and solved using systematic exploration strategies. The course covers both single-agent and multi-agent environments, including adversarial search techniques such as Min-Max and alpha-beta pruning used in game-playing scenarios. Building on this, students are introduced to uninformed and informed search strategies, emphasizing efficiency and optimality through algorithms like iterative deepening, bidirectional search, A*, AO*, and recursive best-first search.

The course further develops a strong foundation in **knowledge representation and logical reasoning**, focusing on propositional and first-order logic. Learners explore syntax, semantics, inference rules, and proof techniques, which are essential for constructing intelligent systems capable of drawing valid conclusions. This logical framework is extended through the study of rule-based systems and other representation schemes such as semantic networks, frames, and scripts, enabling structured representation and reasoning over domain knowledge.

To address real-world uncertainty, the course introduces **probabilistic reasoning and decision-making**, including Bayesian networks, probabilistic inference, and alternative uncertainty models like Dempster-Shafer theory. Concepts from utility theory, decision theory, and elementary game theory equip learners with tools to model rational decision-making in uncertain and dynamic environments. The study of expert systems further illustrates how AI techniques are applied to emulate human expertise in specific domains, highlighting knowledge representation, inference mechanisms, and practical applications.

A dedicated unit on **intelligent agents** provides insight into different types of agents, their architectures, task environments, and learning capabilities. Learners gain an understanding of how agents perceive their environment and act rationally to achieve goals. The course also includes a practical component

on **AI programming using Python**, where learners work with widely used libraries such as NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, and PyTorch. Through hands-on implementation of key algorithms, including search techniques, game-playing strategies, and model evaluation, students develop practical problem-solving and programming skills.

Finally, the course highlights the **diverse applications of AI** across sectors such as healthcare, finance, transportation, education, and customer service. These examples demonstrate the transformative impact of AI technologies in solving complex real-world problems.

Overall, the course aims to build a strong conceptual foundation, analytical thinking, and practical competence, preparing learners for advanced study and professional roles in the rapidly evolving field of Artificial Intelligence.



ignou
THE PEOPLE'S
UNIVERSITY

Block

1

Artificial Intelligence – Introduction

UNIT 1

Introduction to Artificial Intelligence	13
--	-----------

UNIT 2

Problem Solving Using Search	43
-------------------------------------	-----------

UNIT 3

Uninformed & Informed Search	91
---	-----------

BLOCK-1 INTRODUCTION

The Block-1, titled “Artificial Intelligence -Introduction,” is comprised of three units. The details are as follows:

- Unit 1 Introduction to Artificial Intelligence
- Unit-2 Problem Solving using Search
- Unit-3 Uninformed and Informed Search

Unit 1- “Introduction to Artificial Intelligence” introduces the fundamental concepts of Artificial Intelligence (AI), including its meaning, history, and significance in modern applications. It explains the distinction between knowledge and intelligence, discusses the key components of intelligence such as learning and reasoning, and presents different approaches to AI. The unit also compares AI with Machine Learning and Deep Learning, highlights major application areas, and introduces the concept of intelligent agents, thereby providing a comprehensive foundation for understanding the field of AI.

Unit-2 “Problem Solving using Search,” focuses on problem-solving in Artificial Intelligence using state space search techniques. It explains how real-world problems can be formulated in terms of initial states, goal states, and operators, and solved by searching through possible states to find an optimal solution path. The unit introduces key concepts such as state space representation, search trees and graphs, and path cost, along with classical examples like the water jug problem, 8-puzzle, and N-Queen problem. It also highlights the importance of selecting appropriate search strategies and control mechanisms to efficiently explore the problem space and reach the desired goal.

Finally, Unit-3 “Uninformed and Informed Search” introduces search strategies in Artificial Intelligence, focusing on how problems are solved by exploring a state space from an initial state to a goal state. It covers both uninformed (blind) search techniques such as Breadth-First Search (BFS), Depth-First Search (DFS), Iterative Deepening, and Bidirectional Search, and informed (heuristic) search methods like Best-First Search, Greedy Search, and the A* algorithm. The unit also discusses the formulation and evaluation of search algorithms based on criteria such as completeness, optimality, time complexity, and space complexity. Additionally, it introduces advanced concepts like problem reduction, AO* algorithm, and memory-bounded search techniques, providing a comprehensive understanding of efficient problem-solving methods in AI.

UNIT 1 INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Structure	Page No.
1.0 Introduction	13
1.1 Objectives	14
1.2 Basics of Artificial Intelligence (AI)	15
1.3 Brief history of Artificial Intelligence	18
1.4 Components of Intelligence	20
1.5 Approaches to Artificial Intelligence	24
1.6 Comparison between Artificial Intelligence (AI) Machine Learning (ML) and Deep Learning (DL)	27
1.7 Application Areas of Artificial Intelligence Systems	29
1.8 Intelligent Agents	32
1.9 Summary	41
1.10 Solutions/Answers	41
1.11 Further Readings	42

1.0 INTRODUCTION

Today, artificial intelligence (AI) is used in a wide variety of applications, including engineering, technology, the military, opinion mining, sentiment analysis, and many more. It is also used in more advanced domains, such as language processing and applications for aerospace.

AI is everywhere in today's world, and people are gradually becoming accustomed to its presence. It is utilised in systems that recognise both voices and faces. In addition to this, it can provide you with shopping recommendations that are tailored to your own purchasing preferences. Finding spam and preventing fraudulent use of credit cards is made much easier when you have this skill. The most cutting-edge technology currently on the market are virtual assistants such as Apple's Siri, Amazon's Alexa, Microsoft's Cortana, and Google's own Google Assistant. You may already be familiar with the technology involved in artificial intelligence (AI). Are you?

AI has become very popular all over the world today. It imitates human intelligence in machines by programming them to do the same things people do. As a technology, AI is going to have a bigger impact on how people live their daily lives. Everyone wants to connect to AI as a technology these days. Before we can understand AI, we need to know and talk about some basic things. For example, what is the difference between knowledge and intelligence? The key to starting this unit is the answer to this question.

The accumulation of information and abilities that a person has gained through their life experiences is known as knowledge, while intelligence refers to one's capacity to put one's knowledge into practice. To put it simply, knowledge is what we have learned over the years, and it expands as time passes. Because of this, it represents the culmination of everything that we have realised over the course of our lives. It is important to highlight that having information does not automatically make one intelligent; rather, intelligence is what makes one

smart.

There is a well-known proverb that says, "marks are not the measure of intelligence." This is due to the fact that intelligence is not a measurement of how much information one possesses. In fact, it is the measure of how much we comprehend and put into practice. People who are knowledgeable may gain a lot of information, but an intelligent person understands how to comprehend, analyse, and use the information. You could have a lot of knowledge but still be the least intelligent person in the room. Knowledge and intelligence are inextricably linked, and each contributes to the other's development. Knowledge enables one to learn the understandings that others have of things, whereas intelligence is the foundation for one's ability to grasp the things themselves.

Now that we have an understanding of the distinction between intelligence and knowledge, our next issue is: what exactly is artificial intelligence? Incorporating intelligence into a machine is related to the field of Artificial Intelligence, whereas both concepts, namely the representation of knowledge and its engineering, are the basis of traditional AI research. This topic will be discussed in section 1.3 of this unit, but in a nutshell, incorporating intelligence into a machine is related to the field of Artificial Intelligence. Knowledge engineering is a subfield of artificial intelligence (AI) that applies rules to data in order to simulate the way in which an expert would think about the information. It does this by analysing the structure of a job or a decision in order to figure out how one arrives at a conclusion.

In the subsequent units of this course, you will learn about some of the concepts that are essential for knowledge representation, such as frames, scripts, and other related topics. In addition, this course will address the issues that are associated with the knowledge representation for uncertain situations, such as employing the method of fuzzy logic, rough sets, and the Dempster-Shafer theory, among other relevant topics.

Some of the prerequisites to get started with this subject.

- a) Strong understanding of basic concepts of Mathematics viz. Algebra, Calculus, Probability, and Statistics.
- b) Experience in programming using Python or Java.
- c) A good understanding of algorithms.
- d) Reasonably good data analytics skills.
- e) Fundamental knowledge in discrete mathematics.
- f) Finally, internal will to learn

1.1 OBJECTIVES

After going through this unit, you will be able to:

- Understand the difference between knowledge and intelligence
- Answer – what is AI?
- Identify various approaches of AI
- Compare Artificial Intelligence (AI), Machine Learning (ML) & Deep Learning (DL).
- Understand the concept of agents in AI

1.2 BASICS OF ARTIFICIAL INTELLIGENCE

Knowledge and intelligence are two important concepts, and we were able to gain an understanding of the fundamental distinction between the two terms. Now that we have your attention, let's talk about what artificial intelligence actually is. The meaning of Artificial Intelligence will be covered in this section; however, before we get started, it is important to note that the field of Artificial Intelligence is related to the process of incorporating intelligence into machines; the specifics of this process, as well as the mechanism itself, will be covered in this unit as well as the subsequent units of this training.

The following is a list of eight definitions of artificial intelligence that have been provided by well-known authors of artificial intelligence textbooks.

- 1) According to Haugeland (1985), "the exciting new effort to make computers think... Machines with minds, in the full and literal sense."
- 2) According to Bellman (1978), "the automation of behaviours that we connect with human thinking, activities such as decision-making, problem-solving, and learning..."
- 3) According to Charniak and McDermott (1985), "the study of mental capabilities through the application of computer models" (also known as "The Study of Mental Capabilities")
- 4) According to Winston (1992), "the study of the calculations that make it possible to perceive, reason, and act."
- 5) According to Kurzweil (1990), "the art of building machines that execute functions that demand intellect when performed by people."
- 6) According to Schalkoff (1990), "a field of study that aims to explain and replicate intelligent behaviour in terms of computational processes."
- 8) According to Luger and Stubblefield (1993), "the discipline of computer science that is concerned with the automation of intelligent behaviour."

According to the concepts presented earlier, there are four distinct objectives that might be pursued in the field of artificial intelligence. These objectives are as follows:

- The creation of systems that think in the same way as people do.
- The creation of systems that are capable of logical thought.
- The creation of machines that can mimic human behaviour.
- The creation of systems that behave in a logical manner

In addition, we discovered through our conversation in the earlier section 1.1 of this Unit, that Artificial Intelligence (AI) is the intelligence that is incorporated into machines; in other words, AI is the ability of a machine to display human-like capabilities such as reasoning, learning, planning, and creativity. We learned this information because AI is the ability of a machine to display human-like capabilities. Taking into account the Emerging AI technologies to sense, interpret, and act according to the circumstances,

relevant exemplary solutions are summarised in Figure 1, which attempts to encapsulate the understanding of the question "What is Artificial Intelligence?"

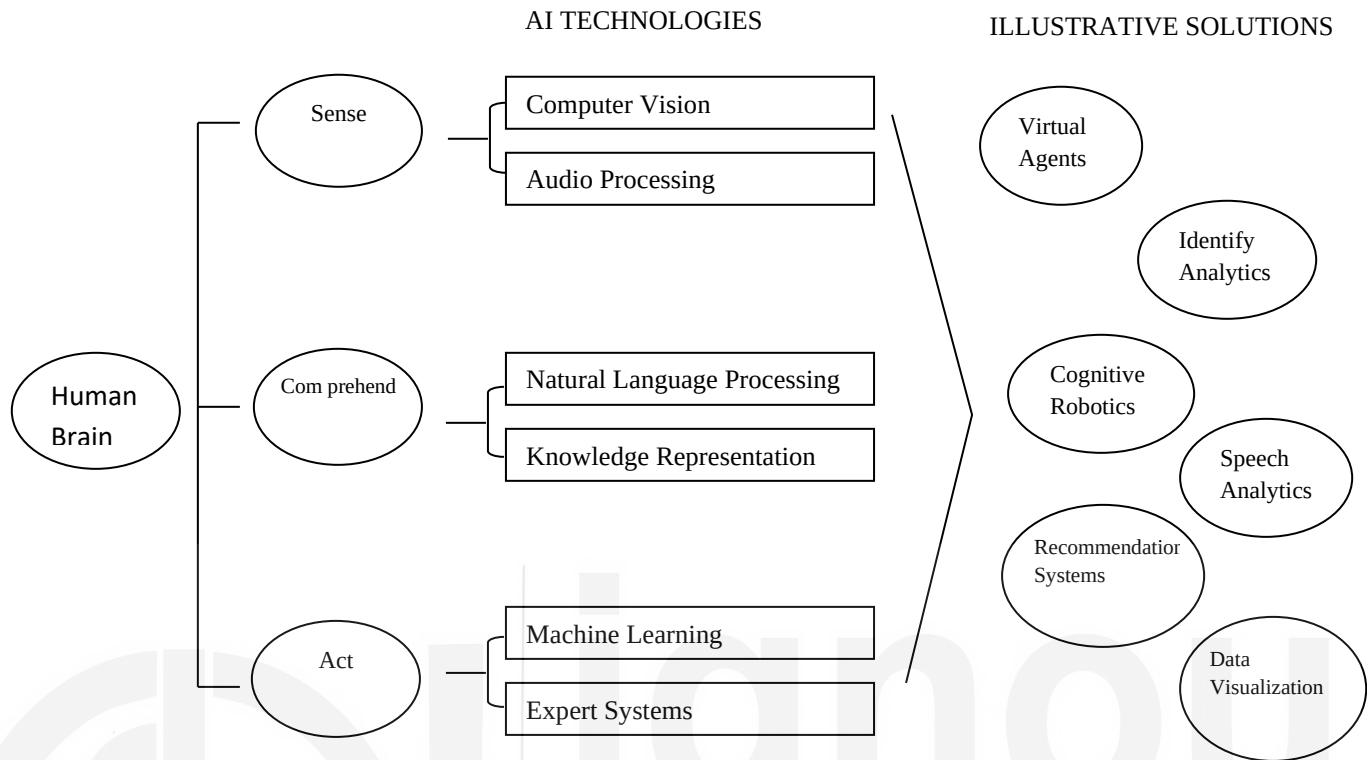


Figure 1: What is Artificial Intelligence? – Emerging AI Technologies

The applications of artificial intelligence (AI) that are shown in figure-1 do not all require the same kinds of AI. In a general sense, one may say that AI can be categorised into the following levels:

- software level (i.e., Embodied AI) where the software level consists of things like search engines, virtual assistants, speech and facial recognition systems, picture analysis tools, and other things like that.
- Hardware level (Embedded) includes Robots, autonomous vehicles, drones, the Internet of Things, and other technologies that fall under the category of embedded artificial intelligence (AI).

On the basis of the functionalities, the AI can be classified based on Type 1 and Type 2, as shown in Figure 1(a).

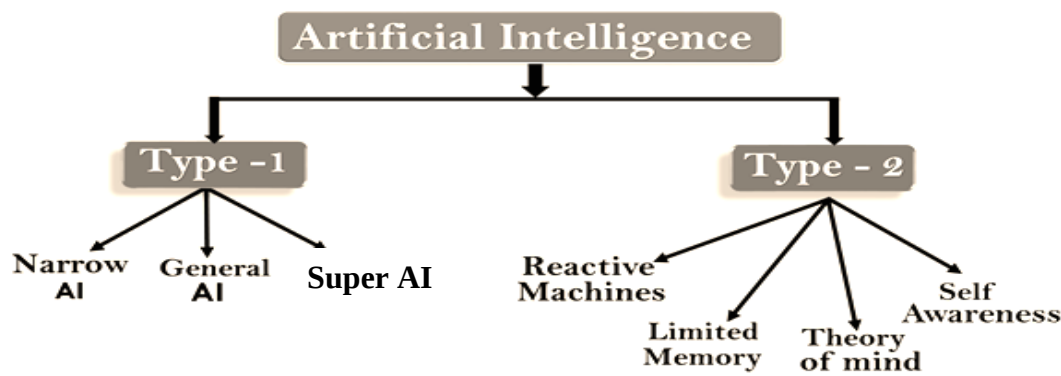


Figure 1(a) – Classification of Artificial Intelligence

Here's a brief introduction to the first type of AI i.e., Type 1 AI. Following are the three stages of Type 1 - Artificial Intelligence:

- a) Artificial Narrow Intelligence-(ANI)
- b) Artificial General Intelligence-(AGI)
- c) Artificial Super Intelligence-(ASI)

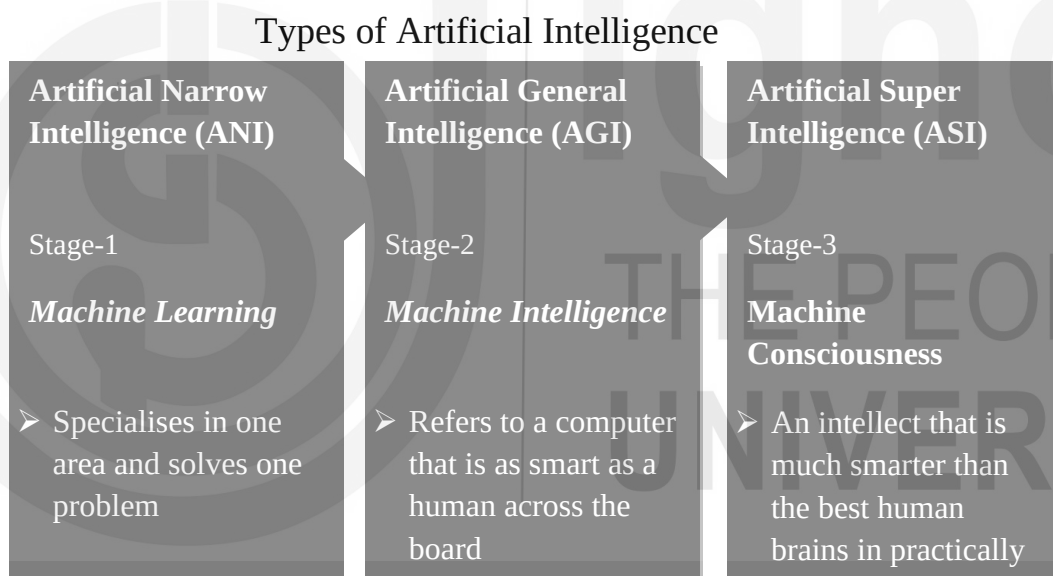


Figure 1(b) –Three Stages of Type-I Artificial Intelligence

The various categories of Artificial Intelligence are discussed as follows:

- a) Artificial Narrow Intelligence (ANI), also called Weak AI or Narrow AI: Weak AI is a term for thinking that is "simulated." Such systems seem to act intelligently, but they don't have any awareness of what they are doing. For example, a chatbot might talk to you in a way that seems natural, but it doesn't know who it is or why it's talking to you. Artificial intelligence is a system that was built to do a certain job.
- b) Artificial General Intelligence (AGI): Strong or General Artificial Intelligence, also called "actual" thinking. That is, acting like a smart human and thinking like one with a conscious, subjective mind. For instance, when two humans talk, they probably both know who they are, what they're doing, and why.

Systems with strong or general artificial intelligence can do things that

people can do. These systems tend to be harder to understand and more complicated. They are set up to handle situations where they might need to solve problems on their own without help from a person. Uses for these kinds of systems include self-driving cars and operating rooms in hospitals.

- c) Artificial Super Intelligence (ASI) - Super intelligence: The term "super intelligence" usually refers to a level of general and strong AI that is smarter than humans, if that's even possible. The ASI is seen as the logical next step after the AGI because it can do more than humans can. This includes making decisions, making rational decisions, and even things like building emotional relationships. There is a marginal difference between AGI and ASI.

☞ Check Your Progress 1

1. How does knowledge differ from intelligence? What do you understand by the term Artificial Intelligence (AI)? List the various technologies and their corresponding illustrative solutions.

.....

.....

.....

2. Classify AI on the basis of the functionalities of AI

.....

.....

.....

3. Compare ANI, AGI and ASI, in context of AI

.....

.....

.....

1.3 BRIEF HISTORY - ARTIFICIAL INTELLIGENCE

AI's ideas come from early research into how people learn and think, as well as the ancient concept that a computer could act like a person. The idea of machines that can think for themselves originates in Greek mythology. From 384 BC to 322 BC, Aristotle developed an informal system of syllogistic logic, which served as the foundation for the first formal system of deductive reasoning.

At the start of the 17th century, René Descartes proposed that animal bodies are merely complex machines. Later in that century, in 1642, Blaise Pascal built the first mechanical digital calculator. In the 1800s, George Boole developed "Boolean algebra," a binary mathematical system that represented

the fundamental "laws of thought." During this same century, Charles Babbage and Ada Byron worked together on programmable mechanical calculators.

In the late 19th and early 20th centuries, mathematicians and philosophers like Gottlob Frege, Bertrand Russell, Alfred North Whitehead, and Kurt Gödel built upon Boole's initial logic concepts to create mathematical representations of logical problems.

The advent of electronic computers marked a major step forward in studying intelligence. In 1943, Warren McCulloch and Walter Pitts created a Boolean circuit model of the brain, publishing their work on how neural networks can perform mathematical operations in the paper "A Logical Calculus of Ideas Immanent in Nervous Activity."

In 1950, Alan Turing published a paper titled "Computing Machinery and Intelligence," which provided a comprehensive overview of AI. This paper discussed how to solve problems by using heuristics as guides to search through a space of possible solutions, using the game of chess to explain how machines can think. Turing even suggested that a machine could modify its own instructions to learn from experience. (To learn more about Alan Turing, visit turing.org.uk).

In 1951, Marvin Minsky and Dean Edmonds built the SNARC (Stochastic Neural Analog Reinforcement Computer), which was the first randomly wired neural network learning machine, operating with a network of 40 neurons and 3,000 vacuum tubes. Shortly after, between 1952 and 1956, Arthur Samuel developed a series of programmes to help people play checkers.

In 1956, a well-known meeting took place at Dartmouth College, where the pioneers of the field met for the first time and officially chose the name "Artificial Intelligence." At this meeting, Allen Newell and Herbert Simon introduced The Logic Theorist, which many consider to be the first running artificial intelligence programme.

In 1959, Herbert Gelernter created the Geometry Theorem Proving Machine. This was followed in 1961 by James Slagle's MIT Ph.D. dissertation on a LISP-based programme called SAINT, which a first-year college student could use to solve calculus problems. In 1963, Thomas Evans developed a programme called "Analogy" that could solve geometric analogy problems typical of IQ tests. That same year, Edward A. Feigenbaum and Julian Feldman compiled *Computers and Thought*, the first collection of articles about artificial intelligence.

In 1965, J. Allen Robinson introduced the Resolution Method, a way to prove things mechanically that allowed formal logic to work efficiently as a language for representing programmes. In 1967, Feigenbaum, Joshua Lederberg, Bruce Buchanan, and Georgia Sutherland at Stanford demonstrated how the DENDRAL programme could understand the mass spectra of organic chemical compounds, making it the first successful programme based on domain-specific scientific knowledge. In 1969, the SRI robot Shakey proved that it was possible to integrate mobility, vision, and autonomous problem-solving all at the same time.

From 1969 to 1979, the first knowledge-based systems were established. Early AI had been based on general, weak systems that lacked specific knowledge. AI researchers soon realised that for machines to reason about complex tasks, they needed substantial knowledge about a narrow field.

As part of this movement, MYCIN demonstrated the power of rule-based systems in 1974 by representing knowledge and drawing conclusions for medical diagnosis and treatment. Around the same time, Marvin Minsky introduced the concept of "frames," plans were initiated for the Knowledge Representation Version 2 course at CSE, IIT Kharagpur, and logic-based programming languages like Prolog and Planner rose to prominence.

By the 1980s, specialized Lisp machines were being manufactured and commercialised. In 1985, neural networks experienced a major resurgence in popularity, followed by a renewed interest in probabilistic and decision-theoretic methods in 1988. In 1989, Dean Pomerleau developed ALVINN (Autonomous Land Vehicle in a Neural Network) at CMU. This system learned to drive by watching a human operator, using a neural network that processed a 30x32 unit image from a two-dimensional camera to control steering. The system successfully steered a vehicle from the East Coast to the West Coast of the United States—a distance of about 2,850 miles—with a human driving for only about 50 of those miles.

During the 1990s, AI made significant progress across many areas, including machine learning, data mining, intelligent tutoring, case-based reasoning, multi-agent planning and scheduling, uncertain reasoning, natural language understanding and translation, computer vision, virtual reality, and gaming. At MIT, Rodney Brooks' COG Project made substantial cooperative progress toward building a humanoid robot.

Key milestones of the late 1990s included the 1997 debut of the official RoboCup soccer competition, where 40 teams of robots interacted on a tabletop field. That same year, IBM's Deep Blue chess programme defeated the reigning world champion, Garry Kasparov. Additionally, as the World Wide Web grew rapidly, web crawlers and other AI-based information extraction programmes became increasingly critical.

By 2000, autonomous systems entered extreme environments. The Nomad robot travelled to remote parts of Antarctica to search for meteorite samples, while robotic space probes began working autonomously to explore space by monitoring their environments, making decisions, and taking actions to reach their targets. In April 2004, the initial three-month missions of NASA's Mars rovers concluded successfully; the Spirit rover spent two months reaching and exploring a group of Martian hills to study uniquely eroded rocks, while its twin rover, Opportunity, examined rock layers inside an impact crater. Finally, as the Internet continued its rapid expansion, demand spiked for intelligent internet agents capable of tracking user behaviour, searching for required data, and filtering out the most useful information.

1.4 COMPONENTS OF INTELLIGENCE

According to the dominant school of thought in psychology, human

intelligence should not be viewed as a singular talent or cognitive process but rather as a collection of distinct components. The majority of attention in the field of artificial intelligence research has been paid to the following aspects of intelligence: learning, reasoning, problem-solving, perception, and language comprehension.

Learning

There are numerous approaches to develop a learning system. Making mistakes is the simplest way to learn. A basic software that solves "mate in one" chess issues, for example, might test different moves until it finds one that answers the problem. The programme remembers which move worked so that the next time the computer is given the identical situation, it can provide an immediate response. The simple act of memorising things like answers to problems, words in a vocabulary list, and so on is known as "rote learning" or memorization.

We'll talk about another classification that doesn't depend on the way knowledge is represented or how it is represented. According to this system, there are five ways to learn:

- (i) Rote Learning or memorising.
- (ii) Learning by instructions
- (iii) Learning by analogy.
- (iv) Learning by deduction
- (v) Learning by induction

Rote Learning: Rote learning is the simplest method of learning since it involves the least amount of interpretation. The information is simply copied into a database in this method of learning. This is the technique for memorising multiplication tables.

On a computer, rote learning is relatively simple to implement. The difficulty of implementing what is known as generalisation is more difficult. Generalized learning allows the student to perform better in situations they haven't faced before. A programme that learns the past tenses of regular English verbs by rote will not be able to form the past tense of "jump" until it has been presented with "jumped" at least once, whereas a programme that can generalise from examples will be able to learn the "added" rule and thus form the past tense of "jump" even if it has never encountered this verb before. Modern techniques allow programmes to generalise complex rules based on data.

Learning by instructions: The next method of learning is to be instructed. Because new knowledge must be contributed to an existing knowledge base in order to be useful, this type of learning necessitates greater inference. When a teacher instructs a student, this is the type of learning that occurs.

Learning by analogy: When you learn by analogy, you generate new ideas by connecting previously learned concepts. Textbooks frequently employ this method of instruction. For example, in the text, some problems are solved as examples, and students are subsequently given problems that are comparable to the examples. This type of learning also occurs when someone who can drive a light car, attempts to drive a heavy vehicle.

Learning by deduction: Finally, we discuss deductive learning, which is based on deductive inference, a non-debatable mode of thinking. By the irrefutable method of reasoning, we mean that if the hypotheses (or given facts) are accurate, the conclusion arrived through deductive (i.e., any irrefutable) reasoning is always correct. This is the most common method of thinking in mathematics.

Learning by induction: Learning through induction is the most common method of learning. This is a method of learning that employs inductive reasoning, which is a style of reasoning that involves drawing a conclusion from a large number of good instances. If we encounter a lot of cows, we might notice that they have four legs, are white, and have two horns in the same position on their head. Even though inductive reasoning frequently leads to valid conclusions, the conclusions are not always unarguable. For example, with the above-mentioned concept of cow, we might come across a black cow, a three-legged cow that has lost one leg in an accident, or a single-horned cow. Inductive learning is a crucial component of an agent's learning architecture. An agent learns based on:

- What it is learning, such as concepts, problem-solving techniques, or game-playing techniques, etc.
- The representation, predicate calculus, frame, script, and other elements that were employed.
- The critic, who expresses their opinion of the agency in general.

Building directly on the idea of a 'critic' who evaluates performance, modern inductive systems operate through structured feedback mechanisms. Learning based on feedback is normally categorized as Supervised learning, Unsupervised learning, Reinforcement Learning, and Delayed reinforcement learning.

Supervised Learning: It has a function that learns from inputs and outputs that are shown as examples. Some examples of this kind of learning are figuring out useful things about the world from what you see, making a map from the current state's conditions to actions, and learning how the world changes over time.

Unsupervised Learning: There is no way to know what the inputs are and what the expected outputs are in this type of learning. So, the learning system has to figure out on its own which properties of objects it doesn't know about are important. For example, figuring out the shortest way to get from one city to another in a country you know nothing about.

Reinforcement (Rewards) for Learning: In some problems, the task or problem can only be seen, not said. Also, the job may be an ongoing one. The user tells the agent how happy or unhappy he or she is with the agent's work by sometimes giving the agent positive or negative rewards (i.e., reinforcements). The agent's job is to get as many rewards (or reinforcements) as possible. In a simple goal-attainment problem, the agent can be rewarded when it reaches the goal and punished when it doesn't. You need an action plan to get the most out of this kind of task. But when it comes to tasks that never end, the future reward might be endless, making it hard to decide how to get the most out of it.

One way to move forward in this kind of situation is to ignore future rewards after a certain point. That is, the agent may want rewards that will come soon more than rewards that will come a long time from now.

Delayed-reinforcement learning: It is the process of determining how to behave in situations when rewards are contingent on previous actions.

Reasoning

To reason is to draw conclusions that are appropriate for the situation. Both deductive and inductive reasoning can be used to draw conclusions. An example of a deductive inference is, "Fred is either in the museum or in the cafe. He isn't in the cafe, so he must be in the museum." An example of inductive inference is, "In the past, accidents just like this one have been caused by instrument failure." The difference between the two is that in the deductive case, the truth of the premises guarantees the truth of the conclusion, while in the inductive case, the truth of the premises supports the conclusion that instrument failure caused the accident, but further evidence could show that the conclusion is actually false, even though the premises are true.

Programming computers to make inferences, especially deductive inferences, has got a lot of success. But you can't say that a programme can reason just because it can draw conclusions. To reason, you have to draw conclusions that make sense for the task or situation at hand. Giving computers the ability to tell what is important and what isn't is one of the hardest problems AI has to face.

Problem-solving

Problems usually go like this: given these data, find x. AI is used to solve a very wide range of problems. Some examples are finding the best way to win a board game, figuring out who someone is from a picture, and planning a series of steps that will allow a robot to do a certain task. Methods for solving problems can be either specific or general. A special-purpose method is made to solve a specific problem and often takes advantage of very specific parts of the situation where the problem is happening. A general method can be used to solve many different kinds of problems. The difference between the current state and the goal state can be reduced step by step with means-end analysis, which is a technique used in AI. The programme chooses actions from a list of ways, which for a simple robot might include pick up, put down, move forward, move back, move left, and move right, until the current state is changed into the goal state.

Perception

Perception involves scanning the surroundings with numerous sense organs, genuine or artificial, and internal processes for analysing the scene into objects, their features, and relationships. The fact that one and the same item can have various appearances depending on the angle from which it is viewed, whether or not parts of it are projecting shadows, and so on, complicates analysis.

Artificial perception has progressed to the point where a self-controlled car-like device can drive at modest speeds on the open road, and a mobile robot can search a suite of bustling offices for and remove empty drink cans. FREDDY, a stationary robot with a moving TV 'eye' and a pincer 'hand,' was one of the first systems to merge perception and action (constructed at Edinburgh University during the period 1966-1973 under the direction of Donald Michie). FREDDY could recognise a wide range of items and could be taught to create simple artefacts from a jumble of parts, such as a toy automobile.

Language Comprehension

A language is a set of signs with predetermined meaning. For example, traffic signs establish a mini-language; it is a matter of convention that the hazard-ahead sign signifies trouble ahead. This language-specific meaning-by-convention is distinct from what is known as natural meaning, as evidenced by phrases like "Those clouds signify rain" and "The pressure drop suggests the valve is malfunctioning." The productivity of full-fledged human languages, such as English, separates them from other types of communication, such as bird sounds and traffic sign systems. A productive language is rich enough to allow for the creation of an infinite number of different sentences.

1.5 APPROACHES TO ARTIFICIAL INTELLIGENCE

In the previous sections of this unit, we learned about various concepts of Artificial Intelligence, but now the question is "how do we measure if Artificial Intelligence is making a machine to behave or act, or perform like a human being or not?"

Perhaps, in the future, we will reach a point where AI can behave like humans, but what guarantee do we have that this will continue? Is it possible to make a system that acts like a human to test the certainty of Artificial Intelligence? The following approaches constitute the foundation for evaluating an AI entity's human-likeness:

- Turing Test
- Approach of The Cognitive Modelling
- Approach of The Law of Thought
- Approach of The Rational Agent

Let's take a look at how these approaches perform:

In the past, researchers have worked hard to reach all four of these goals. But it is hard to find a good balance between approaches that focus on people and approaches that focus on logic. People are often "irrational" in the sense of being "emotionally unstable," so it's important to tell the difference between human and rational behaviour.

Researchers have found through their studies that a human-centered approach

must be an empirical science with hypotheses and experiments to prove them. In a rationalist approach, math and engineering are used together. People in each group sometimes say bad things about the work done by the other groups, but the truth is that each way has led to important discoveries. Let's try to answer the question, "**What is the Turing Test in AI?**" Alan Turing, who is the most well-known name among the pioneers, thought about how to test an A.I. product to see if it was intelligent. Alan Turing came up with the idea for the Turing Test in 1950 (Turing). To see if something is smart. Below is a summary of the Turing test. See Figure 2 for more details.

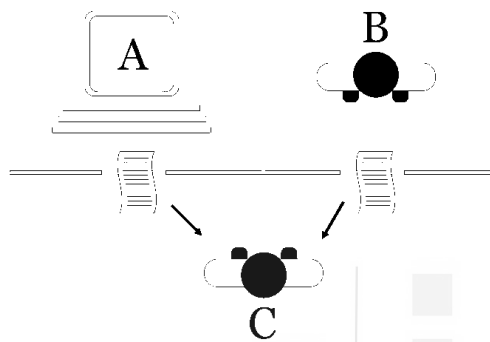


Figure 2: Turing Test

For the purpose of the test, there are three rooms that will be used for the test. In one of the rooms, there is a computer system that is said to be smart. In each of the other two rooms, there is one person sitting. One of the people, whom we'll call C, is supposed to ask questions from the computer, and the other person, who we'll call B, without knowing who each question is for and, of course, to figure out who the computer is. On the other hand, the computer would reply in a way that would keep C from finding out who it is.

The only way for the three of them to talk to each other is through computer terminals. This means that the identity of the computer or person B can only be determined by how intelligent or not the responses are, not by any other human or machine traits. If C can't figure out who the computer is, then the computer must be smart. More accurately, the computer is smart if it can hide its identity from C.

Note that for a computer to be considered smart, it should be smart enough not to answer too quickly, at least not in less than a hundredth of a second, even if it can do something like find the sum of two numbers with more than 20 digits each.

Criticism of the Turing Test: There have been a number of criticisms of the Turing test as a machine intelligence test. The Chinese Room Test, developed by John Searle, is one of the most well-known criticisms. The crux of the Chinese Room Test, which we'll discuss below, is that convincing a system, say A, that it possesses qualities of another system, say B, does not suggest that system A actually possesses those qualities. For example, a male human's ability to persuade people that he is a woman does not imply that he is capable of bearing children like a woman.

The scenario for the Chinese Room Test takes place in a single room with two windows. A Shakespeare scholar who knows English but not Chinese is sitting in the room with a kind of Shakespeare encyclopedia. The encyclopedia is printed so that for every pair of pages next to each other, one page is written in Chinese characters and the other page is an English translation of the Chinese page. Through one of the windows, Chinese characters with questions about Shakespeare's writing are sent to the person inside. The person looks through the encyclopaedia and, when he or she

finds the exact copy of the sequence of characters sent in, reads the English translation, thinks of the answer, and writes it down in English for his or her own understanding. The person then looks in the encyclopaedia for the corresponding sequence of Chinese characters and sends the sequence of Chinese characters through the other window. Now, Searle says that even though the scholar acts as though he or she knows Chinese, this is not the case. Just because a system can mimic a quality doesn't mean that it has that quality.

Thinking humanly: It is the cognitive modeling approach to thinking like a human. From this point of view, the Artificial Intelligence model is based on Human Cognition, which is the core of the human mind. This is done through three approaches, which are as follows:

- Introspection, which means to look at our own thoughts and use those thoughts to build a model.
- Psychological Experiments, which means running tests on people and looking at how they act.
- Brain imaging, which means to use an MRI to study how the brain works in different situations and then copy that through code.

Thinking rationally, i.e., the laws of thought approach: This approach relates to use the laws of thought to think logically: The Laws of Thought are a long list of logical statements that tell our minds how to work. This method, called "Thinking Rationally," is based on these laws. By putting in place algorithms for artificial intelligence, these laws can be written down and made to work. But solving a problem by following the law is very different from solving a problem in the real world. These are the biggest problems with this approach.

Acting rationally, i.e., the rational agent approach: In every situation, a rational agent approach tries to find the best possible outcome. This means that it tries to make the best decision it can, given the circumstances. It means that the agent approach is much more flexible and open to change. The Laws of Thought approach, on the other hand, says that a thing must act in a way that makes sense. But there are some situations where there is no logically right thing to do, and there is more than one way to solve the problem, each with different results and trade-offs. At that point, the rational agent method works well.

 Check Your Progress 2

- 4. Briefly discuss the various components of intelligence
.....
.....
.....
- 5. How do we measure if Artificial Intelligence is making a machine to behave or act or perform like a human being or not?
.....
.....
.....
- 6. What is the Turing Test? What is the criticism of the Turing Test?
.....

1.6 COMPARISON - ARTIFICIAL INTELLIGENCE, MACHINE LEARNING & DEEP LEARNING

Artificial intelligence is a big field that includes a lot of different ways of doing things, from top-down (knowledge representation) to bottom-up (machine learning). In recent years, people have often talked about three related ideas: artificial intelligence (AI), machine learning (ML), and deep learning (DL). AI is the most general term, machine learning is a part of AI, and deep learning is a type of machine learning. Figure 5 shows how these three ideas are related to each other. Figure 2(b) shows that AI is a broad field with many different subdomains. However, AI's recent rise in popularity is largely due to how well machine learning, especially deep learning, works. So, this entry will talk about these two areas of AI: Machine Learning (ML) and Deep Learning (DL), Figure 2 (a).

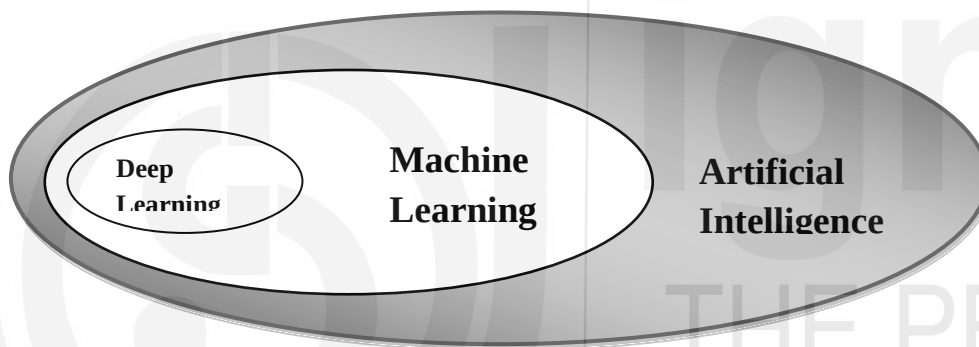


Fig 2(a): AI, ML, DL

SUB DOMAINS OF ARTIFICIAL INTELLIGENCE

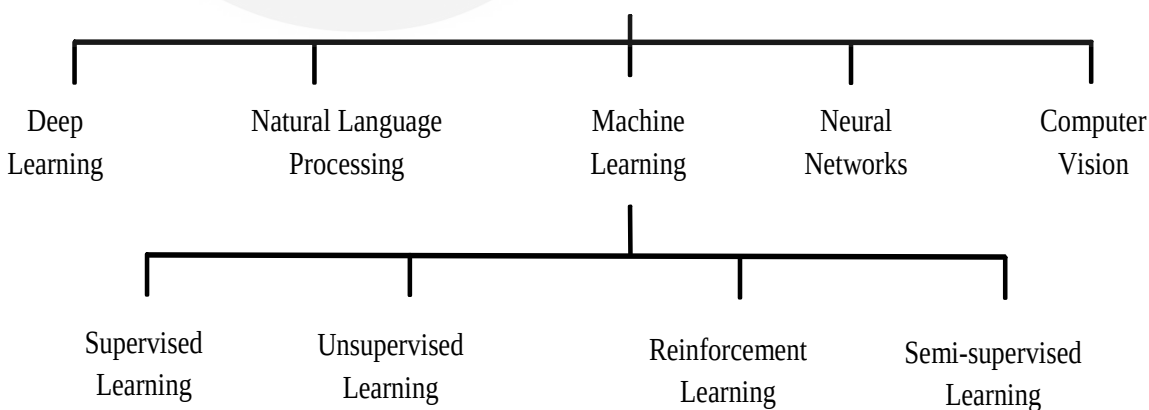


Figure 2(b): Various Sub Domains of Artificial Intelligence

To make a system that is artificially intelligent, you have to carefully do Reverse-Engineering of human traits and machine abilities. Also, to understand how an AI system really works, you have to get a good grasp of the different

parts of AI and how they can be used in different industries or industrial fields.

Introduction to Machine Learning (ML)

Machine learning is a branch of artificial intelligence (AI). It explains one of the most important ideas in AI, which has to do with learning through experience and not through being taught. One of the most recent advances in AI, this way of learning is made possible by applying machine learning to very large data sets. Machine learning algorithms find patterns and learn how to make predictions and recommendations by using data and experiences instead of explicit programming instructions. The algorithms also change as they get new data and learn from their experiences. This makes them more effective over time.

Algorithms for machine learning are based on ways that people learn from their experiences. This means that they are programmed to learn from what they do and get better at what they do. They don't need to be told what to do to get the desired results. They are set up so that people can learn by looking at the data sets they can access and comparing what they see to examples of the final results. They also look for patterns in the output and try to figure out how to use the different parts to get the output they want.

ML shows a machine how to draw conclusions and make decisions based on what it has learned in the past. It looks for patterns and analyzes past data to figure out what these patterns mean so that a possible conclusion can be reached without the need for human experience. Businesses save time and make better decisions by using automation to evaluate data and come to conclusions.

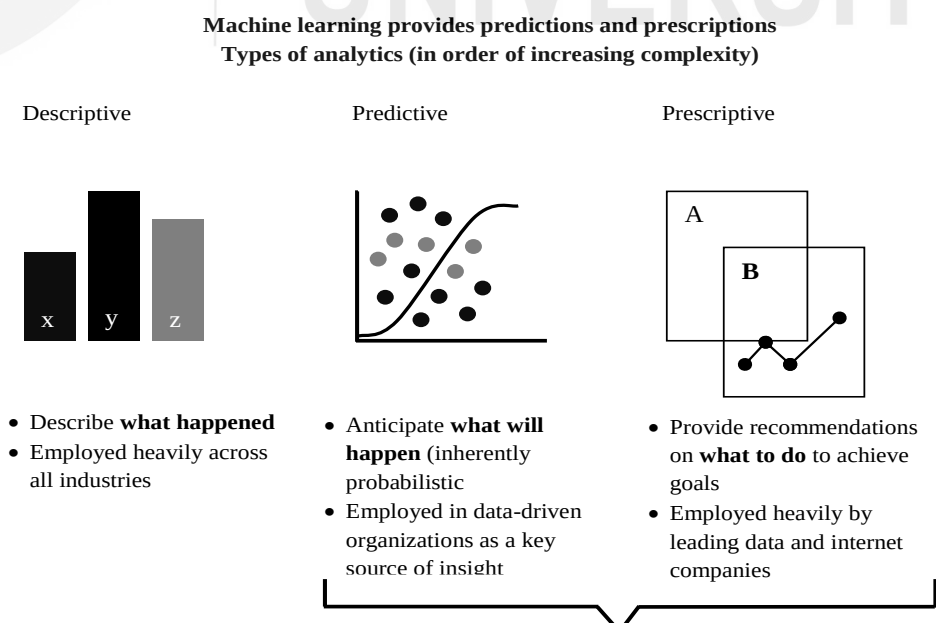


Figure 3: Machine Learning: Descriptive, Predictive and Prescriptive Analytics

Deep Learning is a subfield of machine learning that focuses on algorithms called "Artificial Neural Networks" (ANN) that are based on how the brain is built and how it works. Deep Learning is a type of machine learning that can handle a wider range of data sources, needs less pre-processing of data, and often gives more accurate results than traditional machine learning methods.

A neural network is made up of layers of software-based calculators called "neurons" that are linked together. This neural network can take in a huge amount of data and process it through many layers. At each layer, the network learns more complex features of the data. The network can then decide what to do with the data, find out if it was right, and use what it has learned to decide what to do with new data. Deep learning is a way of programming computers that uses the way neural networks work to teach computers to do things that humans do naturally. So, Deep Learning is a way to teach a computer model to run classification algorithms based on an image, text, or sound. Once a neural network knows what an object looks like, it can spot that object in a new picture.

Deep learning is becoming more popular because its models can get better results. It uses large sets of labeled data and neural network architectures to train the models.

Check Your Progress 3

7. Compare Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).

.....

.....

.....

8. Compare Descriptive, Predictive and Prescriptive analytics performed under Machine Learning.

.....

.....

.....

1.7 APPLICATION AREAS OF ARTIFICIAL INTELLIGENCE SYSTEMS

Artificial intelligence is the most important factor in the transformation of economies straight from the ground up, and it is contributing as an efficient alternative. It has a lot of potential to perform optimization in any industry, whether it is smart cities, the health sector, agriculture, or any other prospective sector of relevance, and below we have included a few of the

systems in which AI is functioning as the major source of competitive advantage:

- a) **Healthcare:** The application of AI in healthcare can help address issues of high barriers to access to healthcare facilities, particularly in rural areas that suffer from poor connectivity and a limited supply of healthcare professionals. This is especially true in areas where the supply of healthcare professionals is limited. The deployment of use cases such as AI-driven diagnostics, personalised treatment, early diagnosis of potential pandemics, and imaging diagnostics, amongst others, is one way to accomplish this goal.

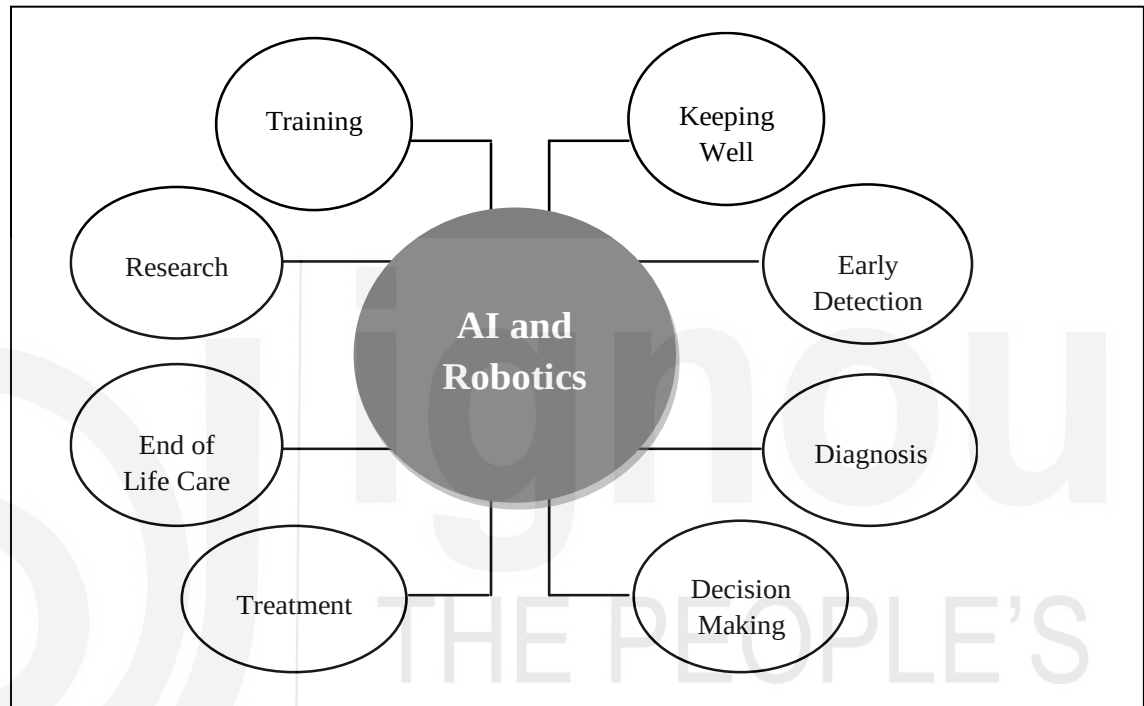


Figure 4: Potential Use of AI in Health Care

- b) **Agriculture:** AI has the potential to bring in a food revolution while simultaneously satisfying the ever-increasing need for food (global need to produce 50 percent more food and cater to an additional 2 billion people by 2050 as compared to today). It also has the ability to resolve issues such as inadequate demand prediction, a lack of secure irrigation, and the abuse or misuse of pesticides and fertilizers. These are only some of the problems that it could solve. The increase of crop output through real-time advising is one example of a use case. Other use cases include the advanced detection of pest infestations and the forecast of crop prices to advise sowing methods.

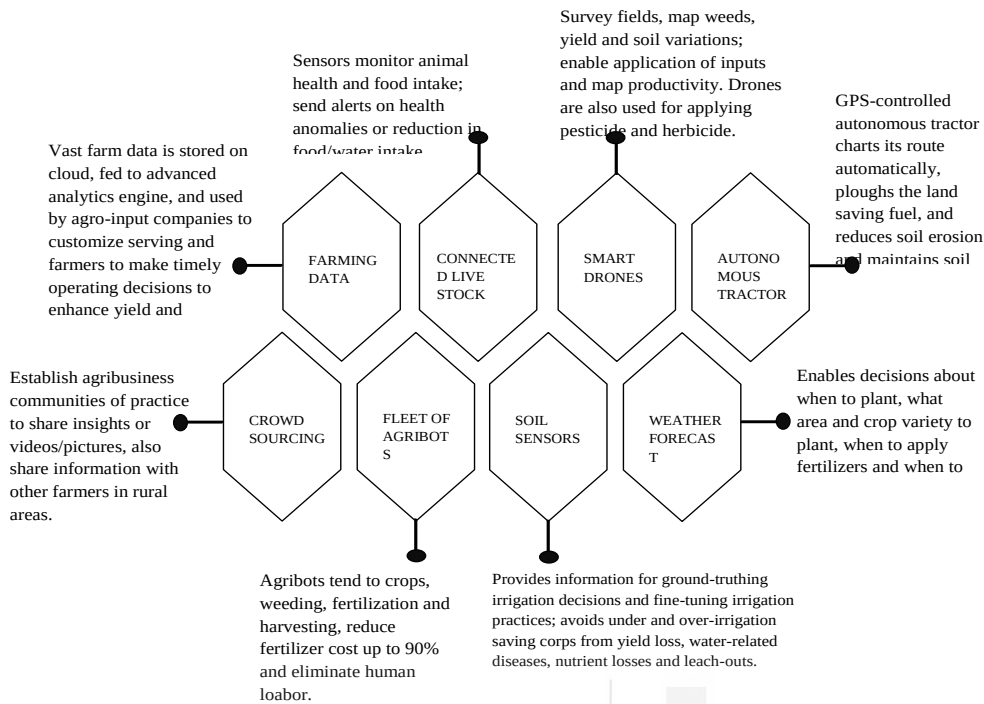


Figure 5: AI for Precision Farming

All of the stages of the agricultural value chain indicated above in Figure 5 have the potential for the application of artificial intelligence and other associated technologies to have an impact on the levels of production and efficiency at those stages.

c) Smart Mobility, including Transports and Logistics: Autonomous fleets for ride sharing, semi-autonomous features such as driver assistance, and predictive engine monitoring and maintenance are all possible use cases for smart mobility, which includes transportation and logistics. Other areas where AI can have a positive impact include self-driving trucks and delivery, as well as better traffic control.

d) Retail: The retail industry was one of the first to use AI solutions. For example, personalised suggestions, browsing based on user preferences, and image-based product search have all been used to improve the user experience. Other use cases include predicting what customers will want, keeping track of inventory better, and managing deliveries more efficiently.

e) Manufacturing: AI-based solutions are expected to help the manufacturing industry the most. This will make possible the "Factory of the Future" by allowing flexible and adaptable technical systems to automate processes and machinery that can respond to new or unexpected situations by making smart decisions. Impact areas include engineering (AI for R&D), supply chain management (predicting demand), production (AI can cut costs and increase efficiency), maintenance (predictive maintenance and better use of assets), quality assurance (e.g., vision systems with machine learning algorithms to find flaws and differences in product features), and in-plant logistics and warehousing.

f) Energy: In the energy sector, possible use cases include modelling and forecasting the energy system to make it less unpredictable and make

balancing and using power more efficient. In renewable energy systems, AI can help store energy through smart meters and intelligent grids. It can also make photovoltaic energy more reliable and less expensive. AI could also be used to predict maintenance of grid infrastructure, just like it is in manufacturing.

g) Smart Cities: Integrating AI into newly built smart cities and infrastructure could also help meet the needs of a population that is moving to cities quickly and improve the quality of life for those people. Some possible use cases include controlling traffic to reduce traffic jams and managing crowds better to improve security.

h) Education and Skilling: Quality and access problems in the education sector might be fixed by AI. Possible uses include adding to and improving the learning experience through personalized learning, automating and speeding up administrative tasks, and predicting when a student needs help to keep them from dropping out or to suggest vocational training.

i) Financial industry: The financial industry also uses AI. For example, it helps the fraud department of a bank find and flag suspicious banking and finance activities like unusual debit card use and large account deposits. AI is also used to make trading easier and more efficient. This is done by making it easier to figure out how many securities are being bought and sold and how much they cost.

Top Used Applications of Artificial Intelligence

- Tools and checkers for plagiarism
- Recognizing faces
- Putting an AI autopilot on commercial planes
- Applications for sharing rides (E.g., Uber, Lyft)
- E-mail spam filters; voice-to-text features; search suggestions
- Google's predictions based on AI (E.g., Google Maps)
- Protecting against and stopping fraud.
- Smart personal assistants (E.g., Siri, Alexa)

There are various ways to use artificial intelligence. The technology can be used in different industries and sectors, but the adoption of AI by different sectors has been affected by technical and regulatory challenges, with the biggest factor being how it will affect business.

1.8 INTELLIGENT AGENTS

An agent may be thought of as an entity that acts, generally on behalf of someone else. More precisely, an agent is an entity that perceives its environment through sensors and acts on the environment through actuators. Some experts in the field require an agent to be autonomous and goal-directed also.

A percept may be thought of as an input to the agent through its sensors, over a unit of time, sufficient enough to make some sense from the input.

Percept sequence is a sequence of percepts, generally long enough to allow the agent to initiate some action.

In order to further have an idea about what a computer agent is, let us consider one of the first definitions of agent, which was coined by John McCarthy and his friends at MIT.

A software agent is a system that, when given a goal to be achieved, could carry out the details of the appropriate (computer) operations, and further, in case it gets stuck, it can ask for advice and can receive it from humans, may even evaluate the appropriateness of the advice, and then act suitably.

Essentially, a computer agent is a computer software that has the following attributes:

- (i) It has autonomous control, i.e., it operates under its own control.
- (ii) It is perceptive, i.e., it is capable of perceiving its own environment.
- (iii) It persists over a long period of time.
- (iv) It is adaptive to changes in the environment.
- (v) It is capable of taking over others' goals.

As the concept of Intelligent Agents is relatively new, different pioneers and other experts have been conceiving and using the term in different ways. There are two distinct but related approaches for defining an agent. The first approach treats an agent as an ascription, i.e., the perception of a person (which includes expectations and points of view), whereas the other approach defines an agent on the basis of the description of the properties that the agent to be designed is expected to possess.

Let us first discuss the definition of an agent according to the first approach. Among the people who consider an agent as an ascription, a popular slogan is "Agent is that agent does". In an everyday context, an agent is expected to act on behalf of someone to carry out a particular task, which has been delegated to it. But to perform its task successfully, the agent must have knowledge about the domain in which it is operating and also about the properties of its current user in question. In the course of normal life, we hire different agents for different jobs based on the required expertise for each job. Similarly, a non-human intelligent agent is also embedded with the required expertise of the domain as per the requirements of the job under consideration. For example, a football-playing agent would be different from an email-managing agent, although both will have the common attribute of modelling their user.

According to the second approach, an agent is defined as an entity that functions continuously and autonomously, in a particular environment, which may have other agents also. By continuity and autonomy of an agent, it is meant that the agent must be able to carry out its job in a flexible and intelligent fashion, and further is expected to adapt to the changes in its environment without requiring constant human guidance or intervention. Ideally, an agent that functions continuously in an environment over a long period of time would also learn from its experience. In addition, we expect an agent that lives in a multi-agent environment to be able to communicate and cooperate with them, and perhaps move from place to place in doing so.

According to the second approach to defining an agent, an agent is supposed to possess some or all of the following properties:

- **Reactivity:** The ability of sensing the environment and then act accordingly.
- **Autonomy:** The ability to move towards its goal, changing its moves or strategy, if required, without much human intervention.
- **Communicating ability:** The ability to communicate with other agents and humans.
- **Ability to coexist by cooperating:** The ability to work in a multi-agent environment to achieve a common goal.
- **Ability to adapt to a new situation:** The ability to learn, change, and adapt to the situations in the world around it.
- **Ability to draw inferences:** The ability to infer or conclude facts, which may be useful, but are not available directly.
- **Temporal continuity:** The ability to work over long periods of time.
- **Personality:** The ability to impersonate or simulate someone, on whose behalf the agent is acting.
- **Mobility:** The ability to move from one environment to another.

Task environments or problem environments are the environments that include all the elements involved in the problems for which agents are thought of as solutions. Task environments will vary with every new task or problem for which an agent is being designed. Specifying the task environment is a long process that involves looking at different measures or parameters. Next, we discuss a standard set of measures or parameters for specifying a task environment under the heading PEAS.

PEAS (Performance, Environment, Actuators, Sensors)

For designing an agent, the first requirement is to specify the task environment to the maximum extent possible. The task environment for an agent to solve one type of problems, may be described by the four major parameters namely, performance (which is actually the expected performance), environment (i.e., the world around the agent), actuators (which include entities through which the agent may perform actions) and sensors (which describes the different entities through which the agent will gather information about the environment).

The four parameters may be collectively called PEAS. We explain these parameters further through an example of an automated agent, which we will preferably call an automated public road transport driver. This is a much more complex agent than the simple boundary following robot, which we have already discussed.

Example (An Automated Public Road Transport Driver Agent)

We describe the task environment of the agent on the basis of PEAS.

Performance Measures: Some of the performance measures that can easily be perceived by an automated public road transport driver would be:

- Maximizing the safety of passengers
- Maximizing the comfort of passengers

- Ability to reach the correct destination
- Ability to minimize the time to reach the destination, obeying traffic rules
- Causing minimum discomfort or disturbance to other agents
- Minimizing costs, etc.

Environment (or the world around the agent). We must remember that the environment or the world around the agent is extremely uncertain or open ended. There are unlimited combinations of possibilities of the environmental situations that such an agent could face. Let us enumerate some of the possibilities or circumstances that an agent might face:

- Variety of roads, e.g., from 12-lane expressways, freeways to dusty rural bumpy roads; different road rules, including the ones requiring left-hand drive in some parts of the world and right-hand drive in other parts.
- The degree of knowledge of various places through which and to which driving is to be done.
- Various kinds of passengers, including high cultured to almost ruffians, etc.
- All kinds of other traffic, possibly including heavy vehicles, ultra-modern cars, three-wheelers, and even bullock carts.

Actuators: These include the following:

- Handling steering wheel, brakes, gears, and accelerator
- Understanding the display screen
- A device or devices for all communication required

Sensors: The agent acting as an automated public road transport driver must have some way of sensing the world around it, i.e., the traffic around it, the distance between the automobile and the automobiles ahead of it, and its speed, the speeds of neighboring vehicles, the condition of the road, any turn ahead, etc. It may use sensors like odometer, speedometer, sensors telling the different parameters of the engine, and Global Positioning System (GPS) to understand its current location and the path ahead. Also, there should be some sort of sensors to calculate its distance from other vehicles, etc.

We must remember that the agent example, the automated public road transport driver, which we have considered above, is quite difficult to implement. However, many other agents operate in comparatively simpler and less dynamic environments, e.g., a game playing robot, an assembly line robot control, and an image processing agent, etc.

In respect of the design and development of intelligent agents, with the passage of time, the momentum seems to have shifted from hardware to software, the latter being thought of as a major source of intelligence. But, obviously, some sort of hardware is essentially needed as a home for the intelligent agent.

There are two parts of an agent or its structure:

- A (hardware) device with sensors and actuators in which the agent will reside, called the *architecture* of the agent.
- An agent program that will convert or map the percepts into actions.

Also, the agent program and its architecture are related in the sense that for a different agent architecture, a different type of agent program is required and vice versa. For example, in the case of a *boundary following robot*, if the robot does not have the capability of sensing adjacent cells to the right, then the agent program for the robot has to be changed.

Next, we discuss different categories of agents, which are differentiated from each other on the basis of their agent programs. The capability to write efficient agent programs is the key to success for developing efficient rational agents. Although the table-driven approach (in which an agent acts on the basis of the set of all possible percepts by storing these percepts in tables) to design agents is possible, the approach of developing equivalent agent programs is found to be much more efficient.

Next, we discuss some of the general categories of agents based on their programs. Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better actions over time. These are given below:

- *SR (Simple Reflex) agents*
- *Model-based reflex agents*
- *Goal-based agents*
- *Utility-based agents*
- *Stimulus-Response Agents*
- *Learning agents*

SR (Simple Reflex) agents: These are the agents or machines that have no internal state (i.e., they don't remember anything) and simply react to the current percepts in their environments. An interesting set of agents can be built, the behaviour of the agents in which can be captured in the form of a simple set of functions of their sensory inputs. One of the earliest implemented agents of this category was called *Machina Speculatrix*. This was a device with wheels, a motor, photo cells, and vacuum tubes, and was designed to move in the direction of light of less intensity and to avoid the direction of the bright light. **A boundary following robot is also an SR agent.** For an automobile-driving agent, some aspects of its behavior, like applying brakes immediately on observing either the vehicle immediately ahead applying brakes or a human being coming just in front of the automobile suddenly, show the simple reflex capability of the agent. Such a simple reflex action in the agent program of the agent can be implemented with the help of simple condition-action rules.

For example: **IF** *a human being comes in front of the automobile suddenly*
THEN *apply brakes immediately.*

Although implementation of SR agents is simple, **on the negative side**, this type of agents has very limited intelligence because ***they do not store or remember anything.*** As a consequence, they cannot make use of any previous

experience. In summary, they do not learn. Also, **they are capable of operating correctly only if the environment is fully observable.**

Model-Based Reflex agents: Simple Reflex agents are not capable of handling task environments that are not fully observable. In order to handle such environments properly, in addition to reflex capabilities, the agent should maintain some sort of internal state in the form of a function of the sequence of percepts recovered up to the time of action by the agent. Using the percept sequence, the internal state is determined in such a manner that it reflects some of the aspects of the unobservable environment. Further, in order to reflect the unobserved environment properly, the agent is expected to have a model of the task environment encoded in the agent's program, where the model has the knowledge about–

- (i) the process by which the task environment evolves independent of the agent and
- (ii) the effects of the actions of the agent have on the environment.

Thus, in order to handle the partial observability of the environment, the agent should have a model of the task environment in addition to reflex capabilities. Such agents are called **Model-based Reflex Agents**

Goal-Based Agents: In order to design an appropriate agent for a particular type of task, we know that the nature of the task environment plays an important role. Also, the complexity of the agent should be minimal and just sufficient to handle the task in a particular environment. In this regard, first we discussed the simplest type of agents, viz., Simple Reflex Agents. The action of this type of agent is decided by the current precept only. Next, we discussed the Model-Based Reflex Agents, for which an action is decided by taking into consideration not only the latest precept, but the whole precept history summarized in the form of an internal state. Also, the action for this type of agent is decided by taking into consideration the knowledge of the task environment, represented by a model of the environment and encoded into the agent's program. However, in respect of a number of tasks, even this much knowledge may not be sufficient for appropriate action. For example, when we are going from city A to city B, in order to take appropriate action, it is not enough to know the summary of actions and the path that has taken us to some city C between A and B. We also have to remember the goal of reaching city B.

Goal-based agents are driven by the goal they want to achieve, i.e., their actions are based on the information regarding their goal, in addition to, of course, other information in the current state. This goal information is also a part of the current state description, and it describes everything that is desirable to achieve the goal. As mentioned earlier, an example of a goal-based agent is an agent that is required to find the path to reach a city. In such a case, if the agent is *an automobile driver agent*, and if the road splits ahead into two roads, then the agent has to decide which way to go to achieve its goal of reaching its destination. Further, if there is a crossing ahead, then the agent has to decide whether to go straight, to go to the left, or to go to the right. In order to achieve its goal, the agent needs some information regarding the goal, which describes the desirable events and situations to reach the goal. The agent program would

then use this goal information to decide the set of actions to take in order to reach its goal.

Another desirable capability that a good goal-based agent should have is that if an agent finds that a part of the sequence of the previous steps has taken the agent away from its goal, then it should be able to retract and start its actions from a point that may take the agent toward the goal.

In order to take appropriate action, the decision-making process in goal-based agents may be simple or quite complex, depending on the problem. Also, **the decision-making required by the agents of this kind needs some sort of looking into the future.** For example, it may analyze the possible outcome of a particular action before it actually performs that action. In other words, we can say that ***the agent would perform some sort of reasoning of if-then-else type***, e.g., an automobile driver agent having one of its goals as not to hit any vehicle in front of it, when finds the vehicle immediately ahead of it slowing down may not apply brakes with full force and instead may apply brakes slowly so that the vehicles following it may not hit it.

As the goal-based agents may have to reason before they take an action, these agents might be slower than other types of agents, but will be more flexible in taking actions as their decisions are based on the acquired knowledge, which can also be modified. Hence, as compared to SR agents, which may require rewriting of all the condition-action rules in case of a change in the environment, the goal-based agents can adapt easily when there is any change in their goal.

Utility-Based Agents: A goal-based agent's success or failure is judged in terms of its capability for achieving or not achieving its goal. A goal-based agent, for a given pair of environment state and possible input, only knows whether the pair will lead to the goal state or not. Such an agent will not be able to decide in which direction to proceed when there is more than one conflicting goal. Also, in a goal-based agent, there is no concept of partial success or somewhat satisfactory success. Further, if there is more than one method of achieving a goal, then no mechanism is incorporated in a Goal-based agent to choose or find the method that is faster and more efficient method out of the available ones, to reach its goal.

A more general way to judge the success or happiness of an agent may be through assigning to each state a number as an approximate measure of its success in reaching the goal from the state. In case the agent is embedded with such a capability of assigning such numbers to states, then it can choose, out of the reachable states in the next move, the state with the highest assigned number, out of the numbers assigned to various reachable states, indicating possibly the best chance of reaching the goal.

It will allow the goal to be achieved more efficiently. Such an agent will be more useful, i.e., will have more utility. A utility-based agent uses a **utility function**, which maps each of the world states of the agent to some degree of success. If it is possible to define the utility function accurately, then the agent will be able to reach the goal quite efficiently. Also, a utility-based agent is *able to make decisions in case of conflicting goals*, generally choosing the goal with a higher success rating or value. Further, in environments with multiple goals, the utility-based agent quite likely chooses the goal with the least cost or the highest utility goal out of multiple goals.

Stimulus-Response Agents: A stimulus-response agent (or a reactive agent) takes input from the world through sensors, and then takes action based on those inputs through actuators. Between the stimulus and response, there is a processing unit that can be arbitrarily complex. An example of such an agent controls a vehicle in a racing game: the agent “looks” at the road and nearby vehicles, and then decides how much to turn and brake. Such Agents (Stimulus-Response Agents are the Reactive agents) represent a special category of agents, which do not possess internal, symbolic models of their environments; instead, they act/respond in a stimulus-response manner to the present state of the environment in which they are embedded. These agents are relatively simple, and they interact with other agents in basic ways. Nevertheless, complex patterns of behavior emerged from the interactions when the ensemble of agents is viewed globally

Learning Agents: It is not possible to encode all the knowledge in advance, required by a rational agent for optimal performance during its lifetime. This is especially true of real-life, and not just theoretical, environments. These environments are **dynamic** in the sense that the environmental conditions change, not only due to the actions of the agents under consideration, but also due to other environmental factors. For example, all of a sudden, a pedestrian comes just in front of the moving vehicle, even when there is a green signal for the vehicle. In a multi-agent environment, all the possible decisions and actions an agent is required to take are generally unpredictable in view of the decisions taken and actions performed simultaneously by other agents. Hence, **the ability of an agent to succeed in an uncertain and unknown environment depends on its learning capability**, i.e., its capability to change its knowledge of the environment. For an agent with learning capability, some initial knowledge is coded in the agent program, and after the agent starts operating, it learns from its actions the evolving environment, the actions of its competitors or adversaries, etc., so as to improve its performance in an ever-changing environment. If an approximate learning component is incorporated in the agent, then the knowledge of the agent gradually increases after each action, starting from its initial knowledge, which was manually coded into it at the start.

Conceptually, the learning agent consists of four components:

- (i) **Learning Component:** It is the component of the agent, which based on the percepts and the feedback from the environment, gradually improves the performance of the agent.
- (ii) **Performance Component:** It is the component from which all actions originate on the basis of external percepts and the knowledge provided by the learning component.

The design of the learning component and the design of the performance element are very much related to each other because a learning component is of no use unless the performance component can be designed to convert the newly acquired knowledge into more useful actions.

- (iii) **Critic Component:** This component finds out how well the agent is doing with respect to a certain fixed performance standard, and it is also responsible for any future modifications in the performance component.

The critic is necessary to judge the agent's success with respect to the chosen performance standard, especially in a dynamic environment. For example, in order to check whether a certain job is accomplished, the critic will not depend on external percepts only, but it will also compare the current state to the state that indicates the completion of that task.

(iv) Problem Generator Component: This component is responsible for suggesting actions (some of which may not be optimal) in order to gain some fresh and innovative experiences. Thus, ***this component allows the agent to experiment a little*** by traversing sometimes uncharted territories by choosing some new and suboptimal actions. This may be useful because the actions that may seem suboptimal in the short run may turn out to be much better in the long run.

In the case of *an automobile driver agent*, this agent would be of little use if it did not have a learning capability, as the environment in which it has to operate is totally dynamic and unpredictable in nature. **Once the automobile driver agent starts operating, it keeps on learning from its experiences, both positive and negative.** If faced with a totally new and previously unknown situation, e.g., encountering a vehicle coming from the opposite direction on a one-way road, the problem generator component of the driver agent might suggest some innovative action to tackle this new situation. Moreover, the learning becomes more difficult in the case of an automobile driver agent, because the environment is only partially observable.

Different Forms of Learning in Agents: The purpose of embedding learning capability in an agent is that it should not depend totally on the knowledge initially encoded in it and on the external percepts for its actions. The agent learns by evaluating its own decisions and/or making observations of new situations it encounters in the ever-changing environment.

There may be various criteria for developing learning taxonomies. The criteria may be based on –

- The type of knowledge learnt, e.g., concepts, problem-solving, or game playing,
- The type of representation used, e.g., predicate calculus, rules, or frames,
- The area of application, e.g., medical diagnosis, scheduling, or prediction.

☞ Check Your Progress 4

9. What are Intelligent agents in AI? Briefly discuss the properties of Agents.

.....
.....
.....

10. What are Task environments? Briefly discuss the standard set of measures or parameters for specifying a task environment under the heading PEAS.

.....
.....

1.9 SUMMARY

In this unit, we learned about the difference between knowledge and intelligence and also pointed out the meaning of Artificial Intelligence (AI), along with the application of AI systems in various fields. The unit also covered the historical development of the field of AI systems. Along with the development of AI as a discipline, the need for the classification of AI systems was felt, and hence, the unit discussed the classification of AI systems in detail. Further, the unit discussed the concepts of Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL). Finally, the unit discussed the components of Intelligence, which was extended for the understanding of the concepts of Intelligent Agents, with special emphasis on Stimulus - Response Agents

1.10 SOLUTIONS/ANSWERS

Check Your Progress 1

Q1 How does knowledge differ from intelligence? What do you understand by the term Artificial Intelligence (AI)? List the various technologies and their corresponding illustrative solutions.

Sol- Refer to section 1.3

Q2 Classify AI based on the functionalities of AI

Sol- Refer to section 1.3

Q3 Compare ANI, AGI, and ASI in the context of AI

Sol- Refer to section 1.3

Check Your Progress 2

Q4 Briefly discuss the various components of intelligence

Sol – Refer to Section 1.5

Q5: How do we measure if Artificial Intelligence is making a machine to behave or act or perform like a human being or not?

Sol – Refer to Section 1.6

Q6 What is the Turing Test? What is the criticism of the Turing Test?

Sol – Refer to Section 1.6

Check Your Progress 3

Q7 Compare Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).

Sol – Refer to Section 1.7

Q8 Compare Descriptive, Predictive, and Prescriptive analytics performed under Machine Learning.

Sol – Refer to Section 1.7

☞ Check Your Progress 4

Q9 What are Intelligent agents in AI? Briefly discuss the properties of Agents.

Sol – Refer to Section 1.9

Q10 What are Task environments? Briefly discuss the standard set of measures or parameters for specifying a task environment under the heading PEAS.

Sol – Refer to Section 1.9

1.11 FURTHER READINGS

1. Ela Kumar, “Artificial Intelligence”, IK International Publications
2. E. Rich and K. Knight, “Artificial intelligence”, Tata McGraw-Hill Publications
3. N.J. Nilsson, “Principles of AI”, Narosa Publ. House Publications
4. John J. Craig, “Introduction to Robotics,” Addison-Wesley publication
5. D.W. Patterson, “Introduction to AI and Expert Systems,” Pearson publication

THE PEOPLE'S
UNIVERSITY

UNIT 2 PROBLEM SOLVING USING SEARCH

Structure	Page No
2.0 Introduction	43
2.1 Objectives	46
2.2 Introduction to State Space Search	46
2.2.1 Problem Formulation	47
2.2.2 Structure of a State space	47
2.2.3 Problem solution of State space	47
2.3.4 Searching for solution in state spaces formulated	48
2.3 Formulation of 8 puzzle problem from AI perspective	50
2.4 N-queen's problem- Formulation and Solution	52
2.4.1 Formulation of 8 Queen's problem	55
2.4.2 State space tree for 4-Queen's problem	55
2.4.3 Backtracking approach to solve N Queen's problem	57
2.5 Two agent search: Adversarial search	60
2.5.1 Elements of Game playing search	60
2.5.2 Types of algorithms in Adversarial search	62
2.6 Minimax search strategy	63
2.6.1 Minimax algorithm	63
2.6.2 Working of Minimax algorithm	64
2.6.3 Properties of Minimax algorithm	67
2.6.4 Advantages and Disadvantages of Minimax search	67
2.7 Alpha-Beta Pruning algorithm	68
2.7.1 Working of Alpha-Beta pruning	68
2.7.2 Move Ordering of Alpha-Beta pruning	77
2.8 Summary	80
2.9 Solutions/Answers	82
2.10 Further readings	90

2.0 INTRODUCTION

Many AI-based applications need to figure out how to solve problems. In the world, there are two types of problems. First, the problem which can be solved by using a deterministic procedure, and the success is guaranteed. But most real-world problems can be solved only by searching for a solution. AI is concerned with these second types of problems solving.

To build a system to solve a problem, we need to

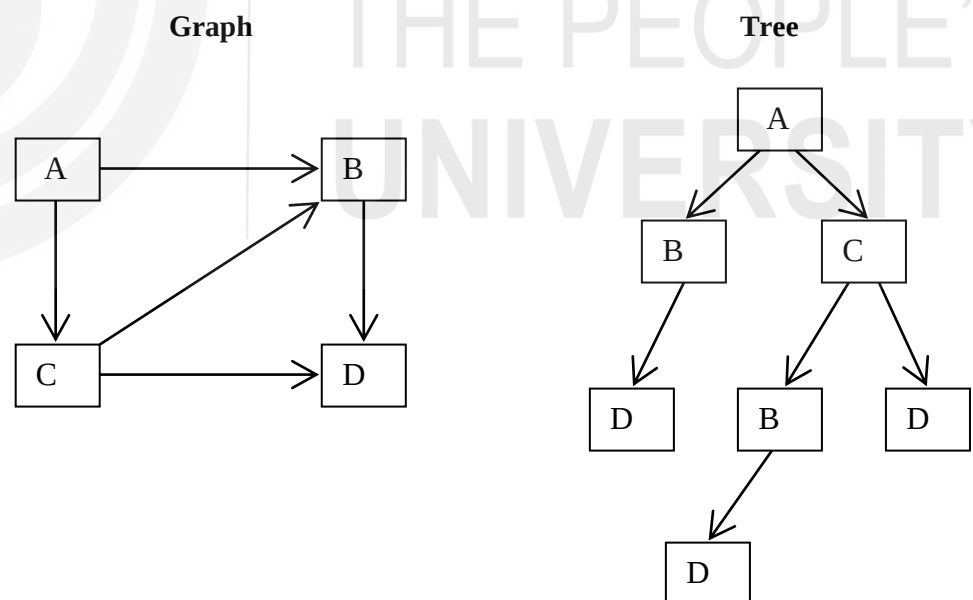
- **Define the problem precisely** - find initial and final configuration for an acceptable solution to the problem.
- **Analyse the problem** - find few important features that may have an impact on the appropriateness of various possible techniques for solving the problem
- **Isolate and represent** task knowledge necessary to solve the problem
- **Choose the best problem-solving technique(s)** and apply it to the particular problem.

To provide a formal description of a problem, we need to do the following:

- Define a **state space** that contains all the possible configurations of the relevant objects.
- Specify one or more states that describe possible situations, from which the problem-solving process may start. These states are called **initial states**.
- Specify one or more than one **goal states**.
- Defining a *set of rules* for the actions (operators) that can be taken.

The problem can then be solved by using the rules, in combination with an appropriate **control strategy**, to move through the problem space until a path from an **initial state** to a **goal state** is found. This process is known as '**search**'. Thus, search is fundamental to the problem-solving process. Search is a general mechanism that can be used when a more direct method is not known. Search provides the framework into which more direct methods for solving subparts of a problem can be embedded. **All AI problems are formulated as search problems.**

A **problem space** is represented by a directed graph, where *nodes* represent *search states*, and *paths* represent the operators that change the state. To simplify search algorithms, it is often convenient to logically and programmatically represent a problem space as a **tree**. A tree usually decreases the complexity of a search at a cost. Here, the cost is due to duplicating some nodes on the tree that were linked numerous times in the graph, e.g., node B and node D shown in the example below.



A **tree** is a graph in which any two vertices are connected by exactly one path. Alternatively, any connected graph with no cycles is a tree.

Before an AI problem can be solved, it must be represented as a **state space**. Here, **state** means representation of elements at a given moment. Among all possible states, there are two special states called the **initial state** (the start point) and the **final state** (the goal state). A **successor function (a set of operators)** is used to change the state. It is used to move from one state to another. A state space is the set of all states reachable from the initial state. A

state space essentially consists of a set of nodes representing each state of the problem, arcs between nodes representing the legal moves from one state to another, an initial state, and a goal state. Each state space takes the form of a tree or a graph. In AI, a wide range of problems can be formulated as a search problem. The process of searching means a sequence of actions that takes you from an initial state to a goal state, as shown in the following figure 1.

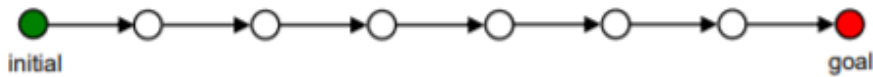


Fig 1: A sequence of actions in a search space from the initial to the goal state.

So, the state space is one of the methods to represent the problem in AI. A set of all possible states reachable from the initial state by taking some sequence of actions (using some operator) for a given problem is known as the **state space** of the problem. A state space represents a problem in terms of **states** and **operators** that change states”.

In this unit, we examine the concept of a **state space** and the different **search processes** that can be used to explore the search space in order to find a solution (Goal) state. In the worst case, search explores all possible paths between the *initial state* and the *goal state*.

For a better understanding of these definitions described above, consider the following 8-puzzle problem:

Eight-Puzzle problem formulation from AI perspective:

initial state: some configuration of the 8-tiles on a 9-cell board.

operators (Action): It’s easier if we focus on the blank. There are 4 operators that are: “Moving the blank”: UP, DOWN, LEFT, and RIGHT.

Uniformed/Blind 3	1	2		3		2
4		5		4	1	5
6	7	8	the blank up	6	7	8

Goal state: Tiles in a specific order

	1	2
3	4	5
6	7	8

Solution: Optimal sequence of operators (Actions)

Path costs:

Cost of each action = 1

Cost of a sequence of actions = the number of actions

A state space representation of the 8-puzzle problem is shown in figure-2

8-Puzzle

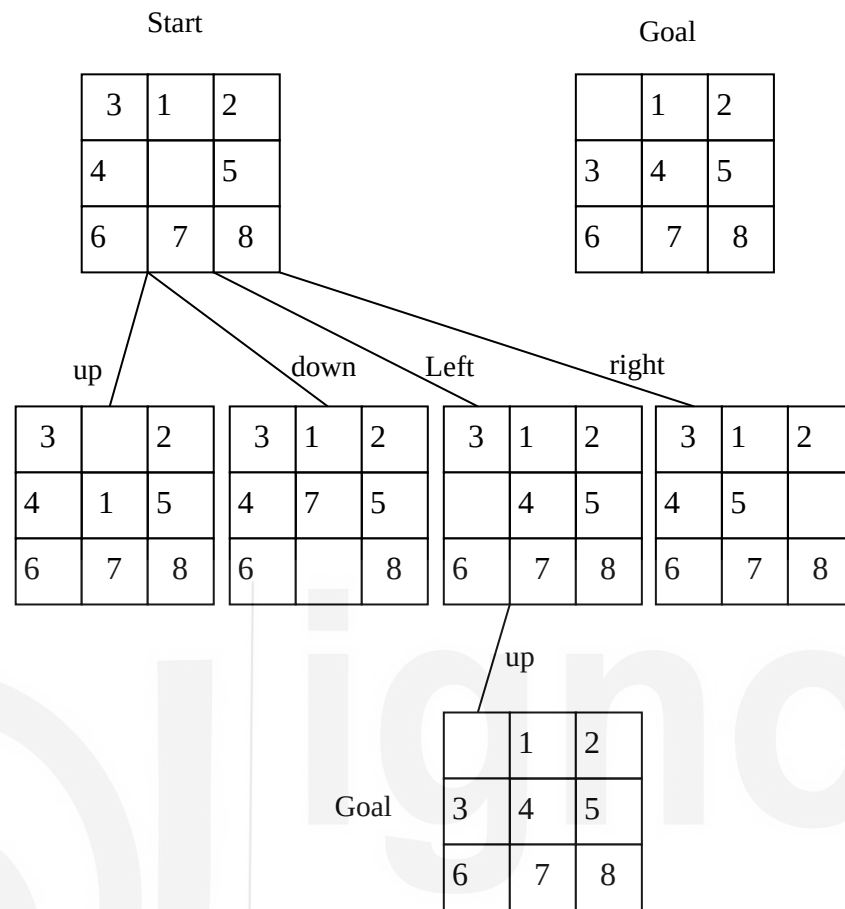


Fig. 2: A state space representation of the 8-puzzle problem generated by “Move Blank” operator

2.1 OBJECTIVES

After studying this unit, you should be able to:

- Understand the state space search
- Formulate the problems in the form of a state space
- Understand how implicit state space can be unfolded during search
- Explain the state space search representation for the Water-Jug, 8-puzzle, and N-Queens problem.
- Solve N Queen’s Problem using Backtracking approach
- Understand adversarial search (two-agent search)
- Differentiate between the Minimax and Alpha-beta pruning search algorithms.

2.2 INTRODUCTION TO STATE SPACE SEARCH

It is necessary to represent an AI problem in the form of a state space before it can be solved. A state space is the set of all states reachable from the initial state. A state space forms a graph in which the nodes are states, and the arcs

between nodes are actions. In the state space, a path is a sequence of states connected by a sequence of actions.

2.2.1 Structure of a state space:

The structures of state space are trees and graphs. A tree has one and only one path from any point to any other point. A graph consists of a set of nodes (vertices) and a set of edges (arcs). Arcs establish relationships (connections) between the nodes, i.e., a graph has several paths to a given node. **Operators** are directed arcs between nodes.

The method of solving problems through AI involves the process of defining the search space, deciding start and goal states, and then finding the path from the start state to the goal state through the search space.

The search process explores the state space. In the worst case, the search explores all possible paths between the initial state and the goal state.

2.2.2 Problem Solution:

In a state space, a **solution is a path** from the initial state to a goal state or sometimes just a goal state. A numeric cost is assigned to each path. It also gives the cost of applying the operators to the states. A path cost function is used to measure the quality of a solution, and out of all possible solutions, an optimal solution has the lowest path cost. The importance of cost depends on the problem and the type of solution required.

2.2.3 Problem formulation:

Many problems can be represented as a **state space**. The state space of a problem includes: an *initial state*, one or more *goal states*, a set of *state transition operators* (or a **set of production rules**), used to change the current state to another state. This is also known as **actions**. A **control strategy** is used that specifies the order in which the rules will be applied. For example, Depth-first search (DFS), Breadth-first search (BFS), etc. It helps to find the goal state or a path to the goal state.

In general, a state space is represented by 4 tuples as follows: $S: [S, s_0, O, G]$

Where **S**: Set of all possible states.

s_0 : start state (initial configuration) of the problem, $s_0 \in S$.

O: Set of production rules (or set of state transition operators) used to change the state from one state to another. It is the set of arcs (or links) between nodes.

The **production rule** is represented in the form of a pair. Each pair consists of a left side that determines the applicability of the rule and a right side that describes the action to be performed, if the rule is applied.

G: Set of Goal states, $G \in S$.

The sequence of actions (or operators) is called a solution path. It is a path from the initial state to a goal state. This sequence of actions leads to a number of states, starting from the initial state to a goal state, as $\{s_0, s_1, s_2, \dots, s_n \in G\}$. A sequence of states is called a path. The cost of a path is a positive number. In most of the cases, the path cost is computed as the sum of the costs of each action.

The following Figure 3 shows a search process in a given state space.

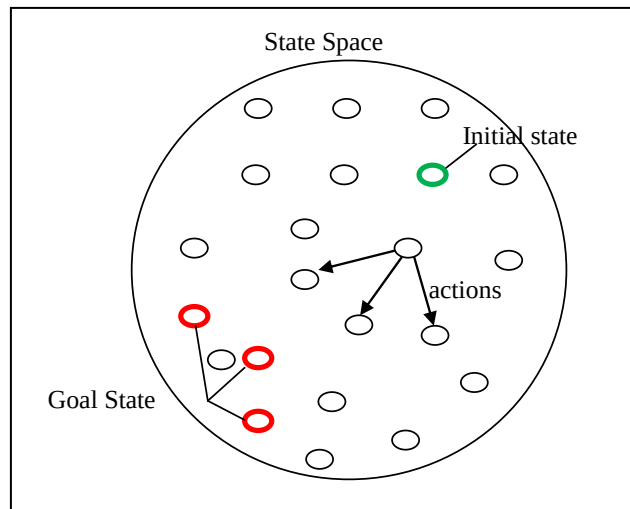


Fig. 3: State space with initial and goal node

We need to identify a sequence of actions that will turn the initial state s_0 into the desired goal state G . State space is commonly defined as a directed graph or as a tree in which each node is a state and each arc represents the application of an operator transforming a state to a successor state.

2.2.4 SEARCHING FOR SOLUTIONS

After formulating our problem, we are ready to solve it. This can be done by searching through the state space for a solution. This search will be applied on a search tree or, generally, a graph that is generated using the initial state and the successor function.

Searching is applied to a search tree generated through state expansion. State expansion applies the successor function to the current state, where each state represents a node in the tree.

Generally, search is about selecting an option and putting the others aside for later in case the first option does not lead to a solution. The choice of which option to expand first is determined by the search strategy used.

Thus, the problem is solved by using the rules (operators), in combination with an appropriate control strategy, to move through the problem space until a path from the **initial state to a goal state** is found. This process is known as **search**. A solution path is a path in the state space from s_0 (initial state) to G (Goal state).

Example 1: State space representation of the Water-Jug problem (WJP):

Problem statement:

Given two jugs, a 4-gallon and a 3-gallon, both of which do not have measuring indicators on them. The jugs can be filled with water with the help of a pump that is available (as shown in Figure 4)

The question is “how can you get exactly 2 gallons of water into a 4-gallon jug?”

The Water Jug Problem

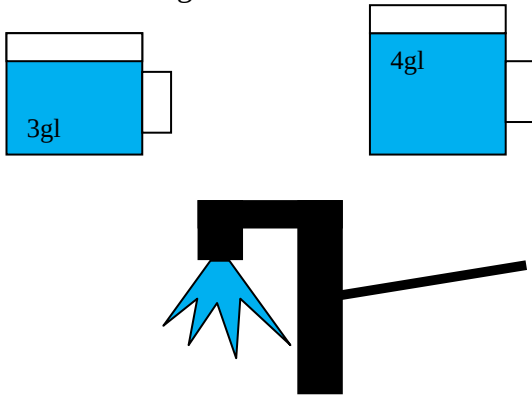


Fig. 4: The Water Jug problem

The state space or production rule of this problem can be defined as a collection of ordered pairs of integers (x, y) where $x = 0, 1, 2, 3, 4$ and $y = 0, 1, 2, \text{ or } 3$.

In the order pair (x, y) , x is the amount of water in a four-gallon jug, and y is the amount of water in the three-gallon jug.

State space: all possible combinations of (x, y)

The start state is $(0,0)$ and

Goalstate is $(2, n)$, where $n = 0, 1, 2, 3$

The following table-1 shows the set of **production rules** (actions) that can be used to change one state to another.

Table-1: Production rules for the Water Jug problem

Production Rules		
Rule No	Production	Meaning
R1	$(x, y \mid x < 4) \rightarrow (4, y)$	Fill 4-gallon jug
R2	$(x, y \mid y < 3) \rightarrow (x, 3)$	Fill 3-gallon jug
R3	$(x, y \mid x > 0) \rightarrow (0, y)$	Empty 4-gallon jug
R4	$(x, y \mid y > 0) \rightarrow (x, 0)$	Empty 3-gallon jug
R5	$(x, y \mid x + y \geq 4 \text{ and } y > 0) \rightarrow (4, y - (4 - x))$	Pour water from 3-gallon jug into 4-gallon jug until 4-gallon jug is full
R6	$(x, y \mid x + y \geq 3 \text{ and } x > 0) \rightarrow (x - (3 - y), 3)$	Pour water from 4-gallon jug into 3-gallon jug until 3-gallon jug is full
R7	$(x, y \mid x + y \leq 4 \text{ and } y > 0) \rightarrow (x + y, 0)$	Pour all water from 3-gallon jug into 4-gallon jug.
R8	$(x, y \mid x + y \leq 3 \text{ and } x > 0) \rightarrow (0, x + y)$	Pour all water from 4-gallon jug into 3-gallon jug.
R9	$(x, y \mid x > 0) \rightarrow (x - d, y)$	Pure some water d out from 4-gallon jug
R10	$(x, y \mid y > 0) \rightarrow (x, y - d)$	Pure some water d out from 3-gallon jug

The following 2 solutions are found for the problem “how can you get exactly 2 gallons of water into 4-gallon jug”, as shown in Table-2 and in Table-3.

Solution-1:

Table 2: Getting exactly 2 gallons of water into 4-gallon jug (solution1)

4-gal jug	3-gal jug	Rule applied
0	0	Initial state
4	0	R1 {fill 4-gal jug}
1	3	R6 {Pure all water from 4-gal jug to 3-gal jug}
1	0	R4 {empty 3-gal jug}
0	1	R8 {Pour water from 4 to 3-gal jug}
4	1	R1 {Fill 4-gal jug}
2	3	R6 {Pour water from 4-gal jug to 3-gal jug until it is full}
2	0	R4 {empty 3-gal jug}

Solution-2

Table 3: Getting exactly 2 gallons of water into 4-gallon jug (solution2)

4-gal jug	3-gal jug	Rule applied
0	0	Initial state
0	3	R2 {fill 3-gal jug}
3	0	R7 {Pure all water from 3-gal jug to 4-gal jug}
3	3	R2 {fill 3-gal jug}
4	2	R5 {Pour from 3 to 4-gal jug until it is full}
0	2	R3 {Empty 4-gal jug}
2	0	R7 {Pour all water from 3-gal jug to 4-gal jug}

A state space tree for WJP with all possible solutions is shown in Figure 5.

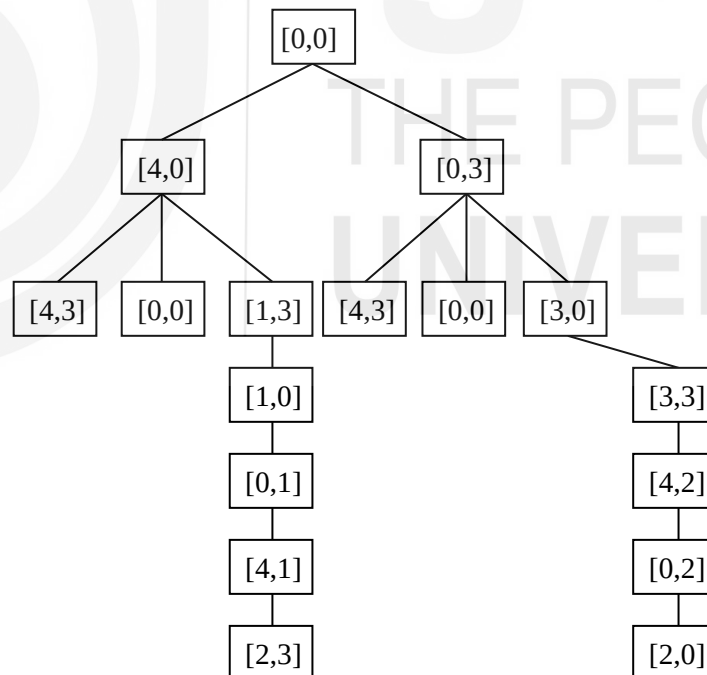


Fig 5: All possible solutions of WJP using the state space tree

2.3 FORMULATION OF THE 8 PUZZLE PROBLEM

The eight-tile puzzle consists of a 3-by-3 (3×3) square frame board that holds eight (8) movable tiles numbered as 1 to 8. One square is empty, allowing the

adjacent tiles to be shifted. The objective of the puzzle is to find a sequence of tile movements that leads from a starting configuration to a goal configuration

The Eight Puzzle Problem formulation:

Given a 3×3 grid with 8 sliding tiles and one “blank.”

Initial state: some other configuration of the tiles, for example

3	1	2
4		5
6	7	8

Goal state:

	1	2
3	4	5
6	7	8

Operator: Slide tiles (Move Blank) to reach the goal (as shown below). There are 4 operators that are “Moving the blank”:

Move the blank UP,

Move the blank DOWN,

Move the blank LEFT and

Move the blank RIGHT.

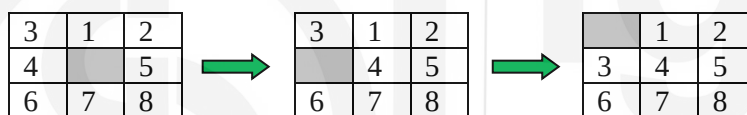


Fig. 6: Moving Blank LEFT and then UP

Path Cost: Sum of the cost of each path from the initial state to the goal state. Here, the cost of each action (blank move) = 1, so the cost of a sequence of actions = the number of actions. An optimal solution has the lowest cost path.

Performing State-Space Search: Basic idea:

If the initial state is a goal state, return it.

If not, apply the operators to generate all states that are one step from the initial state (its successors)

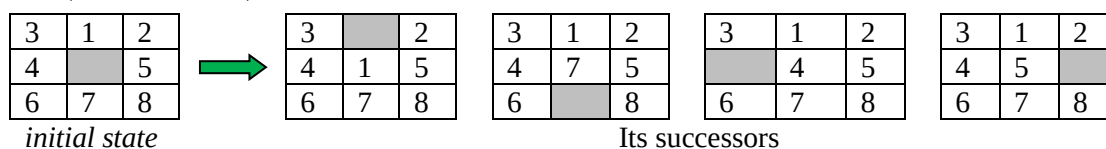


Fig. 7: All possible successors for a given initial state

Consider the successor (and their successors...) until you find a goal state.

Different search strategies consider the state in different orders. They may use different data structures to store the states that have yet to be considered.

State-Space Search Tree for 8-Puzzle problem:

The predecessor reference connects the search nodes, creating a data structure known as a tree.

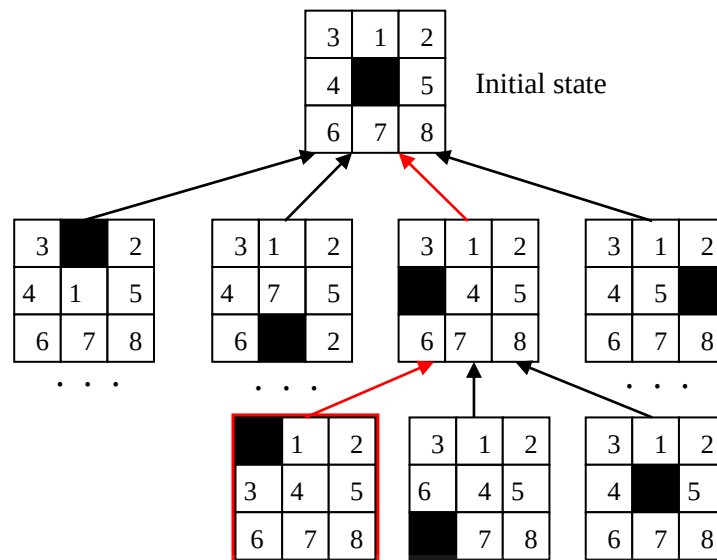


Fig. 8: Tracing a tree bottom-up from the goal state to the initial state

When we reach a goal, we trace up the tree to get the solution, i.e., the sequence of actions from the initial state to the goal.

Q.1 Find the minimum cost path for the 8-puzzle problem, where the start and goal state are given as follows:

1	2	3
4	8	-
7	6	5

Start State

1	2	3
4	5	6
7	8	-

Goal State

Initial State: $\{(1, 2, 3), (4, 8, -), (7, 6, 5)\}$

Successor State: $\{(1, 2, 3), (4, 8, 5), (7, 6, -)\}$; Move 5 up or – to down

$\{(1, 2, 3), (4, 8, 5), (7, -, 6)\}$; Move 6 right or – to left

$\{(1, 2, 3), (4, -, 5), (7, 8, 6)\}$; Move 8 down or – to up

$\{(1, 2, 3), (4, 5, -), (7, 8, 6)\}$; Move 5 to left or – to right

Goal State: $\{(1, 2, 3), (4, 5, 6), (7, 8, -)\}$; Move 6 up or – to down

PATH COST=5

2.4 N QUEEN'S PROBLEM-FORMULATION AND SOLUTION

The N-Queen problem is the problem of placing N Queens (Q1, Q2, Q3, ..., Qn) on an $N \times N$ chessboard so that no two queens attack each other. The colour of the queens is meaningless in this puzzle, and any queen is assumed to attack any other. So, a solution requires that no two queens share the same row, column, or diagonal.

The N-queen problem must follow the following rules:

1. There is at most one queen in each column.
2. There is at most one queen in each row.
3. There is at most one queen in each diagonal.

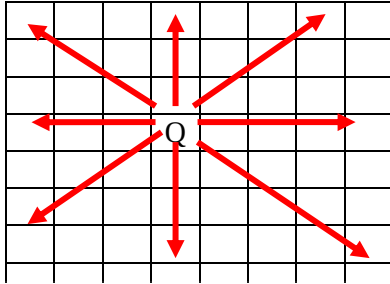


Fig. 9: No two queens placed on the same row, column, or diagonal

The N Queen's problem was originally proposed in 1848 by the chess player Max Bazzel, and over the years, many mathematicians, including Gauss, have worked on this puzzle. In 1874, S. Gunther proposed a method of finding solutions by using determinants, and J.W.L. Glaisher refined this approach.

The solutions that differ only by summary operations (rotations and reflections) of the board are counted as one. For 4 queens' problems, there are 16_{C_4} possible arrangements on a 4×4 chessboard and there are only 2 possible solutions for 4 Queen's problem. Note that there is only 1 unique solution, out of 2 possible solutions, as the second solution is just a mirror image of the first solution

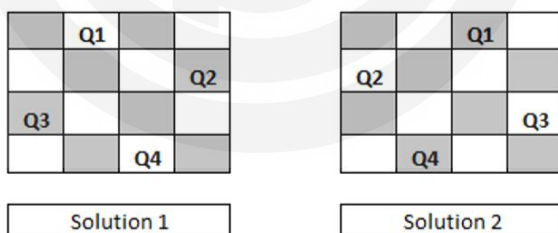


Fig 10 Two possible solutions of 4-Queen's problem

Similarly, the one possible solution for the 8-queens problem is shown in Figure 11.

The 8-queen problem is computationally very expensive since the total number of possible arrangements of queens on a 8×8 chessboard is $64_{C_8} = 64!/(56! \times 8!) \approx 4.4 \times 10^9$. Note that the 8-Queens problem has 92 **distinct** solutions and 12 **unique** solutions, as shown in table-4.

		column →							
		1	2	3	4	5	6	7	8
Row ↓	1				Q				
	2						Q		
	3								Q
	4		Q						
	5							Q	
	6	Q							
	7			Q					
	8					Q			

Fig. 11: One possible solution of 8 Queens problem

8-tuple = (4, 6, 8, 2, 7, 1, 3, 5)

The following table-4 summarizes both distinct and unique solutions for the problem of 1-Queen to 26 Queens problem. In general, there is no known formula to find the exact number of solutions for the N-queens problem.

Table-4 Solution of N Queen's problem for N=1 to N=26, both Unique and Distinct

No. of Queen's	Unique Solution	Total distinct solutions
1	1	1
2	0	0
3	0	0
4	1	2
5	2	10
6	1	4
7	6	40
8	12	92
9	46	352
10	92	724
11	341	2,680
12	1787	14,200
13	9233	73,712
14	45,752	365,596
15	285,053	2,279,184
16	1,846,955	14,772,184
17	11,977,939	95,815,104
18	83,263,591	666,090,624
19	621,012,754	4,968,057,848
20	4,878,666,808	39,029,188,884
21	39,333,324,973	314,666,222,712
22	336,376,244,042	2,691,008,701,644
23	3,029,242,658,210	24,233,937,684,440
24	28,439,272,956,934	227,514,171,973,736
25	275,986,683,743,434	2,207,893,435,808,352
26	2,789,712,466,510,289	22,317,699,616,364,044

2.4.1 Formulation of 4-Queen's problem:

States: any arrangement of 0 to 4 queens on the board

Initial state: 0 queens on the board

Successor function: Add a queen in any square

Goal test: 4 queens on the board, none attacked

For the initial state, there are 16 successors. At the next level, each of the states has 15 successors, and so on down the line. This search tree can be restricted by considering only those successors where no queens are attacking each other. To do that, we have to check the new queen with all the other queens on the board. In this way, the answer is found at a depth of 4. For the sake of simplicity, you can consider the problem of 4-Queens and see how the 4-Queens problem is solved using the concept of "Backtracking".

2.4.2 State space tree for the 4 Queens' Problem

We place the queen row-by-row (i.e., Q_1 in row 1, Q_2 in row 2 and so on). Backtracking gives "all possible solutions". If you want an optimal solution, then go for Dynamic programming.

Let's see the Backtracking method. There are $16C_4$ ways to place a queen on a 4x4 chess board as shown in the following state space tree (Figure 12). In a tree, the value (i, j) means in the i^{th} row, j^{th} queen is placed.

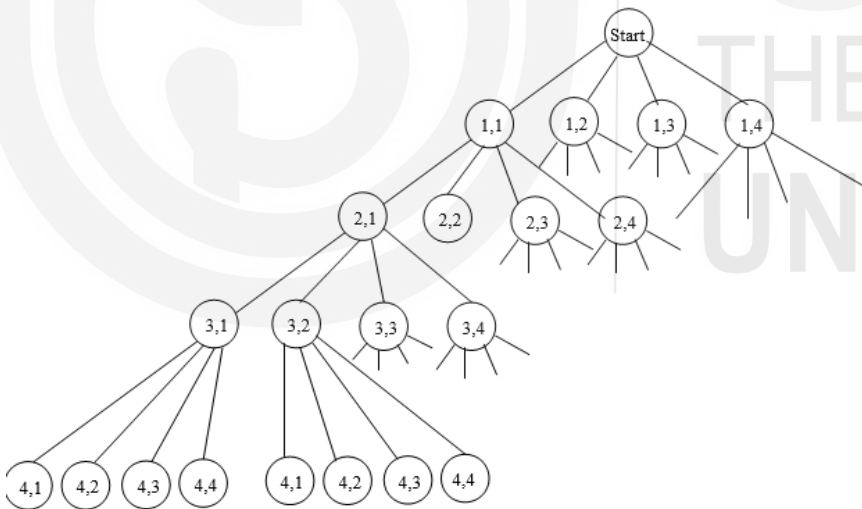


Fig. 12: State-space tree showing all possible ways to place a queen on a 4x4 chess board

So, to reduce the size (not anywhere on the chessboard, since there are $16C_4$ possibilities), we place queens row-by-row, and no Queen in the same column. This tree is called a permutation tree (**here we avoid same row or same column but allow diagonals**)

$$\text{Total nodes} = 1 + 4 + 4 \times 3 + 4 \times 3 \times 2 + 4 \times 3 \times 2 \times 1 = 65$$

The edges are labeled by possible values of x_i . Edges from level 1 to level 2 nodes specify the values for x_1 . Edges from the level i to level $i + 1$ are labeled with the values of x_i .

The solution space is defined by all paths from root node to leaf node. There are $4! = 24$ leaf nodes in the tree. Nodes are numbered as depth first Search. The state space tree for 4-Queen's problem (**avoid same row or same columns but allowing diagonals**) is shown in figure 13.

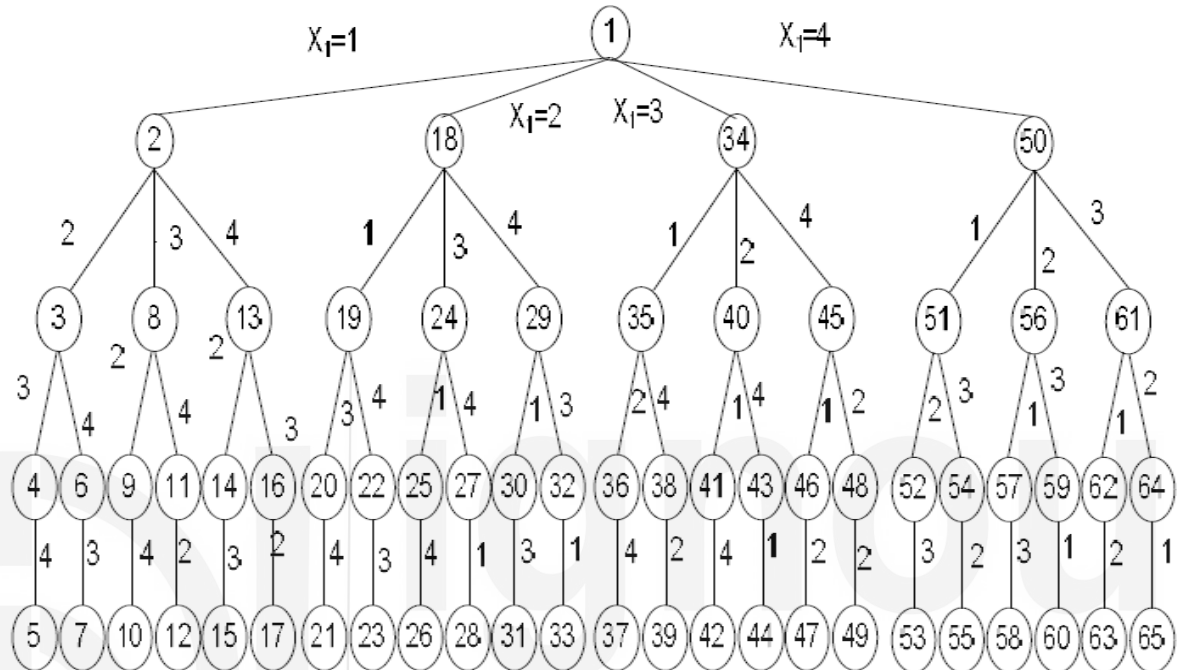


Fig. 13: State space tree for 4 queen's problems (allowing same diagonal but not same row and column)

The two solutions are found in the tree, as
 $(x_1, x_2, x_3, x_4) = (2, 4, 1, 3)$ and
 $(x_1, x_2, x_3, x_4) = (3, 1, 4, 2)$, which is shown in Figure 14.

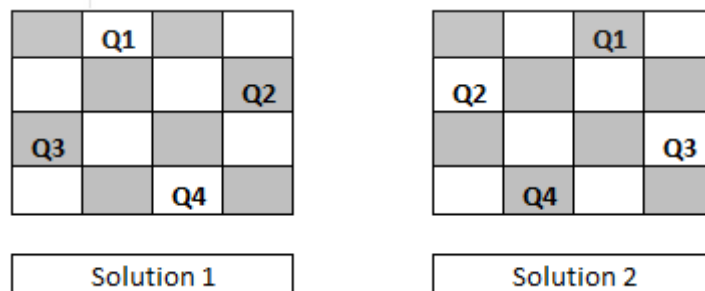


Fig. 14: Two possible solutions of 4-Queen's problem

Note that the second solution is just a mirror image of the first solution.

We can further reduce the search space, as shown in Figure 6, by avoiding the diagonal also. Now you can avoid the **same row**, **avoid same columns**, and **avoid same diagonals**, while placing any queen. In this case, the state space tree looks as shown in Figure 15.

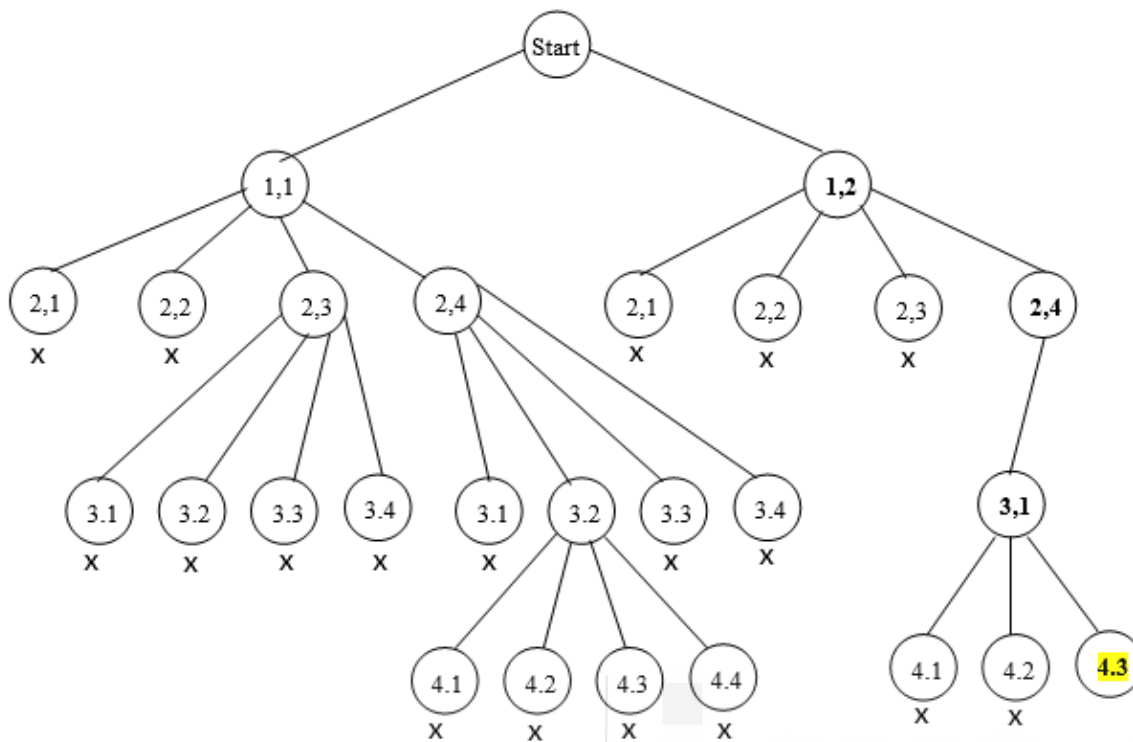


Fig 15: State space tree for 4 queen's problem (avoiding same row, columns, and diagonal)

Note that Queens are placed row-by-row, that is, Q_1 in row 1, Q_2 in row 2 and so on. In a tree, node (1,1) is a promising node (no queen attack) as Q_1 is placed in 1st row and 1st column. Node (2,1) is a non-promising node because we cannot place Q_2 in the same column (as Q_1 is already placed in column 1). Note that a non-promising node is marked as X. So, we try (2, 2) again, non-promising (due to same column), next try (2,3), it's a promising node, so proceed and try to place 3rd queen on 3rd row. But in 3rd row, all positions (3,1), (3,2), (3,3) and (3,4) are non-promising and we cannot place the Q_3 in any of this position. So, we backtrack to (1,1) and try for (2,4) and so on. Backtracking approach gives "all possible solutions". Figure 7 shows one possible solution for 4-Queen's problem as $\{(1,2), (2,4), (3,1), (4,3)\}$. This can also be written as $(x_1, x_2, x_3, x_4) = (2, 4, 1, 3)$. There are 2 possible solution of 4-Queen's problem. Another solution is $(x_1, x_2, x_3, x_4) = (3, 1, 4, 2)$, which is a mirror image of 1st solution.

2.4.3 Backtracking Approach to solve N queen's Problem:

Consider the chessboard squares' indices of the 2-Dimensional array $[1 \dots n, 1 \dots n]$. We observe that every element on the same diagonal that rows from the upper left to the right has the same $(row - column)$ value. It is called the **left diagonals**. Similarly, every element on the same diagonal that goes from the upper right to the lower left has same $(row + column)$ value. This is called **Right Diagonals**. For example, consider a 5×5 chessboard as $[1 \dots 5, 1 \dots 5]$ (as shown in figure 16).

Case 1: (Left diagonal):- Suppose queens are palced in same diagonal in locations: (1, 2), (2, 3), (3, 4), (4, 5) or (1, 4), (2, 5) or any other same left diagonal value. Observe that every element on the same diagonal has the same $(row - column)$ value. Similarly,

Case2: (right diagonal):- suppose queens are placed in same diagonal in locations: (1,3), (2,2), (3,1) or (1,4), (2,3), (3,2), (4,1) or any other same right diagonal value. Observe that every element on the same diagonal has the same $(row + column)$ value.

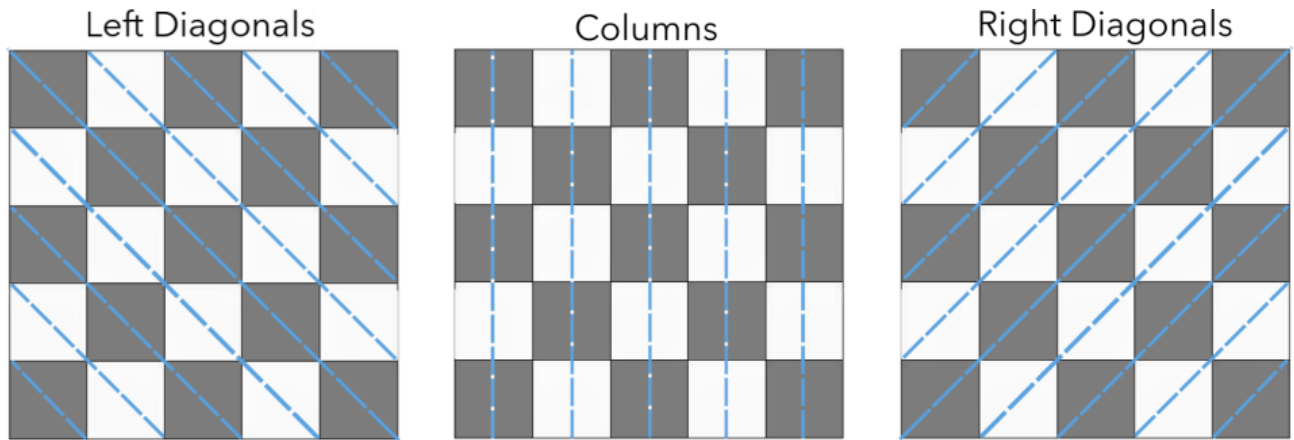


Fig. 16: Left diagonal and right diagonal for 5×5 chessboard.

Suppose two queens are placed at positions (i, j) and (k, l) then they are on the same diagonal if and only if:

$$(i - j) = (k - l) \text{ or } (j - l) = (i - k) \text{ -----(1) [left diagonal]}$$

$$(i + j) = (k + l) \text{ or } (j - l) = (k - i) \text{ ----- (2) [right diagonal]}$$

From equations (1) and (2), we can combine and write a one condition to check the diagonal as: **$abs(j - l) = abs(i - k)$** .

Algorithm NQueen (k, n)

// This procedure prints all possible placements of n queens on $n \times n$ chessboard so that they are non-attacking.

```
{
1. for i=1 to n do
2. {
3.   if place(k, i) then
4.   {
5.     x[k]=i;
6.     if (k==n) then print(x[1...n])
7.     else
8.       NQueen(k+1, n);
9.   }
10. }
11. }
```

Algorithm place(k, i)

/*This algorithm returns true if a queen can be placed in the kth row, ith column. Otherwise, it returns false. X[] is a global array. Abs® returns the absolute value of r */.

```

1. {
2.   for j=1 to k-1 do
3.     {
4.   if(x[j]=i)    // in the same column
5. Or (abs(x[j]-i)==abs(j-k))// in the
                        //same diagonal
6.       Return false;
7.     }
8.   Return true;
9. }

```

Check Your Progress 1

1. What are the various factors need to be taken into consideration when developing a state space representation?

.....

.....

2. Consider the following Missionaries and cannibals problem:
Three missionaries and three cannibals are on the side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side, without ever leaving a group of missionaries outnumbered by cannibals.

- a) Formulate the missionaries and cannibals problem.
- b) Solve the problem formulated in part (a)
- c) Draw the state- space search graph for solving this problem.

.....

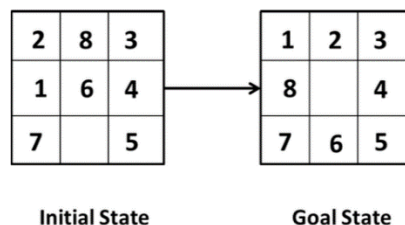
.....

3. Draw a state space tree representation to solve the Tower of Hanoi problem. (Hint: You can take the number of disks $n=2$ or 3).

.....

.....

4. Draw the state space tree for the following 8-puzzle problem, where the start and goal states are given below. Also, find the minimum cost path for this 8-puzzle problem. Each blank move is having cost=1.



.....

.....

5. Discuss a Backtracking algorithm to solve a N-Queen's problem. Draw a state space tree to solve a 4-Queen's problem.

.....

.....

2.5 ADVERSARIAL SEARCH-TWO AGENT SEARCH

In computer science, a search algorithm is an algorithm for finding an item with specified properties among a collection of items. The items may be stored individually as records in a database, or may be elements of a search space defined by a mathematical formula or procedure.

Adversarial search is a **game-playing** technique where the agents are surrounded by a competitive environment. A conflicting goal is given to the agents (multi-agent). These agents compete with one another and try to defeat one another in order to win the game. Such conflicting goals give rise to the adversarial search. Here, game-playing means discussing those games where **human intelligence** and **logic factor** is used, excluding other factors such as **luck**. **Tic-tac-toe, chess, checkers**, etc., are such types of games where no luck factor works; only the mind works.

Mathematically, this search is based on the concept of '**Game Theory.**' *According to game theory, a game is played between two players. To complete the game, one has to win the game, and the other loses automatically.'*



We are opponents- I win, you loose.

Techniques required to get the best optimal solution

There is always a need to choose those algorithms that provide the best optimal solution in a limited time. So, we use the following techniques, which could fulfil our requirements:

- **Pruning:** A technique that allows ignoring the unwanted portions of a search tree, which make no difference in its final result.
- **Heuristic Evaluation Function:** It allows to approximate the cost value at each level of the search tree, before reaching the goal node.

2.5.1 Elements of Game Playing search

To play a game, we use a game tree to know all the possible choices and to pick the best one. There are the following elements of a game-playing:

- **S₀:** It is the **initial state** from where a game begins.
- **PLAYER (s):** It defines which player is having the current turn to make a move in the state.
- **ACTIONS (s):** It defines the set of legal moves that a player can make to change the state.

- **RESULT (s, a):** It is a transition model that defines the result of a move.
- **TERMINAL-TEST (s):** Determines whether the game has ended. It returns true if state s is a terminal state, and false otherwise. States where the game is over are called terminal states.
- **UTILITY (s, p):** It defines the final value with which the game has ended. This function is also known as **the objective function** or **Payoff function**. This utility function gives a numeric value for the outcome of a game i.e.

For example, in **chess** and **tic-tac-toe**, we have two or three possible outcomes. Either win or lose, or draw the match, which we can represent by the values **+1, -1 or 0**. In other words, we can say that **(-1)**, if the PLAYER loses, **(+1)**, if the PLAYER wins, and **(0)**, If there is a draw between the PLAYERS.

Let's understand the working of the elements with the help of a game tree designed for **tic-tac-toe**. Here, the node represents the game state, and edges represent the moves taken by the players. The root of the tree is the initial state. Next level is all of MAX's moves, then next level is all of MIN's moves, and so on. Note that the root has 9 blank squares (MAX), level 1 has 8 blank squares (MIN), level 2 has 7 blank squares (MAX), and so on.

Objective:

Player1: Maximize outcome and **Player2:** Minimize outcome

Terminal (goal) state:

utility: -1, 0, +1 (that is, win for X is +1, win for O is -1, and 0 for a draw).

The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX. High values are assumed to be good for MAX and bad for MIN (which is how the players get their names). It is MAX's job to use the search tree to determine the best move. Note that if MAX wins, then the utility value is +1, if MIN wins, then the utility value is -1, and if DRAW, then the utility value is 0.

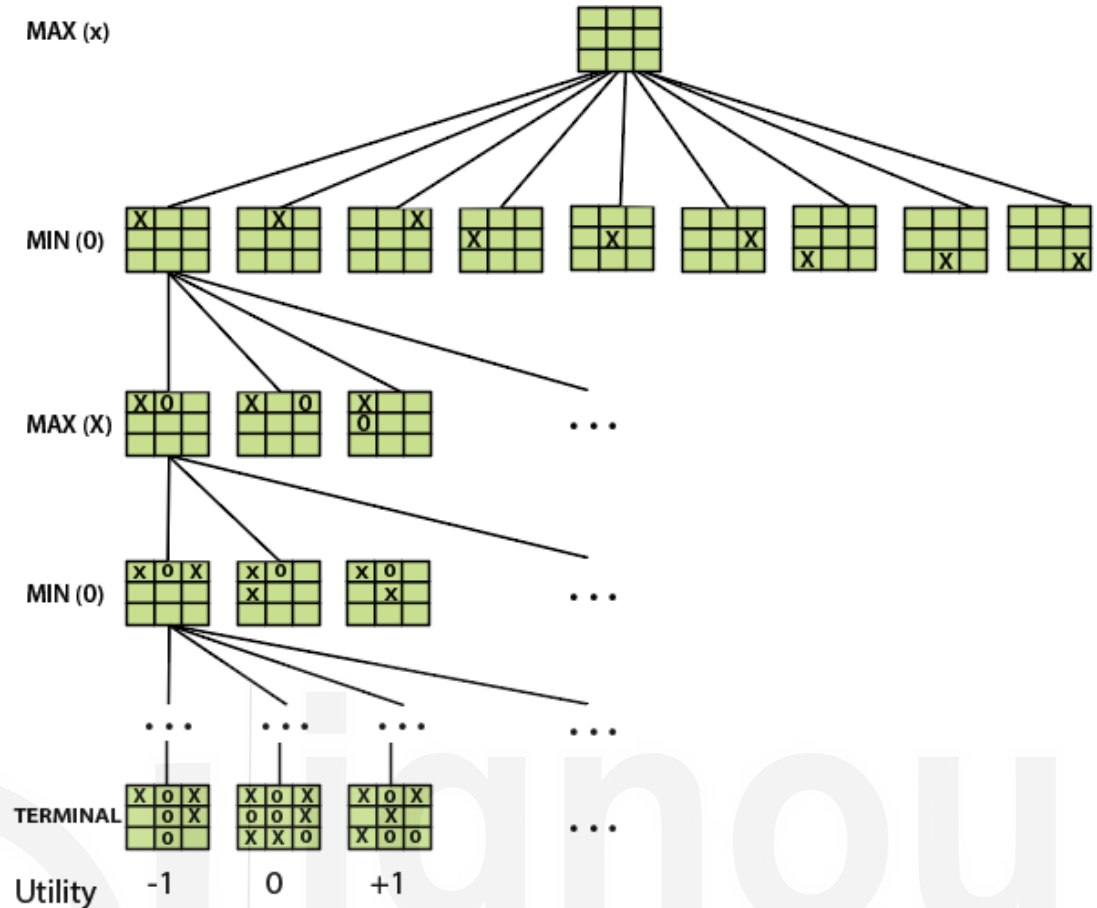


Fig. 17: A game-tree for tic-tac-toe

In a tic-tac-toe game playing, as shown in Figure 17, we have the following elements:

- **INITIAL STATE (S_0):** The top node in the game-tree represents the initial state in the tree and shows all the possible choices to pick out one.
- **PLAYER (s):** There are two players, **MAX** and **MIN**. **MAX** begins the game by picking the best move and placing **X** in the empty square box.
- **ACTIONS (s):** Both players can make moves in the empty boxes, chance by chance.
- **RESULT (s, a):** The moves made by **MIN** and **MAX** will decide the outcome of the game.
- **TERMINAL-TEST(s):** When all the empty boxes are filled, it will be the terminating state of the game.
- **UTILITY:** At the end, we will get to know who wins: **MAX** or **MIN**, and accordingly, the price will be given to them. If **MAX** wins, then the utility value is +1, if **MIN** wins, then the utility value is -1; and if **DRAW**, then the utility value is 0.

2.5.2 Issues in Adversarial Search

In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. But in an **adversarial search**, the result depends on the players who will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and within a limited time. **The Min-**

Max search algorithm is an example of adversarial search. **Alpha-beta Pruning** is used to reduce the search space.

Problem Solving Using
Search

2.6 MIN-MAX SEARCH STRATEGY

In artificial intelligence, minimax is a **decision-making** strategy under **game theory**, which is used to minimize the losing chances in a game and to maximize the winning chances. This strategy is also known as '**Min-Max**,' '**MM**,' or '**Saddle point**.' Basically, it is a two-player game strategy where *if one wins, the other loses the game*. This strategy simulates those games that we play in our day-to-day life. Like, if two people are playing chess, the result will be in favour of one player and against the other one. The person who will make his best try, *efforts as well as cleverness, will surely win*.

We can easily understand this strategy via a **game tree**, where nodes are the states of the game, and edges are moves that were made *by the players in the game*. Players will be two, namely:

- **MIN**: Decrease the chances of **MAX** winning the game.
- **MAX**: Increases his chances of winning the game.

They both play the game alternatively, i.e., turn by turn, and following the above strategy, i.e., if one wins, the other will definitely lose it. Both players look at one another as competitors and will try to defeat one another, giving their best.

In the minimax strategy, the result of the game or the utility value is generated by a **heuristic function** by propagating from the initial node to the root node. It follows the **backtracking technique** and backtracks to find the best choice. MAX will choose that path which will increase its utility value, and MIN will choose the opposite path, which could help it to minimize MAX's utility value.

2.6.1 MINIMAX Algorithm

The MINIMAX algorithm is a backtracking algorithm that backtracks to pick the best move out of several choices. MINIMAX strategy follows the **DFS (Depth-first search)** concept. Here, we have two players, **MIN** and **MAX**, and the game is played alternately between them, i.e., when **MAX** makes a move, then the next turn is of **MIN**. It means the move made by MAX is fixed, and he cannot change it. The same concept is followed in the DFS strategy, i.e., we follow the same path and cannot change in the middle. That's why in the MINIMAX algorithm, instead of BFS, we follow DFS. The following steps are used in the MINMAX algorithm:

- Generate the whole game tree, all the way down to the terminal states.
- Apply the utility function to each terminal state to get its value.
- Use the utility of the terminal states to determine the utility of the nodes one level higher up in the search tree.
- Continue backing up the values from the leaf nodes toward the root, one layer at a time.
- Eventually, the backed-up values reached the top of the tree; at that point, MAX chooses the move that leads to the highest value.

This is called a minimax decision because it maximizes the utility under the assumption that the opponent will play perfectly to minimize it. To better

understand the concept, consider the following game tree or search tree as shown in Figure 18.

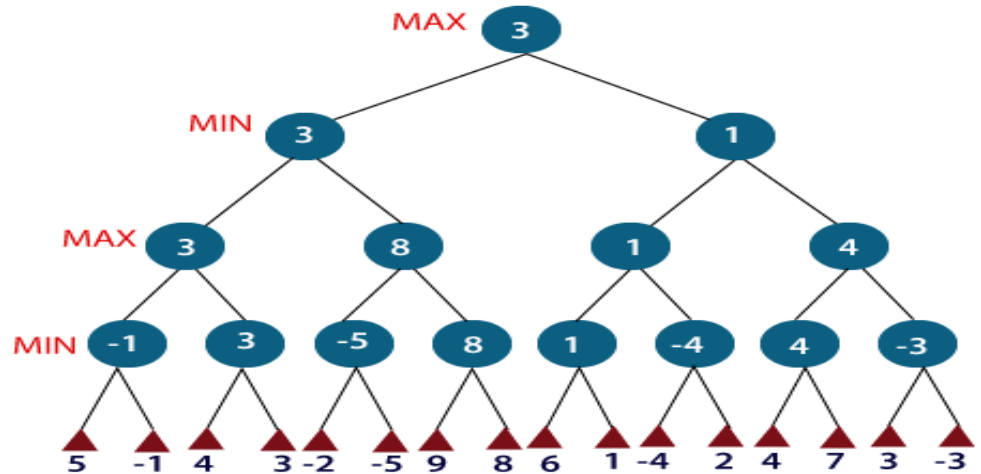


Fig 18 Two player game tree

In the figure above, the two players **MAX** and **MIN** are there. **MAX** starts the game by choosing one path and propagating all the nodes of that path. Now, **MAX** will backtrack to the initial node and choose the best path where his utility value will be the maximum. After this, it's **MIN** chance. **MIN** will also propagate through a path and again will backtrack, but **MIN** will choose the path that could minimize **MAX** winning chances or the utility value.

So, if the level is minimizing, the node will accept the minimum value from the successor nodes. If the level is maximizing, the node will accept the maximum value from the successor.

In other words, we can say that Minimax is a decision rule algorithm, which is represented as a game tree. It has applications in decision theory, game theory, statistics, and philosophy. Minimax is applied in two-player games. One is the MIN player, and the other is the MAX player. By agreement, the root of the game tree represents the MAX player. It is assumed that each player aims to do the best move for himself and therefore the worst move for his opponent in order to win the game. The question may arise, “How to deal with the contingency problem?” The answer is:

- Assuming that the opponent is rational and always optimizes its behaviour (opposite to us), we consider the best response. opponent's
- Then the minimax algorithm determines the best move

2.6.2 Working of Minimax Algorithm

Minimax is applicable for decision making for two agent systems participating in a competitive environment. These two players, P1 and P2, also known as the MIN and MAX players, maximize and minimize the utility value of the heuristic function. The algorithm uses recursion to search through the game tree and compute a minimax decision for the current state. We traverse the complete game tree in a depth-first search (DFS) manner to explore the node. MAX player always selects the maximum value, and MIN always selects the

minimum value from its successor's node. The initial value of MAX and MIN is set to $MAX = -\infty$ and $MIN = +\infty$. This is the worst value assigned initially, and as the algorithm progresses, these values are changed, and finally, we get the optimal value.

Example 1: Let's take an example of a two-player game tree search (shown in Figure 19a) to understand the working of the Minimax algorithm.

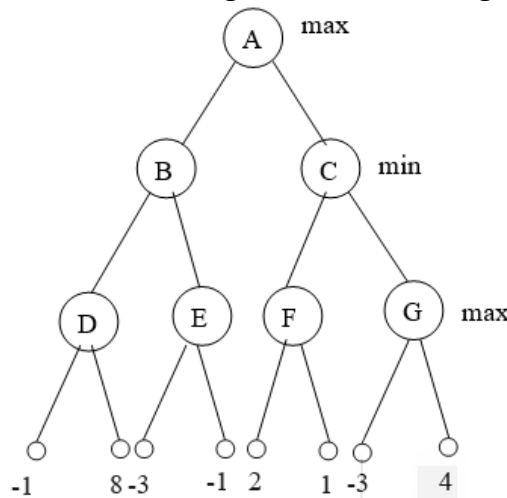


Fig 19a Two player game tree

The initial value of MAX and MIN is set to $MAX = -\infty$ and $MIN = +\infty$. The tree is traversed in a DFS manner. So, we start from node A, then move to node B, and then D.

Now at node D [$MAX = -\infty$]. Now, at D, it first checks the left child (which is a terminal node) with value -1. This node returns a value of $MAX = (-\infty, -1) = -1$. So, the modified value at node D is [$MAX = -1$]. Next, we proceed to the right child of Node D (which has a terminal value of 8) and compare this value (8) with the previous value at node D. That is $MAX = \max(-1, 8) = 8$. So, the final value at node D is 8.

Similarly, the value at node E (which is a Max node) is $MAX = \max(-\infty, -3) = -3$, then $\max(-3, -1) = -1$.

So, at node B, which is at MIN level, select the minimum value from its successor node D and E as $MIN = \min(8, -1) = -1$

Similarly, the value at node F (which is also Max node) is $MAX = \max(-\infty, 2) = 2$, then $\max(2, 1) = 2$, and

The value at node G (which is also MAX node) is $MAX = \max(-\infty, -3) = -3$, and then $\max(-3, 4) = 4$.

Thus, at node C, which is also at MIN level, select the minimum value from its successor node F and G as $MIN = \min(2, 4) = 2$.

Now, the value at node B and C is -1 and 2 respectively.

Thus, finally, the value at node A, which is at MAX level, is $MAX = \max(-1, 2) = 2$.

The final game tree with max or min value at each node and optimal path, with shaded line $A \rightarrow C \rightarrow F \rightarrow 2$, is shown in the following figure 19(b).

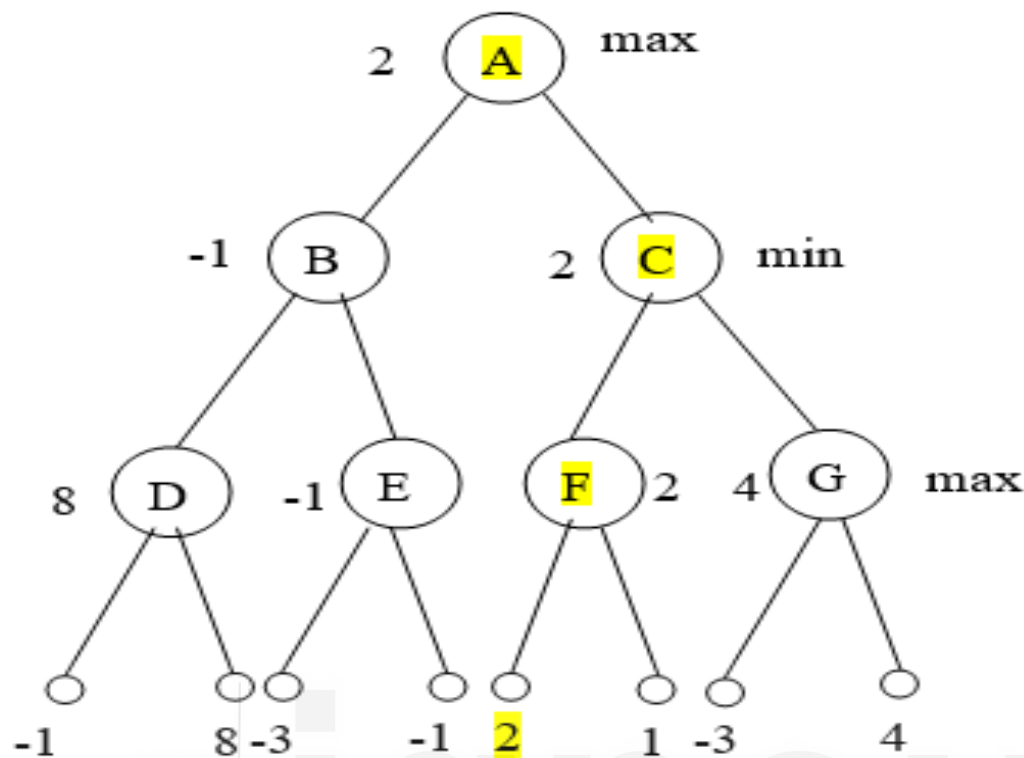
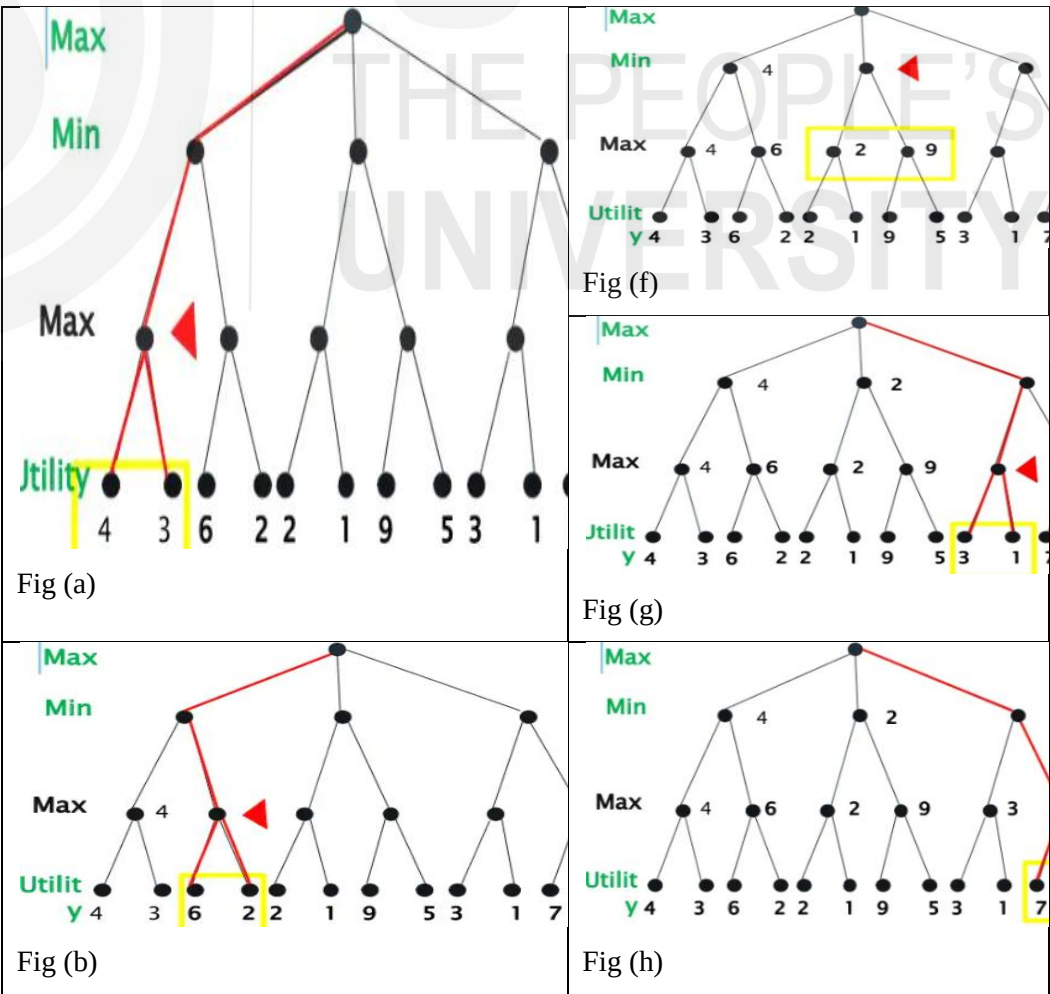
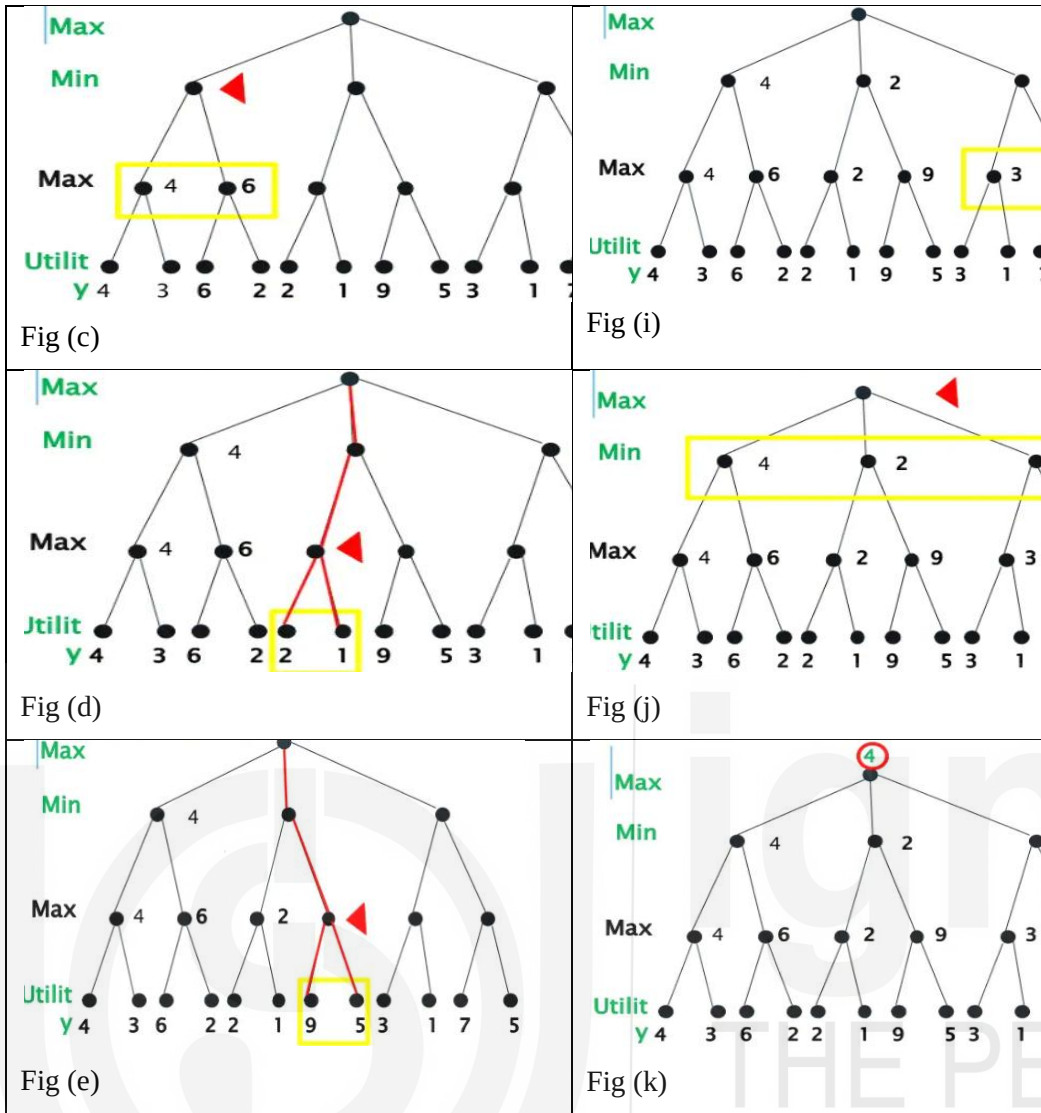


Fig 19(b) Game tree with final value at each node with optimal path

Example 2: Consider the following two-player game tree search. The working of Minimax algorithm is illustrated from fig (a)-fig(k)





2.6.3 Properties of Minimax Algorithm:

1. **Complete:** The Minimax algorithm is complete if the tree is finite.
2. **Optimal:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution. Minimax is Optimal.
3. **Time complexity:** $O(b^m)$, where b : Branching Factor and m is the maximum depth of the game tree.
4. **Space complexity:** $O(bm)$

For example, in a chess playing game, $b = 35$, $m \approx 100$ for “reasonable” games. In this case, the exact solution is completely infeasible

2.6.4 Advantages and disadvantages of Minimax search

Advantages:

- Returns an optimal action, assuming perfect opponent play.
- Minimax is the simplest possible (reasonable) game search algorithm.

Disadvantages:

- It's completely infeasible in practice.

- When the search tree is too large, we need to limit the search depth and apply an evaluation function to the cut-off states.

2.7 ALPHA-BETA PRUNING

The drawback of the Minimax strategy is that it explores each node in the tree deeply to provide the best path among all the paths. This increases its time complexity. If b is the branching factor and d is the depth of the tree, then the time complexity of the MINIMAX algorithm is $O(b^d)$, that is exponential. But as we know, the performance measure is the first consideration for any optimal algorithm. Alpha-beta pruning is a method to reduce (prune) search space. Using Alpha-Beta pruning, the Minimax algorithm is modified. Therefore, alpha-beta pruning reduces this drawback of the minimax strategy by exploring the nodes of the search tree.

The method used in alpha-beta pruning is that its **cut-off the search** by exploring a smaller number of nodes. It makes the same moves as a minimax algorithm does, but it prunes the unwanted branches using the pruning technique (discussed in adversarial search). Alpha-beta pruning works on two threshold values, i.e., α (**alpha**) and β (**beta**).

- α : It is the highest value a **MAX** player can have. The initial value of α is set to a negative infinity value, that is $\alpha = -\infty$. As the algorithm progresses, its value may change and finally get the best (highest) value.
- β : It is the best lowest value a **MIN** player can have. The initial value of β is set to a positive infinity value, that is $\beta = +\infty$. As the algorithm progresses, its value may change and finally get the best (lowest) value.

So, each MAX node has α -value, which never decreases, and each MIN node has β -value, which never increases. The main condition required for alpha-beta pruning is $\alpha \geq \beta$, that is if $\alpha \geq \beta$, then prune (cut) the branches; otherwise, proceed.

Note: The alpha-beta pruning technique can be applied to trees of any depth, and it is possible to prune the entire sub-trees easily.

2.7.1 Working of Alpha-beta Pruning

As we know, there are two-parameter is defined for Alpha-beta pruning, namely alpha (α) and beta (β). The initial value of alpha and beta is set to $\alpha = -\infty$ and $\beta = +\infty$. As the algorithm progresses, its values are changed accordingly. Note that in Alpha-beta pruning (cut), at any node in a tree, if $\alpha \geq \beta$, then prune (cut) the next branch else search is continued. Note the following point for alpha-beta pruning:

- The MAX player will only update the value of α (on MAX level).
- The MIN player will only update the value of β (on MIN level).
- We will only pass the α and β value from top to bottom (that is, from any parent to child node, but never from child to parent node).
- While backtracking the tree, the node values will be passed to the upper node instead of the values of α and β .
- Before going to the next branch of the node in a tree, we check the value of α and β . If the value of $\alpha \geq \beta$, then prune (cut) the next

(unnecessary) branches (i.e., no need to search the remaining branches where the condition $\alpha \geq \beta$ is satisfied) else search continued.

Consider the example below of a game tree where **P** and **Q** are two players. The game will be played alternately, i.e., chance by chance. Let **P** be the player who will try to win the game by maximizing their winning chances. **Q** is the player who will try to minimize **P**'s winning chances. Here, α will represent the maximum value of the nodes, which will be the value for **P** as well. β will represent the minimum value of the nodes, which will be the value of **Q**.

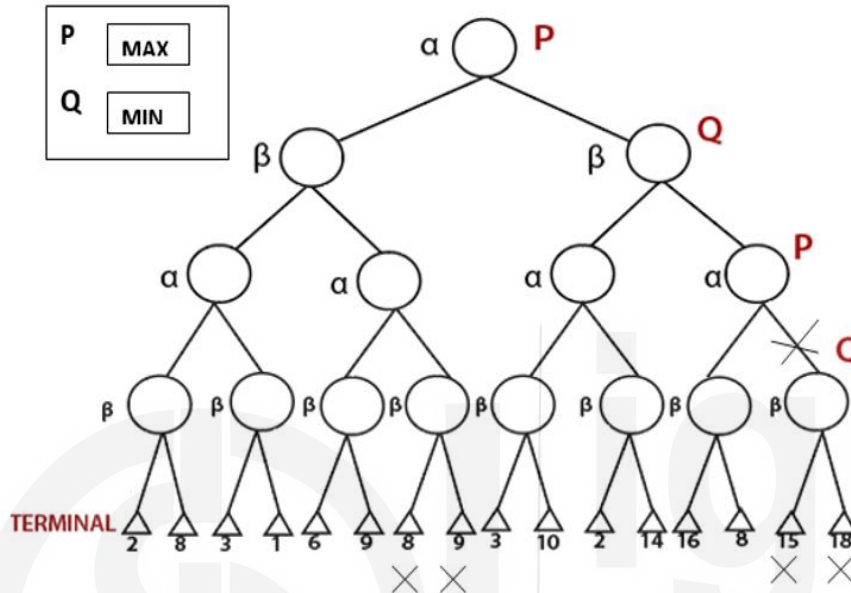


Fig 20: Alpha-beta pruning

- Any one player will start the game. Following the DFS order, the player will choose one path and will reach its depth, i.e., where he will find the **TERMINAL** value.
- If the game is started by player **P**, he will choose the maximum value in order to increase its winning chances with maximum utility value.
- If the game is started by player **Q**, he will choose the minimum value in order to decrease the winning chances of **A** with the best possible minimum utility value.
- Both will play the game alternately.
- The game will be started from the last level of the game tree, and the value will be chosen accordingly.
- Like in Figure 5, the game is started by player **Q**. He will pick the leftmost value of the **TERMINAL** and fix it for beta (β). Now, the next **TERMINAL** value will be compared with the β -value. If the value is smaller than or equal to the β -value, replace it with the current β -value otherwise no need to replace the value.
- After completing one part, move the achieved β -value to its upper node and fix it for the other threshold value, i.e., α .
- Now, it's **P**'s turn; he will pick the best maximum value. **P** will move to explore the next part only after comparing the values with the current ones α -value. If the value is equal to or greater than the current α -value, then only it will be replaced; otherwise, we will prune the values.
- The steps will be repeated until the desired result is obtained.
- So, the number of pruned nodes in the above example is **four**, and **MAX** wins the game with the maximum **UTILITY** value, i.e., **3**.

The rule which will be followed is: “**Explore nodes, if necessary, otherwise prune the unnecessary nodes.**”

Note: It is obvious that the result will have the same **UTILITY** value that we may get from the MINIMAX strategy.

Alpha beta cut-off (or pruning):

1. for each node store limit $[\alpha, \beta]$.

2. Update $[\alpha, \beta]$,

where α is the lower bound at max node; it can't decrease.

β is the upper bound at min node; it can't increase.

3. If α value of a max node is greater than β value of its parent ($\alpha \geq \beta$), the subtree of that max node need not be evaluated (i.e., pruned).

4. If β value of a min node is lesser than α value of its parent ($\beta \leq \alpha$), the subtree of that min node need not be evaluated (i.e., pruned).

Example 1: Let's take an example of two-player search tree (Figure 21) to understand the working of alpha-beta pruning.

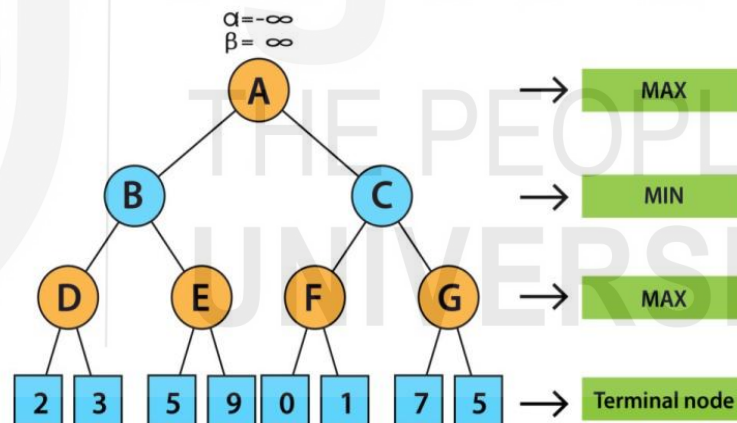


Fig. 21: Two player search tree

We initially start the search by setting the initial value of $\alpha = -\infty$ and $\beta = +\infty$ to root **node A**.

Note the following important point to apply the Alpha-beta pruning:

- We will only pass the α and β value from top to bottom (that is from any parent to child node), but never from child to parent node.
- While backtracking the tree (from bottom to top node), the node values will be passed to upper node instead of values of α and β .
- Before exploring the next branch in a tree, we check $\alpha \geq \beta$. If **YES**, then prune (cut) the next (unnecessary) branches (i.e., no need to search

the remaining branches where the condition $\alpha \geq \beta$ is satisfied) else search continued.

- The MAX player will only update the value of α (on MAX level) and the MIN player will only update the value of β (on MIN level).

Problem Solving Using Search

Step 1: We traverse the tree in a depth-first search (DFS) manner and assign (pass) this value of α and β down to subsequent nodes **B** and then to node **D** as $[\alpha = -\infty; \beta = +\infty]$.

Now at node D $[\alpha = -\infty, \beta = +\infty]$, since node D is at MAX level, so only α value will be changed. Now, at D, it first checks the left child (which is a terminal node) with value 2. This node returns a value of 2. Now, the value of α at node D is calculated as $\alpha = \max(-\infty, 2) = 2$. So the modified value at node D is $[\alpha = 2, \beta = +\infty]$. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is NO since $2 \not\geq +\infty$. So, proceed and search continues for the right child of Node D.

The value of the right child (terminal node with value=3) of D returns a value 3. Now at D, the value of α is compared with the terminal node value 3, that is, $\alpha = \max(2, 3) = 3$. Now the value of Node(D)=3, and the final values of α and β is updated at node D as $[\alpha = 3, \beta = +\infty]$ as shown in figure 21(a).

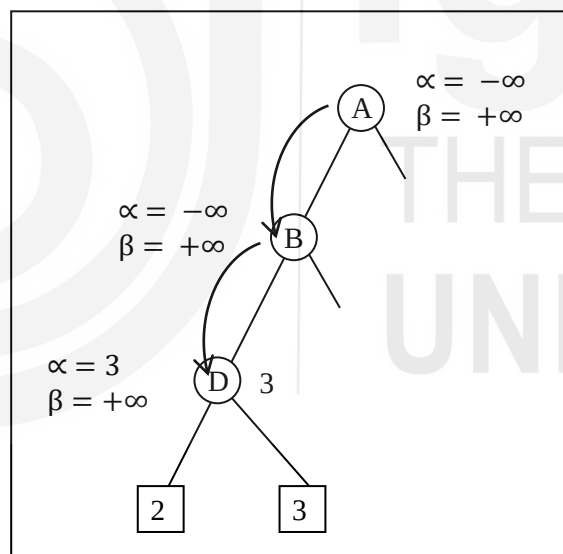


Fig 21(a)

Step 2: We backtrack from node D to B. Note that, while backtracking the node in a tree, the node values of D(=3) will be passed to the upper node B instead of the values of α and β . Now the value of node(B) = node(D) = 3. Since B is at MIN level, so only β value will be changed. Now at node **B** $[\alpha = -\infty, \beta = 3]$ (note that β is changed from $+\infty$ to 3). Here we again check $\alpha \geq \beta$. It is False, so search is continued on right side of B, as shown in Figure 21(b).

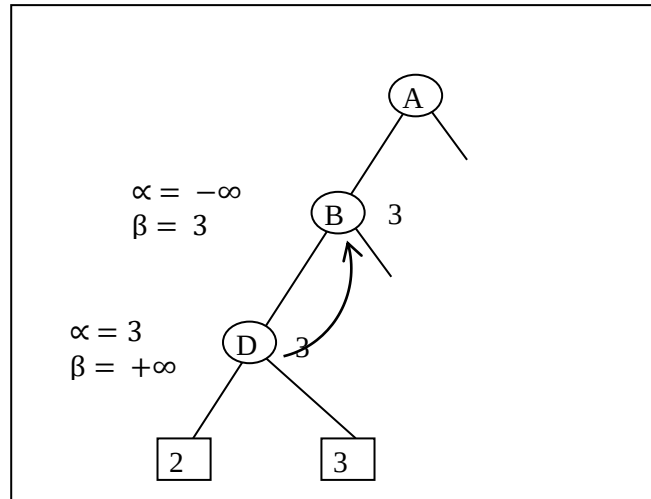


Fig. 21(b)

Step 3: B now calls E, and we pass the α and β value from top node B to bottom node E as $[\alpha = -\infty, \beta = 3]$. Since node E is at MAX level, so only α value will be changed. Now, at E, it first checks the left child (which is a terminal node) with value 5. This node returns a value of 5.

Now, the value of α at node E is calculated as $\alpha = \max(-\infty, 5) = 5$, so value of Node(E)=5 and modified value of α and β at node E is $[\alpha = 5, \beta = 3]$. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is **YES**, since $5 \geq 3$. So, we prune (cut) the right branch of E, as shown in Figure 21(c).

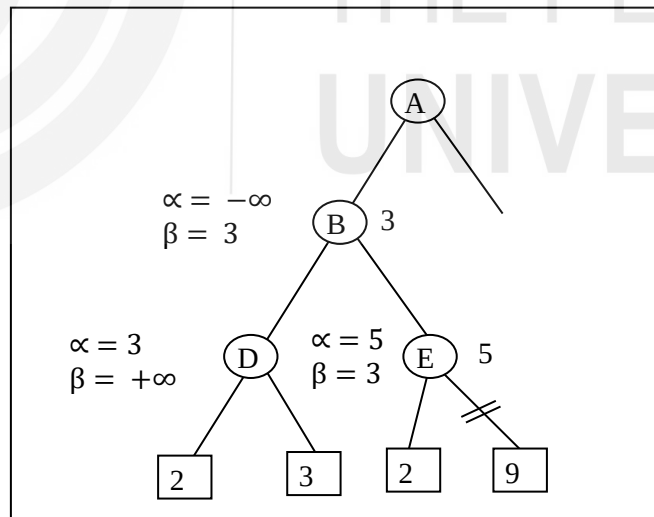


Fig. 21(c)

Step 4: We backtrack from node E to B. Note that, while backtracking, the node values of E(=5) will be passed to the upper node B, instead of the values of α and β . E returns a value 5 to B. Since B is at MIN level, so only β value will be changed. Previously, at node B $[\alpha = -\infty, \beta = 3]$, but now $\beta = \min(3, 5) = 3$, so, there is no change in β value and value of node(B) is still 3. Thus, finally, the modified value at node B is $[\alpha = -\infty, \beta = 3]$.

We backtrack from node B to A. Again, note that, while backtracking the tree, the value of node(B)=3 will be passed to the upper node A, instead of the values of α and β . **Now value of Node(A)=3.**

Since A is at MAX level, so only α value will be changed. Previously, at node A [$\alpha = -\infty, \beta = +\infty$] and after comparing value of node(B)=3 with old value of α at node A, that is $\alpha = \max(-\infty, 3) = 3$. Thus, finally, at node A [$\alpha = 3, \beta = +\infty$] and value of Node(A)=3. we check $\alpha \geq \beta$, it is False, so proceed on the right side. Now, we have completed the left sub-tree of A and proceed towards the right subtree, as shown in Figure 21(d).

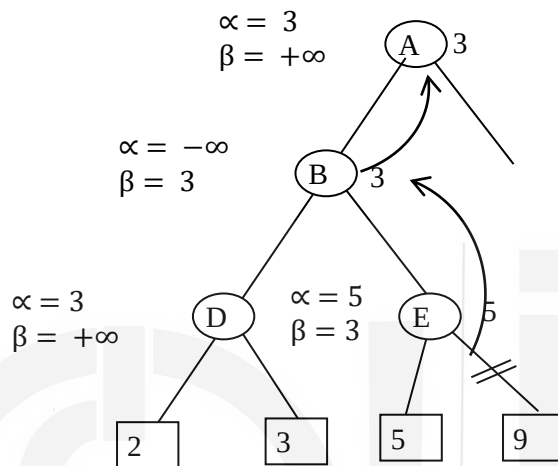


Fig. 21(d)

Step 5. Now at node C, we pass the α and β value from top node A to bottom node C as [$\alpha = 3, \beta = +\infty$]. **Check,** $\alpha \geq \beta$. The answer is NO. So, the search is continued. Now pass the α and β value from top node C to bottom node F as [$\alpha = 3, \beta = +\infty$]. Since F is at MAX level, so only α value will be changed. Now, at F, it first checks the left child (which is a terminal node) with value 0. This node returns a value of 0. Now, the value of α at node F is calculated as $\alpha = \max(3, 0) = 3$. So the modified value at node F is [$\alpha = 3, \beta = +\infty$]. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is NO since $3 \not\geq +\infty$. So, proceed and search continues for the right child of Node F.

The value of the right child (terminal node with value = 1) of F returns a value 1, so finally, the value of node(F)=1. Now at F, the value of α is compared with the terminal node value 1, that is, $\alpha = \max(3, 1) = 3$, and the final values of α and β is updated at node F as [$\alpha = 3, \beta = +\infty$] as shown in figure 21(e).

Step 6. **We** backtrack from node F to C. Note that, while backtracking the tree, the node values of F(=3) will be passed to the upper node C. Now the value of **node(C) = node(F) = 1.**

Since C is at MIN level, so only β value will be changed. Previously, at node C [$\alpha = 3, \beta = +\infty$]. Now, the old value of $\beta = +\infty$ is compared with value of node(F) = node(C) = 1. That is, $\beta = \min(+\infty, 1) = 1$. Thus, finally, at node B [$\alpha = 3, \beta = 1$]. **Now we check,** $\alpha \geq \beta$. It is TRUE, so we prune (cut) the right branch of node C. That is, node G will be pruned, and the algorithm stops searching on the right subtree of node C.

Thus, finally, **we** backtrack from node C to A, and node C returns the value 1 to node A. Since A is a MAX node, only α value will be changed.

Previously, at node A [$\alpha = 3, \beta = +\infty$] and after comparing value of node(C)=1 with old value of α at node A, that is $\alpha = \max(3, 1) = 3$. Thus, finally, at node A [$\alpha = 3, \beta = +\infty$] and value of Node(A)=3. Now, we have completed the right subtree of A also.

Following is the final game tree, showing the nodes that are computed and nodes that are pruned (cut) during the search process of Alpha-beta pruning. Here, the optimal value for the maximizer is 3, and there are 3 terminal nodes that are pruned (9, 7, and 5). The optimal search path is $A \rightarrow B \rightarrow D \rightarrow 3$.

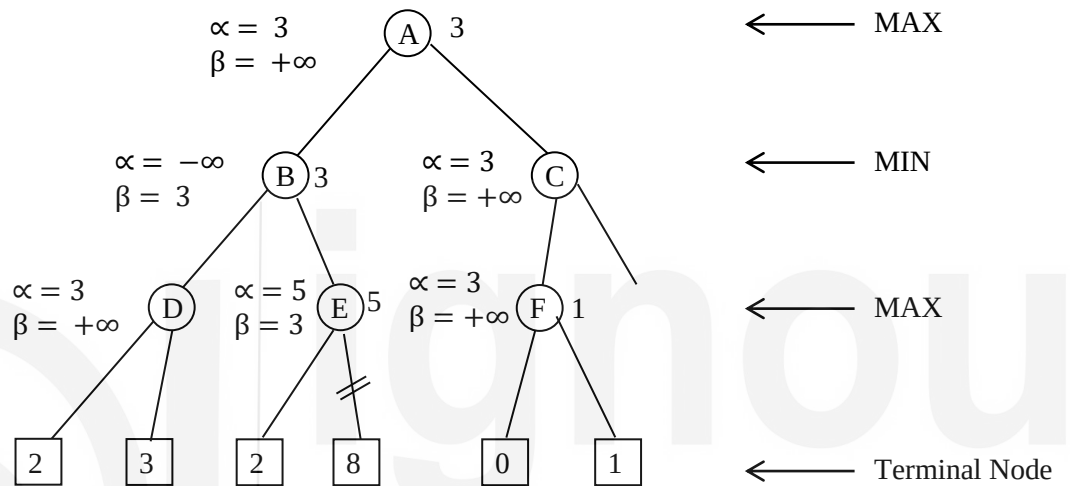


Fig. 21(e)

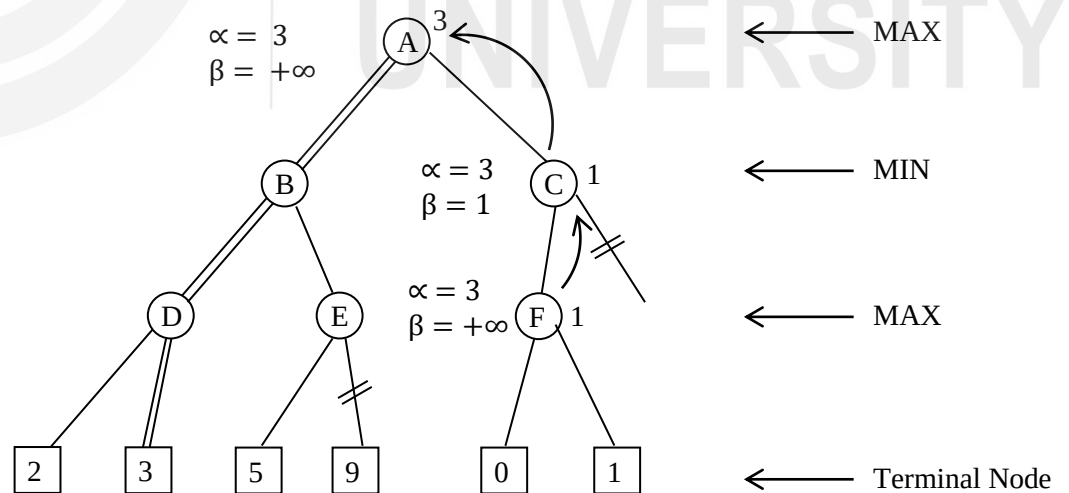


Fig. 21(f)

Example 2: Consider the following game tree (Figure 22) in which the root is the maximizing node and children are visited from left to right. Find which nodes are pruned by the Alpha-beta pruning.

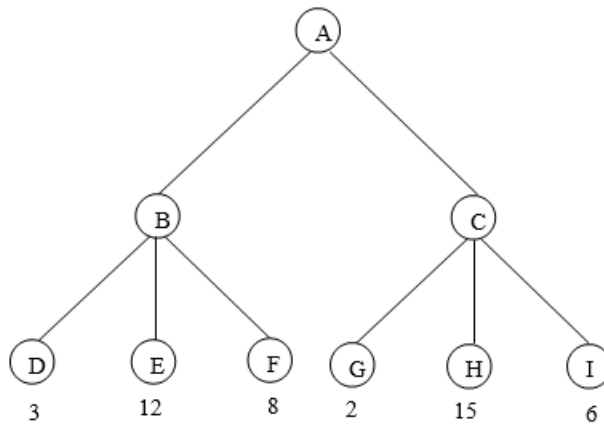


Fig. 22: Two Player search tree

Solution:

Step 1: We start the search by setting the initial value for node A as $[\alpha = -\infty, \beta = +\infty]$. We traverse the node in a depth-first search (DFS) manner, so we assign the same value of α and β to node B $[\alpha = -\infty, \beta = +\infty]$. Since node B is at MIN level, so only β value will be changed at node B. Now, at D, it looks at its left child (terminal node), which returns a value 3 to D. So, we compare the old value of β at node B with this terminal node value 3, that is $\beta = \min(+\infty, 3) = 3$. So modified value of α and β to node B is $[\alpha = -\infty, \beta = 3]$.

To decide whether it is worth looking at the right subtree of B, we check $\alpha \geq \beta$. The answer is NO, since $-\infty \not\geq 3$. So, proceed and search continues for the right child of Node B, that is E.

The terminal value of E = 12. Now, the value of right child terminal E(=12) is compared with the previous old value of $\beta = 3$, that is, $\beta = \min(3, 12) = 3$. So no change in β value. So, at present, the current modified value of α and β at node B is the same $[\alpha = -\infty, \beta = 3]$. Again check, $\alpha \geq \beta$. The answer is NO. So, proceed and search continues for the right child of Node B, that is, F. The terminal value of F=8. The value of the right child terminal at F(=8) is compared with the previous old value of $\beta = 3$, that is, $\beta = \min(8, 3) = 3$. So no change in β value. So, finally, the value of **Node(B)=3** and the modified value of α and β at node B $[\alpha = -\infty, \beta = 3]$ as shown in Figure 22(a)

Step 2: We backtrack from node B to A. Note that, while backtracking the tree, the node values of B(=3) will be passed to the upper node A, instead of the values of α and β . **Now the value of node(A) = node(B) = 3.** Since A is at the MAX level, so only α value will be changed. Previously, at node A $[\alpha = -\infty, \beta = +\infty]$ and after comparing value of node(B)=3 with old value of α at node A, that is $\alpha = \max(-\infty, 3) = 3$. Thus, finally, at node A $[\alpha = 3, \beta = +\infty]$ and value of **Node(A)=3**.

To decide whether it's worth looking at the right node of A or not, we check $\alpha \geq \beta$. The answer is NO since $3 \not\geq +\infty$. So, proceed and search continues for the right child of Node A. Now, we have completed the left sub-tree of A and proceed towards the right subtree.

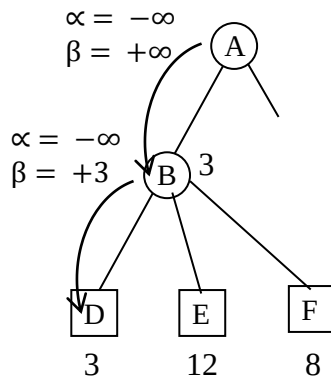


Fig22(a)

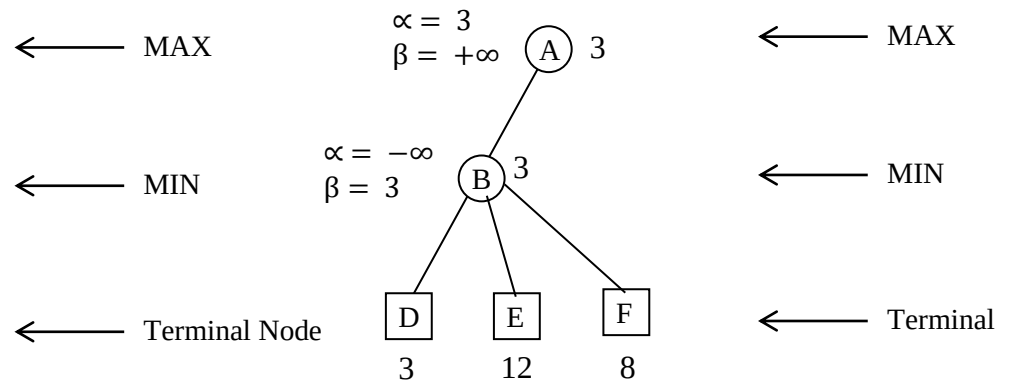


Fig 22(b)

Step 3: Now, at node C, we pass the α and β value from top node A to bottom node C as $[\alpha = 3, \beta = +\infty]$. **Check,** $\alpha \geq \beta$. The answer is NO. So, continue the search on the right side. Since C is at MIN level, so only β value will be changed.

Now, we first check the left child (terminal) of node C, that is, G=2. So, we compare the old value of β at node C with this terminal node value 2, that is $\beta = \min(+\infty, 2) = 2$. So value of Node(C)=2 and modified value of α and β at node C is $[\alpha = 3, \beta = 2]$. Now, before proceeding to the next, we again check $\alpha \geq \beta$. The answer is YES. So, we prune (cut) the right branch of node C. That is **node H, and I** will be pruned (cut), and the algorithm stops searching on the right subtree of node C, as shown in Figure 22(c).

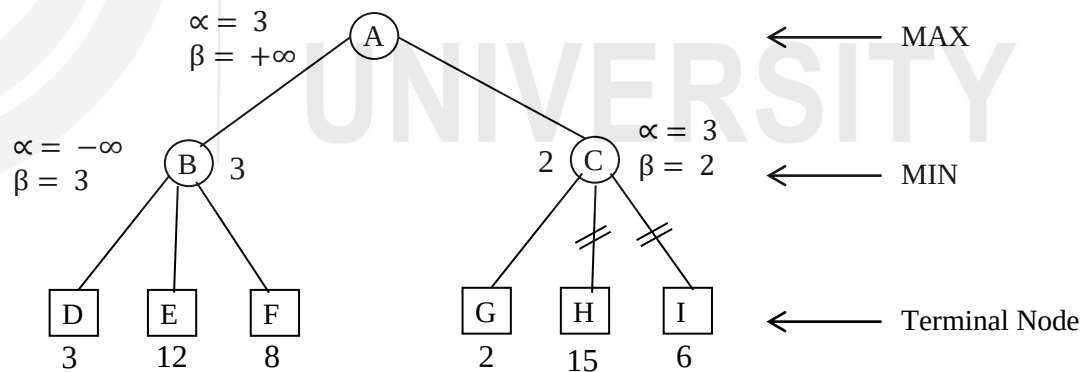


Fig 22(c)

Step 4: Finally, we backtrack from node C to A. Note that, while backtracking the node in a tree, the node values of C(=2) will be passed to the upper node A, instead of the values of α and β . The previous node(A)=3 value is compared with this new node(C)=2 value. The best value at node(A)= $\alpha = \max(3, 2) = 3$.

The previous α and β value at node A $[\alpha = 3, \beta = +\infty]$. Since A is at MAX level, only α value is change. So, we compare old $\alpha = 3$ value with value at node(C)=2. That is $\alpha = \max(3, 2) = 3$. Thus, there is no change in α value as well.

Thus finally, α and β value at node A is $[\alpha = 3, \beta = +\infty]$ and value of Node(A)=3. So, the optimal value for the maximizer is 3, and there are 2 terminal nodes that are pruned (H and I). The optimal search path is $A \rightarrow B \rightarrow D$ (as shown in Figure 22(d)).

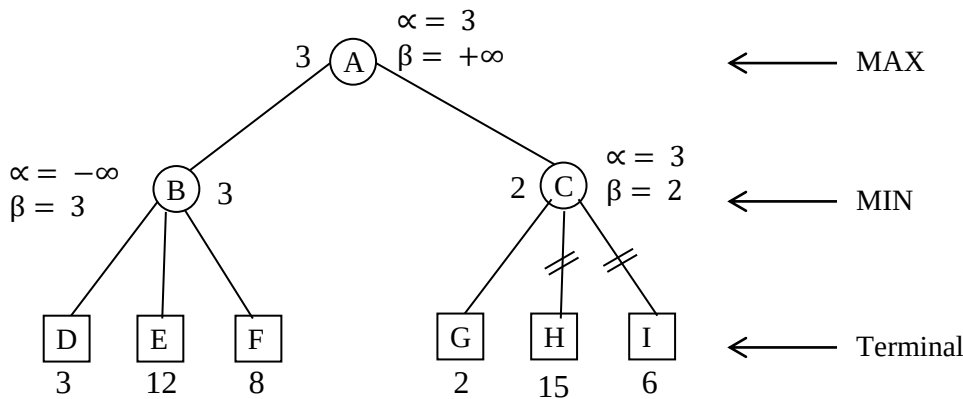


Fig 22(d)

2.7.2 Move ordering of Alpha-beta pruning

The effectiveness of Alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning. We have two types of move ordering:

Worst-case ordering: In some cases, the alpha-beta pruning algorithm does not prune any of the leaves of the tree and works exactly as the MiniMax algorithm. In this case, it consumes more time because of alpha-beta factors; such a move of pruning is called a worst ordering. The time complexity for such an order is $O(b^m)$ where b: Branching Factor and m is the depth of the tree.

Best (ideal) case ordering: The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and the best move occurs at the left side of the tree. We apply DFS, hence it first searches left of the tree and goes deep twice as the minimax algorithm does in the same amount of time. The time complexity for the best-case order is $O(b^{m/2})$ (since we search only the left sub-tree, not the right subtree).

Note that pruning does not affect the final result. Good move ordering improves the effectiveness of pruning. With ideal case ordering, the time complexity is $O(b^{\frac{m}{2}})$

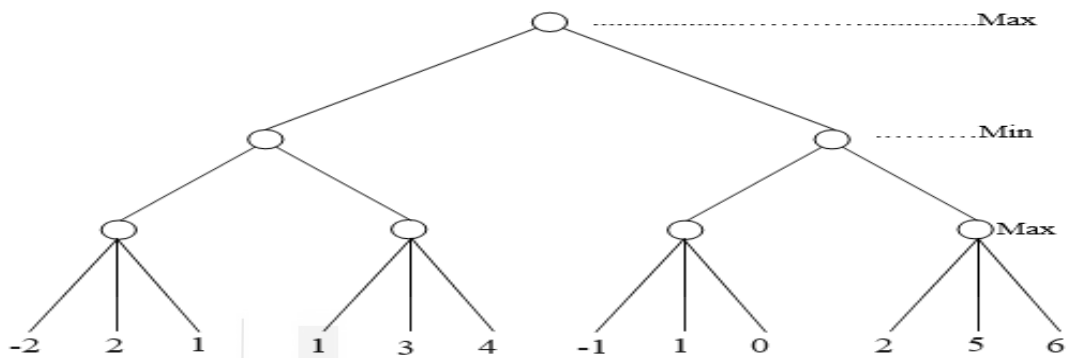
In Alpha-beta pruning:

- α value can never decrease and β value can never increase. Search can be discontinued at a node if:
- It is a Max node and $\alpha \geq \beta$ it is beta cutoff
- It is a Min node and $\beta \leq \alpha$ it is alpha cutoff.

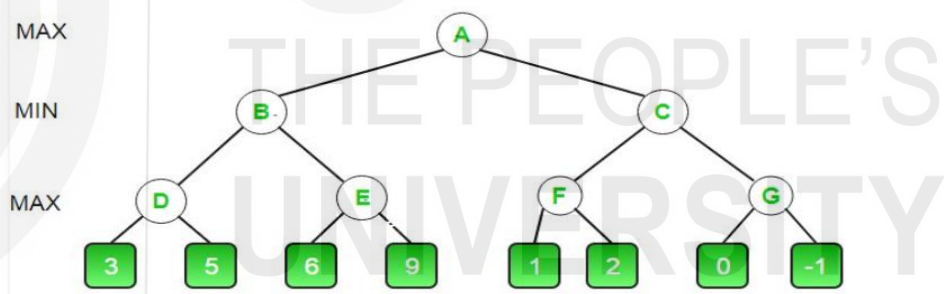
Check Your Progress 2

Q1. Compare the MINIMAX and Alpha-Beta Pruning algorithm with respect to Time complexity.

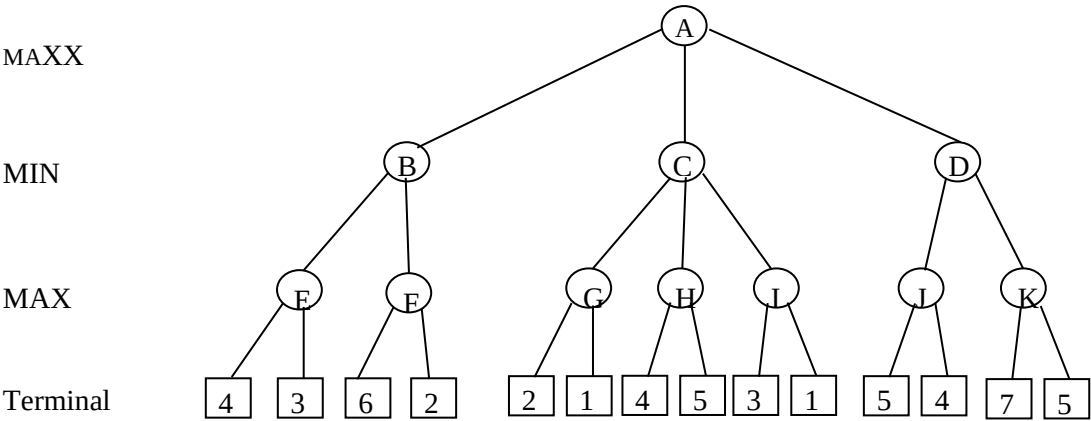
Q2. Consider the following Minimax game tree search in which the root is the maximizing node, and the children are visited from left to right. Find the value of the root node of the game tree?



Q3. Apply Alpha-Beta pruning algorithm on the following graph and find which node(s) are pruned?



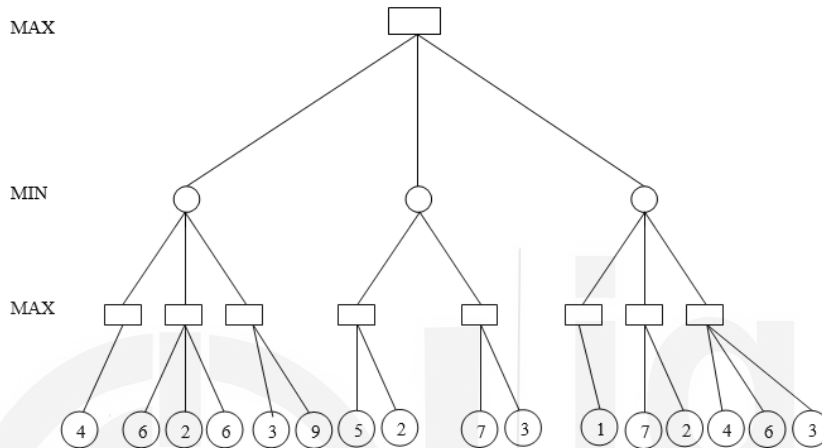
Q4. Consider the following Minimax game tree search in which the root is the maximizing node, and children are visited from left to right.



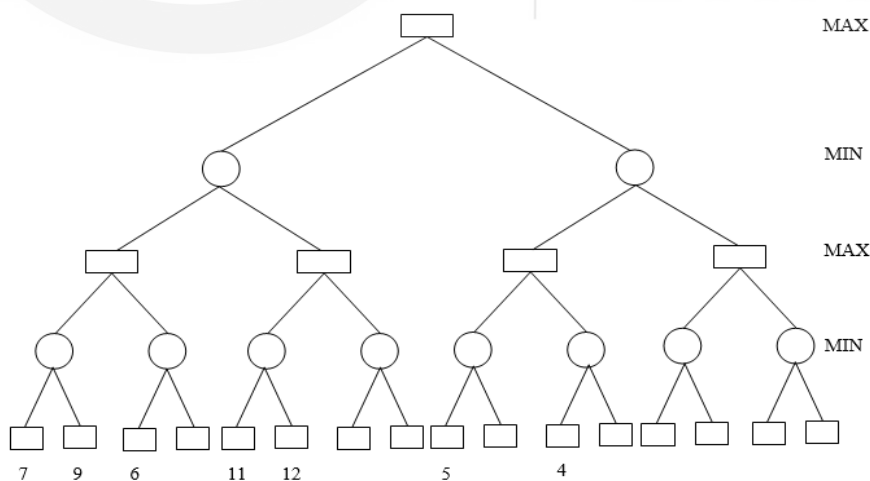
- (a) Find the value of the root node of the game tree?
 (b) Find all the nodes pruned in the tree?
 (c) Find the optimal path for the maximizer in a tree?

Problem Solving Using
Search

Q5. Consider the following Minimax game tree search in which the root is maximizing node, and the children are visited from left to right. Find what will be the value propagated at the root?

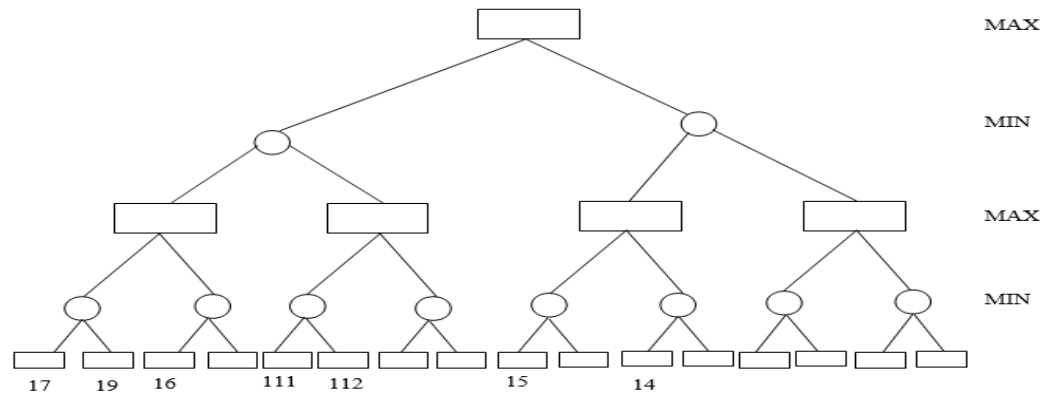


Q6. Consider the following Minimax game tree search in which root is maximizing node and children are visited from left to right. Find the value of the root node of the game tree?



Multiple Choice Question

Q7. Consider the following Minimax game tree search in which root is maximizing node and children are visited from left to right. Find the value of the root node of the game tree?



A. 14

B. 17

C. 111

D. 112

2.8 SUMMARY

- Before an AI problem can be solved, it must be represented in a **state space**. Among all possible states, there are two special states called the **initial state** (the start point) and the **final state** (the goal state).
- A **successor function (a set of operators)** is used to change the state. It is used to move from one state to another.
- A state space is a set of all possible states of a problem.
- A **state space** essentially consists of a set of nodes representing each state of the problem, arcs between nodes representing the legal moves from one state to another, an initial state, and a goal state. Each state space takes the form of a tree or a graph.
- The process of searching means a sequence of actions that takes you from an initial state to a goal state.
- Search is fundamental to the problem-solving process. Search means the problem is solved by using the rules, in combination with an appropriate **control strategy**, to move through the problem space until a path from an **initial state** to a **goal state** is found.
- A **problem space** is represented by a directed graph, where *nodes* represent *search states*, and *paths* represent the *operators* applied to change the state.
- In general, a state space is represented by 4 tuples as follows: $S_s: [S, s_0, O, G]$, Where **S**: Set of all possible states (possibly infinite), **s_0** : start state (initial configuration) of the problem, $s_0 \in S$. **O**: Set of production rules (or a set of state transition operators) used to change the state from one state to another. It is the set of arcs (or links) between nodes.
- Adversarial search is a game-playing technique where the agents are surrounded by a competitive environment. A conflicting goal is given

to the agents (multi-agent). These agents compete with one another and try to defeat one another in order to win the game. Such conflicting goals give rise to adversarial search.

- In a **normal search**, we follow a sequence of actions to reach the goal or to finish the game optimally. However in an **adversarial search**, the result depends on the players who will decide the result of the game. It is also obvious that the solution for the goal state will be an optimal solution because the player will try to win the game with the shortest path and under limited time.
- There are 2 types of adversarial search: **Minimax Algorithm** and **Alpha-beta Pruning**.
- Minimax is a two-player (namely MAX and MIN) game strategy where *if one wins, the other loses the game*. This strategy simulates those games that we play in our day-to-day life. Like, if two people are playing chess, the result will be in favour of one player and will go against the other one. **MIN**: Decreases the chances of **MAX** winning the game, and **MAX**: Increases his chances of winning the game. They both play the game alternatively, i.e., turn by turn, and following the above strategy, i.e., if one wins, the other will definitely lose it. Both players look at one another as competitors and will try to defeat one another, giving their best.
- In the minimax strategy, the result of the game or the utility value is generated by a **heuristic function** by propagating from the initial node to the root node. It follows the **backtracking technique** and backtracks to find the best choice. MAX will choose that path which will increase its utility value, and MIN will choose the opposite path, which could help it to minimize MAX's utility value.
- The drawback of minimax strategy is that it explores each node in the tree deeply to provide the best path among all the paths. This increases its time complexity.
- If b is the branching factor and d is the depth of the tree, then the time complexity of the MINIMAX algorithm is $O(b^d)$ that is exponential.
- Alpha-beta pruning is an advanced version of the MINIMAX algorithm. Therefore, alpha-beta pruning reduces the drawback of the minimax strategy by exploring the nodes of the search tree.
- The alpha-beta pruning method **cuts off the search** by exploring a smaller number of nodes. It makes the same moves as a minimax algorithm does, but it prunes the unwanted branches using the pruning technique.
- Alpha-beta pruning works on two threshold values, i.e., α (**alpha**) and β (**beta**). α : It is the highest value a **MAX** player can have. The initial value of α is set to a negative infinity value, that is $\alpha = -\infty$. As the algorithm progresses, its value may change and finally get the best (highest) value. β : It is the best lowest value a **MIN** player can have. The initial value of β is set to a positive infinity value, that

is $\beta = +\infty$. As the algorithm progresses, its value may change and finally get the best (lowest) value.

- So, each MAX node has α value, which never decreases, and each MIN node has β value, which never increases. The main condition required for alpha-beta pruning is $\alpha \geq \beta$, that is if $\alpha \geq \beta$, then prune (cut) the branches; otherwise, the search continues.
- As we know, there are two-parameter is defined for Alpha-beta pruning, namely alpha (α) and beta (β). The initial value of alpha and beta is set to $\alpha = -\infty$ and $\beta = +\infty$. As the algorithm progresses, its values are changed accordingly. Note that in Alpha-beta pruning (cut), at any node in a tree, if $\alpha \geq \beta$, then prune (cut) the next branch else search is continued.
- The effectiveness of Alpha-beta pruning is highly dependent on the order in which each node is examined.
- **Worst-case ordering:** In some cases, the alpha-beta pruning algorithm does not prune any of the leaves of the tree and works exactly as the MiniMax algorithm. In this case, the time complexity is $O(b^m)$ where b is the branching Factor and m is the depth of the tree.
- **Best (ideal) case ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and the best move occurs on the left side of the tree. The time complexity for the best-case order is $O(b^{m/2})$ (since we search only the left sub-tree, not the right subtree).

2.9 SOLUTIONS/ANSWERS

Check your progress 1:

Answer 1: A number of factors need to be taken into consideration when developing a state-space representation. Factors that must be addressed are:

- What is the goal to be achieved?
- What are the legal moves or actions?
- What knowledge needs to be represented in the state description?
- Type of problem - There are basically three types of problems. Some problems only need a representation, e.g., crossword puzzles. Other problems require a yes or no response, indicating whether a solution can be found or not. Finally, the last type of problems are those that require a solution path as an output, e.g., mathematical theorems, Towers of Hanoi. In these cases, we know the goal state, and we need to know how to attain this state

- Best solution vs. Good enough solution - For some problems, a good enough solution is sufficient. For example: theorem proving eight squares.
However, some problems require a best or optimal solution, e.g., the traveling salesman problem.

Answer 2

(a) Formulation of the Missionaries and Cannibals problem:

State: (#M, #C, 0/1)

Where #M represents the number of missionaries in the left side bank (i.e., left side of the river)

#C: represents the number of cannibals in the left side bank (i.e., left side of the river)

0/1: indicate the boat position of the boat. 0 indicates the boat is on the left side of the river, and 1 indicates the boat is on the right side.

Start state:(3,3,0)

Goal State: (0,0,1)

Operator: State will be changed by moving missionaries and/or cannibals from one side to another using a boat. So, it can be represented as the number of persons on either side of the river. Note that the boat can carry a maximum of 2 persons.

Boat carries: (1,0) or (0,1) or (1,1) or (2,0) or (0,2). Here in (i, j), i represents the number of missionaries and j means number of cannibals.

(b) Solution of the Missionaries and Cannibals problem:

Start state: (3,3,0)

Goal State: (0,0,1)

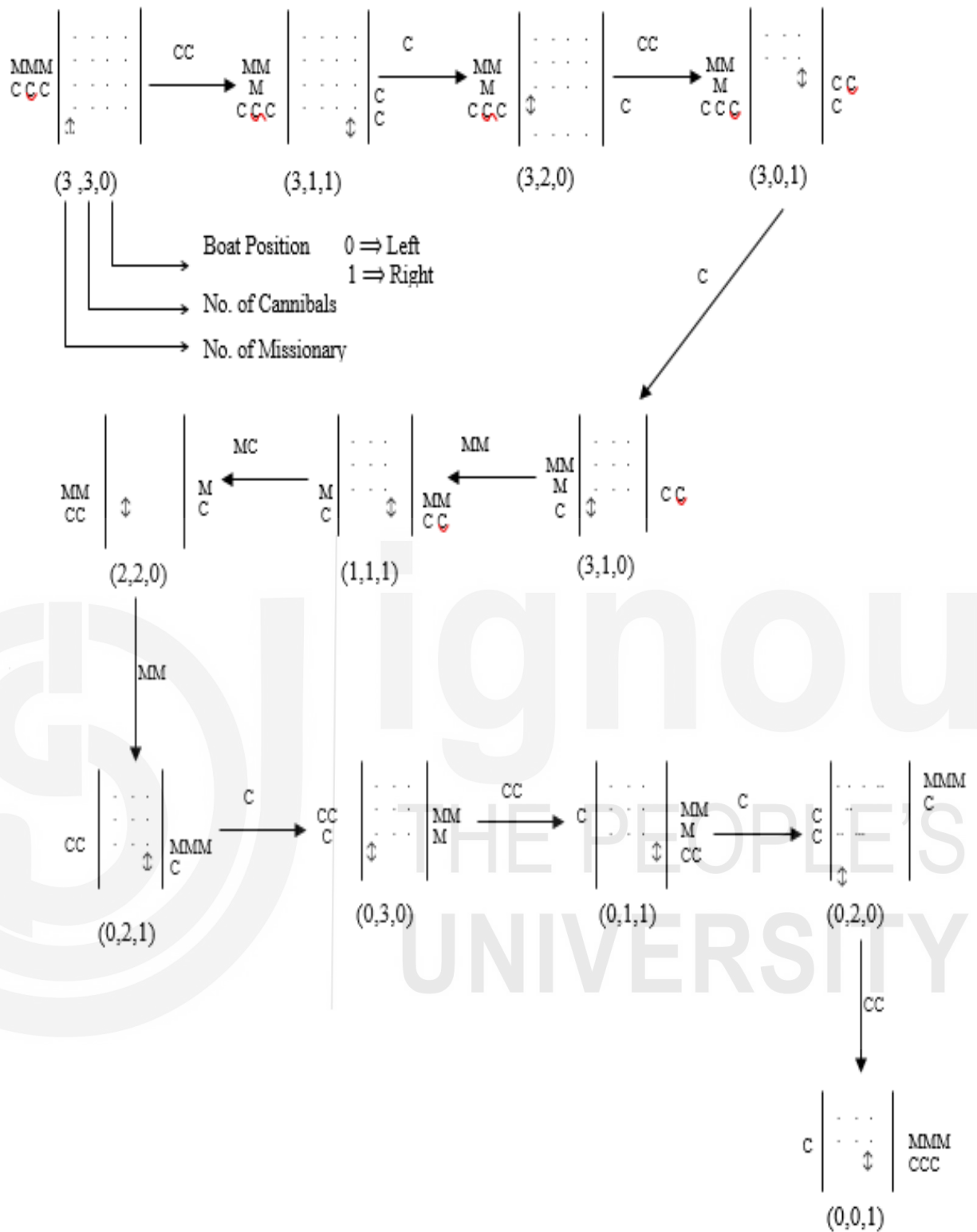


Figure: Solution of the Missionaries and Cannibals problem

A state space tree for this problem is shown below:

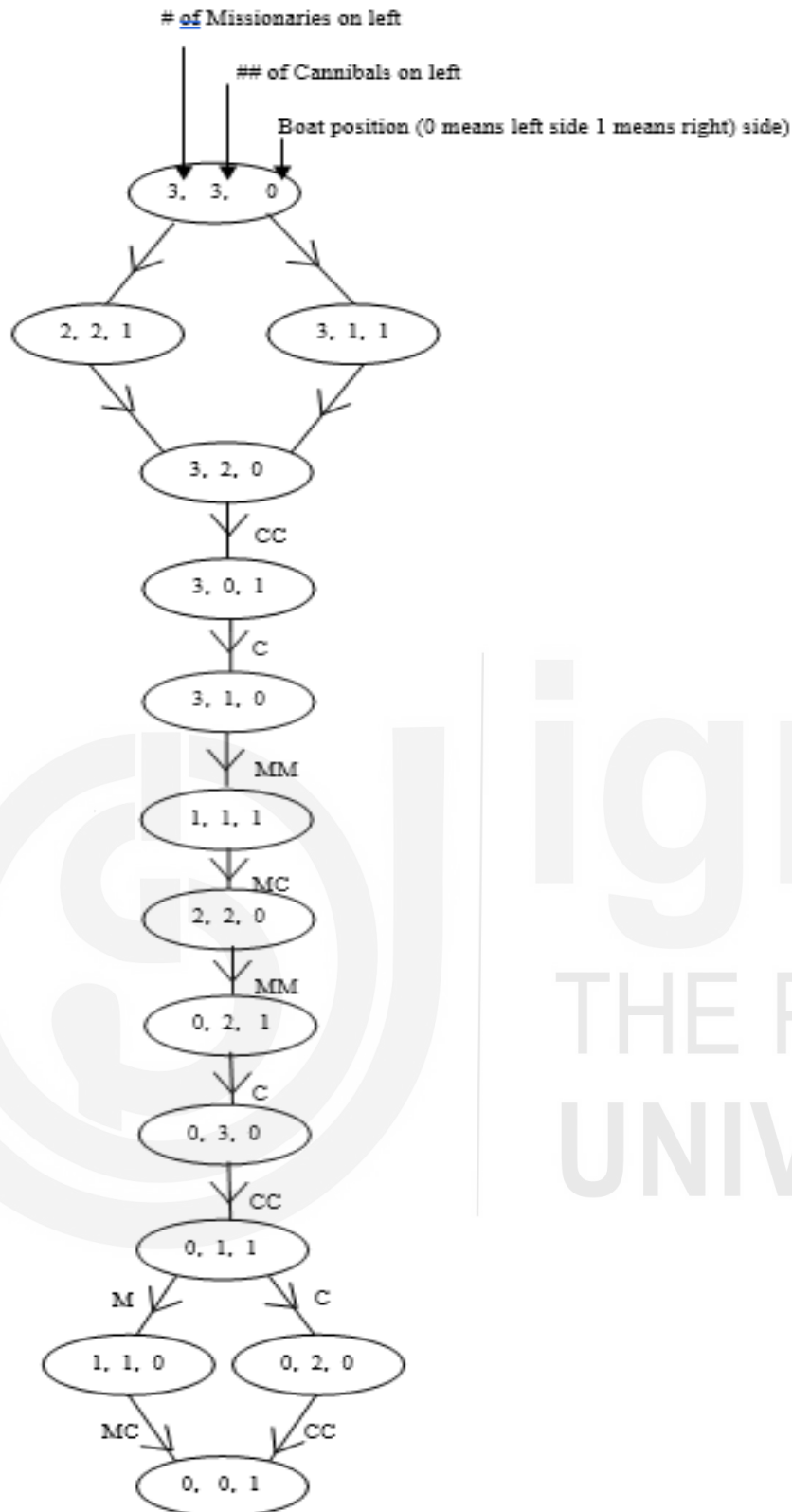


Figure: A state space tree showing all possible solutions of the missionaries and cannibal problem.

Answer 3: Towers Hanoi A possible state space representation of the Towers Hanoi problem using a graph is indicated in **Figure 1**.

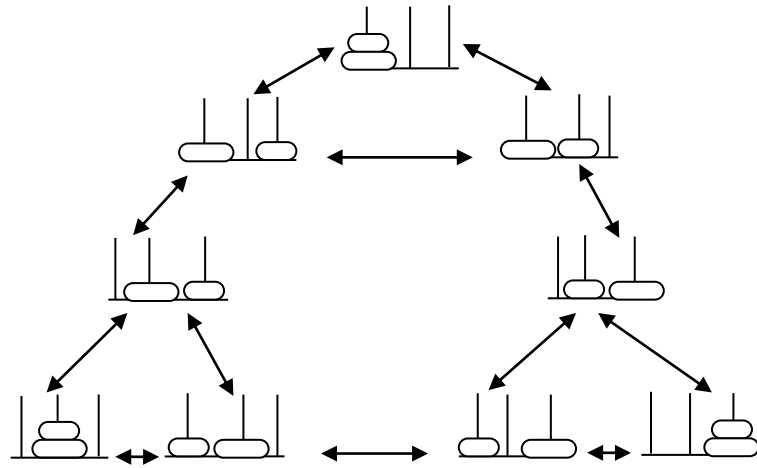
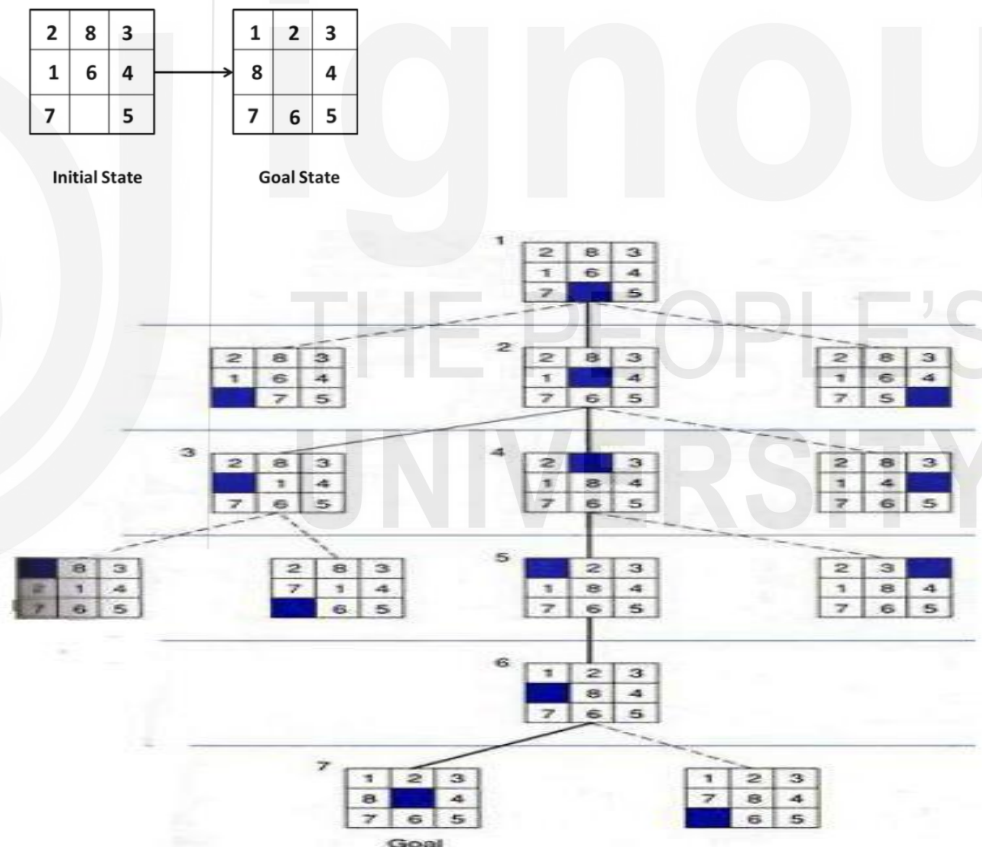


Figure 1: Towers of Hanoi state space representation for n=2

The legal moves in this state space involve moving one ring from one pole to another, moving one ring at a time, and ensuring that a larger ring is not placed on a smaller ring.

Answer 4:



Minimum Cost path for solution= 6

Answer 5: Backtracking Algorithm: The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return false.

- 1) Start in the leftmost column
- 2) If all queens are placed
return true
- 3) Try all rows in the current column.
Do following for every tried row.

a) If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.

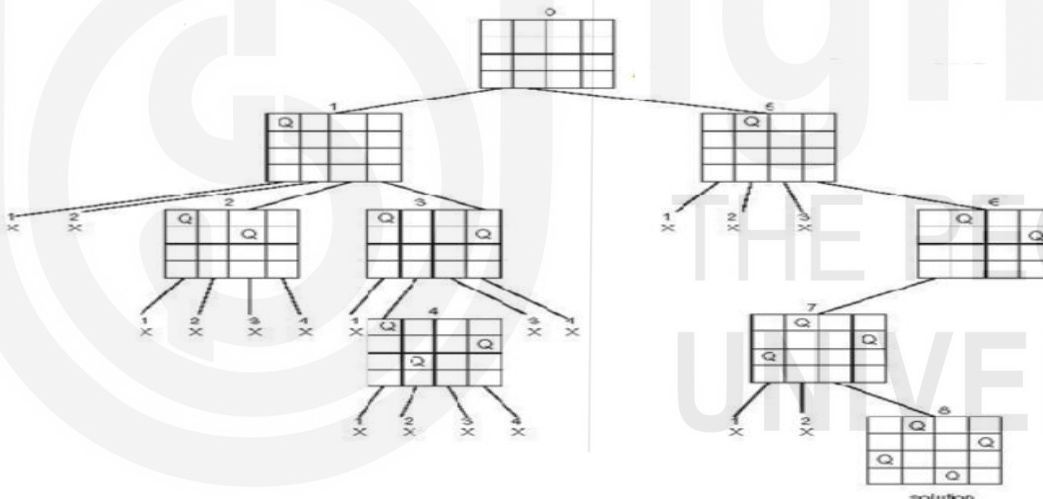
b) If placing the queen in [row, column] leads to a solution then return true.

c) If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.

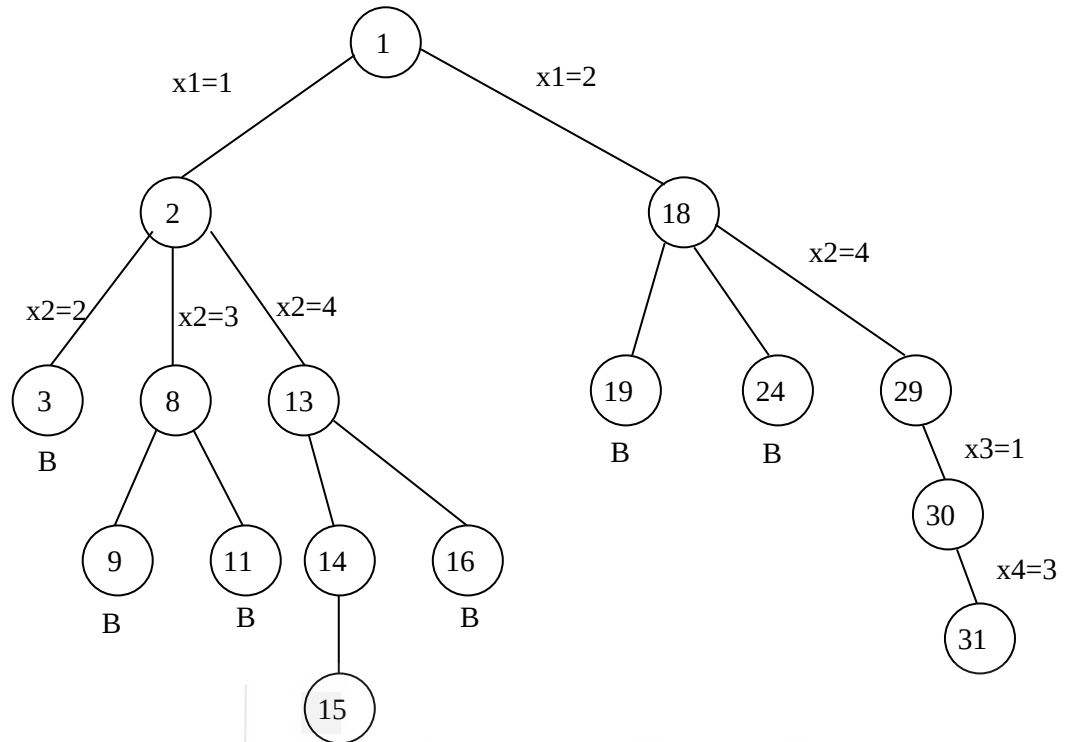
4) If all rows have been tried and nothing worked, return false to trigger backtracking.

State space tree for 4 Queen's problem

A state-space tree (SST) can be constructed to show the solution to this problem. The following SST (figure 1) shows one possible solution $\{x_1, x_2, x_3, x_4\} = \{2, 4, 3, 1\}$ for the 4 Queen's Problem.



Or we can also denote the State space tree as follows:



B denotes the Dead Node (nonpromising node). The figure 1 shows the Implicit tree for 4 queen problem for solution $\langle 2, 4, 1, 3 \rangle$. The Root represents an initial state. The Nodes reflect the specific choices made for the components of a solution. Explore The state space tree using **depth-first search**. "Prune" non-promising nodesdfs stops exploring subtree rooted at nodes leading to no solutions and then backtracks to its parent node

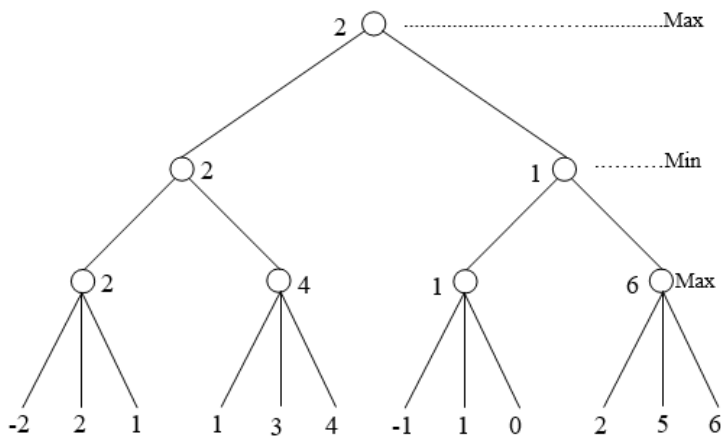
Check your progress 2

Answer1:

Solution: Alpha-beta pruning is an advance version of MINIMAX algorithm. The drawback of minimax strategy is that it explores each node in the tree deeply to provide the best path among all the paths. This increases its time complexity. If b is the branching factor and d is the depth of the tree, then time complexity of MINIMAX algorithm is $O(b^d)$ that is exponential.

Alpha-Beta pruning is a way of finding the optimal Minimax solution while avoiding searching subtrees of moves which won't be selected. The effectiveness of Alpha-beta pruning is highly dependent on the order in which each node is examined. **The** ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best move occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. The time complexity for best case order is $O(b^{m/2})$ (since we search only left sub tree, not a right subtree).

Answer 2: The final game tree with max and min value at each node is shown in the following figure.



Answer3:

Solution: Solve the question as shown in Example 1

The initial call starts from A. We initially start the search by setting the initial value of $\alpha = -\infty$ and $\beta = +\infty$ for root node A. These values are passed down to subsequent nodes in the tree. At A the maximizer must choose max of B and C, so A calls B first. At B it the minimizer must choose min of D and E and hence calls D first. At D, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at D is $\max(-\infty, 3)$ which is 3. To decide whether it's worth looking at its right node or not, it checks the condition $\alpha \geq \beta$. This is false since $\beta = +\infty$ and $\alpha = 3$. So, it continues the search.

D now looks at its right child which returns a value of 5. At D, $\alpha = \max(3, 5)$ which is 5. Now the value of node D is 5. Value at node D=5, move up to node B(=5). Now at node B, β value will be modified as

$$\beta = \min(+\infty, 5) = 5.$$

B now calls E, we pass the α and β value from top node B to bottom node E as $[\alpha = -\infty, \beta = 5]$. Since, node E is at MAX level, so only α value will be change. Now, at E, it first checks the left child (which is a terminal node) with value 6. This node returns a value of 6.

Now, the value of α at node E is calculated as $\alpha = \max(-\infty, 6) = 6$, so value of Node(E)=6 and modified value of α and β at node E is $[\alpha = 6, \beta = 5]$. To decide whether it's worth looking at its right node or not, we check $\alpha \geq \beta$. The answer is **YES**, since $6 \geq 5$. So, we prune (cut) the right branch of E, as shown in figure (a).

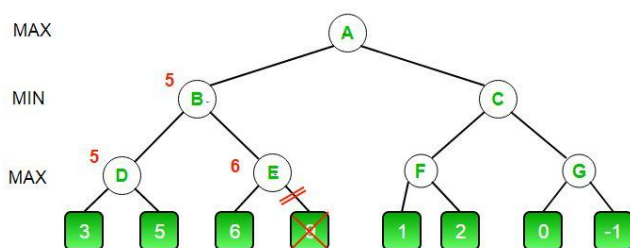


Figure (a) game tree after applying alpha-beta pruning on left side of node A

Similarly, we solve for right sub tree for Node A [refer the example 1 and solve for right sub tree part]. The final tree with node value at every node is shown in the figure(b).

Thus finally, α and β value at node A is $[\alpha = 5, \beta = +\infty]$ and best value at Node(A) = $\max(5, 2) = 5$. So, optimal value for the maximizer is 5 and there are 3 terminal nodes are pruned (9, 0 and -1). The optimal search path is $A \rightarrow B \rightarrow D \rightarrow 5$, as shown in figure (b).

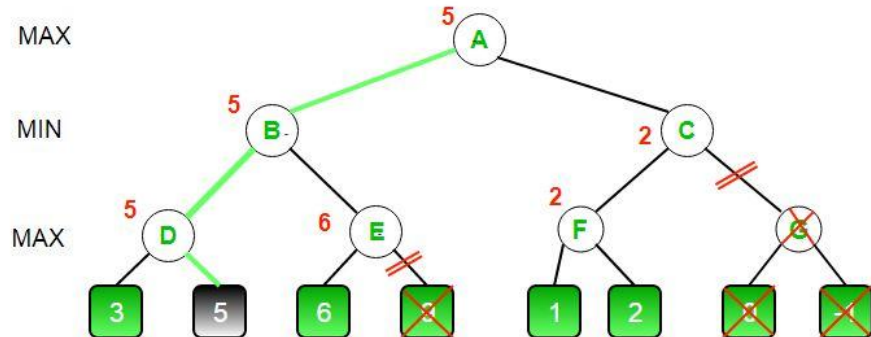
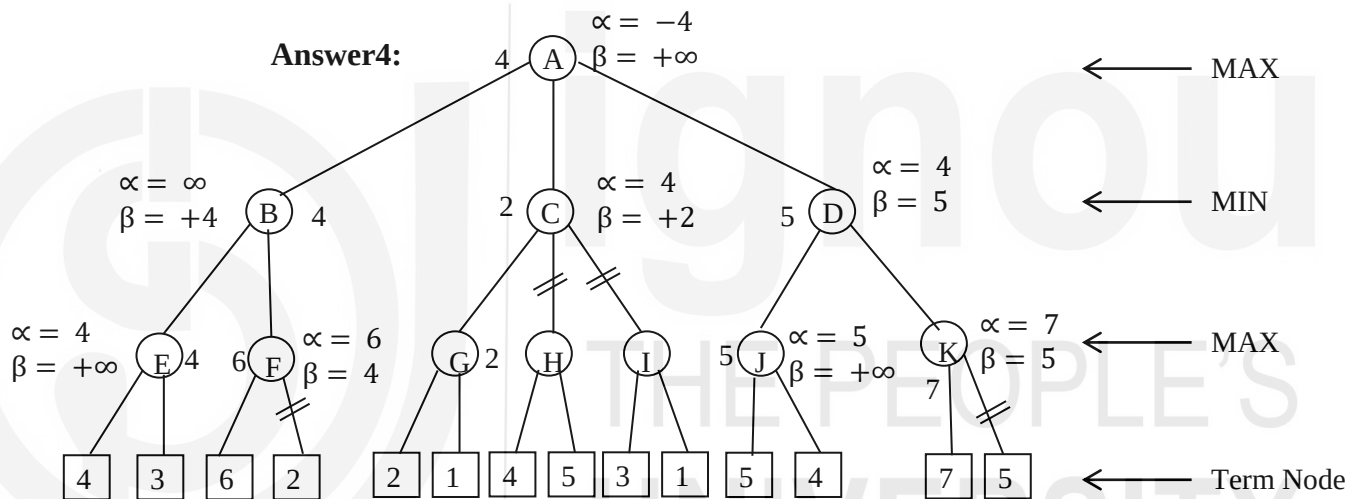


Figure (b) Final tree with node value on every node with prune branches



Answer5: 5

Answer6: 7

Answer 7: Option (B) 17

2.11 FURTHER READINGS

1. Ela Kumar, "Artificial Intelligence", IK International Publications
2. E. Rich and K. Knight, "Artificial intelligence", Tata Mc Graw Hill Publications
3. N.J. Nilsson, "Principles of AI", Narosa Publ. House Publications
4. John J. Craig, "Introduction to Robotics", Addison Wesley publication
5. D.W. Patterson, "Introduction to AI and Expert Systems" Pearson publication

UNIT 3 UNINFORMED & INFORMED SEARCH

Structure	Page No
3.0 Introduction	92
3.1 Objectives	94
3.2 Formulating search in state space	95
3.2.1 Evaluation of the Search Algorithm	97
3.3 Uninformed Search	97
3.3.1 Breath-First search (BFS)	98
3.3.2 Time and space complexity of BFS	101
3.3.3 Advantages & disadvantages of BFS	102
3.3.4 Depth First Search (DFS)	104
3.3.5 Performance of DFS algorithm	104
3.3.6 Advantages and disadvantages of DFS	104
3.3.7 Comparison of BFS and DFS	105
3.4 Iterative Deepening Depth First search (IDDFS)	106
3.4.1 Time and space complexity of IDDFS	108
3.4.2 Advantages and Disadvantages of IDDFS	108
3.5 Bidirectional search	109
3.6 Comparison of Uninformed search strategies	111
3.7 Informed (heuristic) search	112
3.7.1 Strategies for providing heuristics information	113
3.7.2 Formulation of Informed (heuristic) search problem as a state space	114
3.7.3 Best-First search	115
3.7.4 Greedy Best first search	115
3.8 A* Algorithm	120
3.8.1 Working of A* algorithm	122
3.8.2 Advantages and disadvantages of A* algorithm	124
3.8.3 Admissibility properties of A* algorithm	124
3.8.4 Properties of heuristic algorithm	125
3.8.5 Results on A* algorithm	126
3.9 Problem reduction search	126
3.9.1 Problem definition in AND-OR graph	127
3.9.2 AO* algorithm	130
3.9.3 Advantages of AO* algorithm	131
3.10 Memory Bound heuristic search	131
3.10.1 Iterative Deepening A* (IDA*)	131
3.10.2 Working of IDA*	132
3.10.3 Analysis of IDA*	133
3.10.4 Comparison of A* and IDA* algorithm	134
3.11 Recursive Best First search (RBFS)	134
3.11.1 Advantages and disadvantages of RBFS	135
3.12 Summary	137
3.13 Solutions/Answers	140
3.14 Further Readings	144

3.0 INTRODUCTION

Before an AI problem can be solved, it must be represented as a **state space**. In AI, a wide range of problems can be formulated as a search problem. The process of searching means a sequence of action that takes you from an initial state to a goal state, as shown in the following figure 1.



Fig 1: A sequence of actions in a search space from the initial to the goal state.

In Unit 2, we have already examined the concept of a **state space** and adversarial (game playing) search strategy. In many applications, there might be multiple agents or persons searching for solutions in the same solution space. In adversarial search, we need a path to take action towards the winning direction, and to find a path, we need different types of search algorithms.

Search algorithms are one of the most important areas of Artificial Intelligence. This unit will explain all about the search algorithms in AI, which explore the search space to find a solution.

In Artificial Intelligence, Search techniques are **universal problem-solving methods**. Rational agents or **Problem-solving agents** in AI mostly use these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are goal-based agents and use atomic representation.

One **disadvantage** of state space representation is that it is not possible to visualize all states for a given problem. Also, the resources of the computer system are limited to handle a huge state space representation. However, many problems in AI take the form of state-space search.

- The states may represent legal board configurations in a game, locations such as towns and cities in a route-finding problem, or collections of mathematical propositions.
- A state-space is the configuration of the possible states and how they connect to each other, e.g., the legal moves between states.
- When we don't have an algorithm that tells us definitively how to negotiate the state-space, we need to search the state-space to find an optimal path from a start state to a goal state,

We can only decide what to do (or where to go) by considering the possible moves from the current state and trying to look ahead as far as possible. Chess, for example, is a very difficult state space search problem.

Searching is the process of looking for the solution to a problem through a set of possibilities (state space). In general, the searching process starts from the initial state (root node) and proceeds by performing the following steps:

- Check whether the current state is the goal state or not?
- Expand the current state to generate the new sets of states.
- Choose one of the new states generated for search depending upon the search strategy (for example, BFS, DFS, etc.).
- Repeat steps 1 to 3 until the goal state is reached or there are no more state tubes expanded.

Evaluation (properties) of search strategies: A search strategy is characterized by the sequence in which nodes are expanded. Any search algorithm is commonly evaluated according to the following four criteria. The following four essential properties of search algorithms are used to compare the efficiency of any search algorithm.

Completeness: A search algorithm is said to be complete if it guarantees returning a solution if one exists.

Optimality/Admissibility: If the solution found by an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.

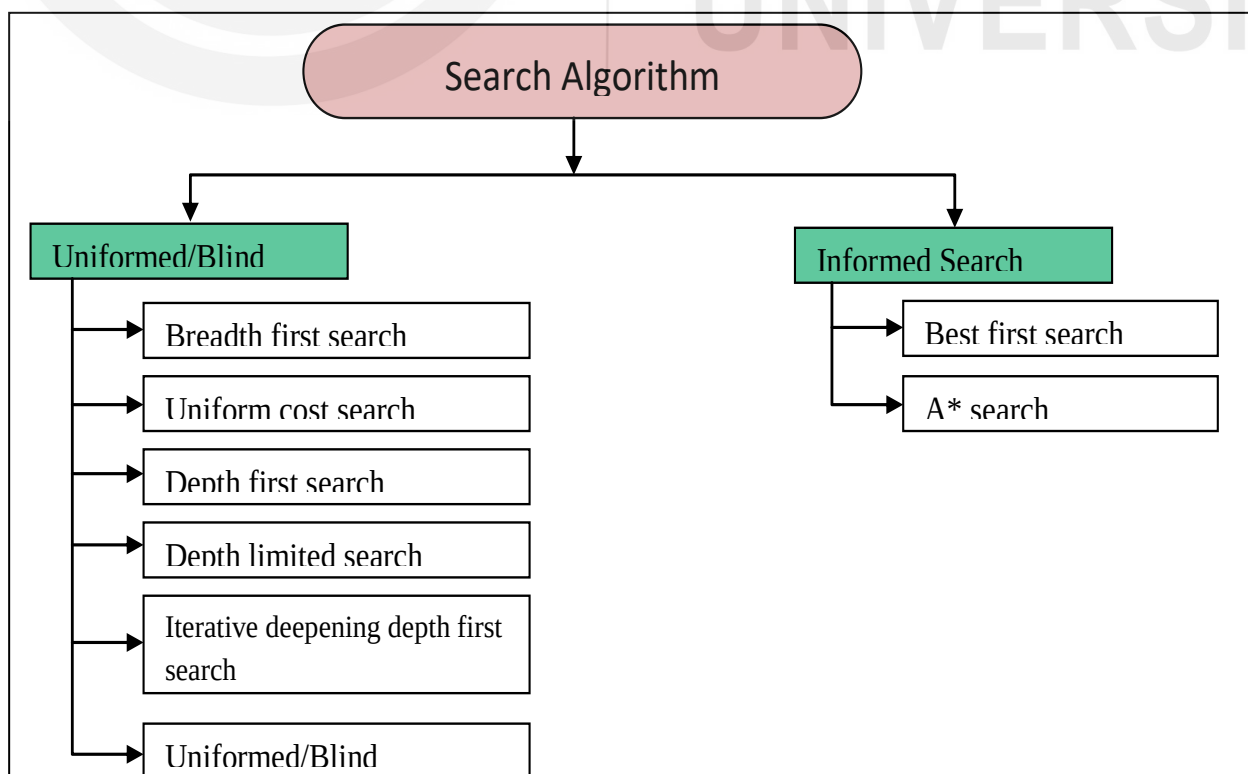
Time Complexity: Time complexity is a measure of the time required for an algorithm to complete its task. Usually measured in terms of the number of nodes expanded during the search.

Space Complexity: It is the maximum storage space required at any point during the search. Usually measured in terms of the maximum number of nodes in memory at a time.

Time and space complexity are measured in terms of:

- b - max branching factor of the search tree
- d - depth of the least-cost solution
- m - max depth of the search tree (may be infinity)

In all search algorithms, the order in which nodes are expanded distinguishes them from one another. There are two broad classes of search methods:



Uninformed search is also called brute force search, Blind search, or Exhaustive search. It is called blind search because of the way in which the search tree is searched without using any information about the search space. It is called brute force because it assumes no additional knowledge other than how to traverse the search tree and how to identify the leaf nodes and goal nodes. This search ultimately examines every node in the tree until it finds a goal state.

Informed search is also called a Heuristic (or guided) search. These are the search techniques where additional information about the problem is provided in order to guide the search in a specific direction. A heuristic is a method that might not always find the best solution, but is guaranteed to find a good solution in a reasonable time. By sacrificing completeness, it increases efficiency.

The following table summarizes the differences between uninformed and informed search:

Uninformed Search	Informed Search
No information about the path cost from the current state to the goal state. It doesn't use domain-specific knowledge during the search process.	The path cost from the current state to the goal state is calculated to select the minimum cost path as the next state. It uses domain-specific knowledge for the searching process.
It finds solutions more slowly than informed search	It finds solutions more quickly.
Less efficient	More efficient
Cost is high.	Cost is low
No guidance is provided regarding the solution; the problem must be solved using only the given information.	It provides guidance toward finding a solution. Additional information can be added as an assumption to solve the problem.
Examples are: ▪ Depth First Search, ▪ Breadth First Search, ▪ Depth limited search, ▪ Iterative Deepening DFS, ▪ Bi-directional search	Examples are: ▪ Best first search ▪ Greedy search ▪ A* search

3.1 OBJECTIVES

After studying this unit, you should be able to:

- Differentiate the Uninformed and informed search algorithms
- Formulate the search problem in the form of a state space
- Explain the differences between various uninformed search approaches, such as BFS, DFS, IDDFS, and Bi-directional search.
- Evaluate the various Uninformed search algorithms with respect to Time, space, and Optimality/Admissibility criteria.

- Explain Informed search, such as Best-First search and A* algorithm.
- Differentiate between the advantages and disadvantages of heuristic search: A* and AO* algorithms
- Differentiate between memory-bound search: Iterative Deepening A* and Recursive Best-First Search.

3.2 FORMULATING SEARCH IN STATE SPACE

A state space is a graph (V, E) where V is a set of nodes and E is a set of arcs, where each arc is directed from one node to another node.

- **V: a node is a data structure that contains** state description, plus, optionally, other information related to the parent of the node, an operation to generate the node from that parent, and other bookkeeping data.
- **E: Each arc** corresponds to an applicable action/operation. The source and destination nodes are called parent (**immediate predecessor**) and child (**immediate successor**) nodes with respect to each other. Ancestors (also called predecessors) and descendants (also called successors) node. Each arc has a fixed, non-negative cost associated with it, corresponding to the cost of the action.

Each node has a set of successor nodes. Corresponding to all operators (actions) that can apply at source node's state. Expanding a node is generating successor nodes and adding them (and associated arcs) to the state-space graph. One or more nodes may be designated as start nodes.

A **goal** test predicate is applied to a node to determine if its associated state is a goal state. A **solution** is a sequence of operations that is associated with a path in a state space from a start node to a goal node. The cost of a **solution** is the sum of the arc costs on the solution path.

State-space search is the process of searching through a state space for a solution by making explicit a sufficient portion of an implicit state-space graph to include a goal node.

Hence, initially $V = \{S\}$, where S is the start node; when S is expanded, its successors are generated, and those nodes are added to V , and the associated arcs are added to E . This process continues until a goal node is generated (included in V) and identified (by goal test).

To implement any **Uninformed search** algorithm, we always initialize and maintain a list called **OPEN** and put the start node of G in **OPEN**. If, after some time, we find **OPEN** is empty and we are not getting "goal node", then terminate with failure. We select a node n from **OPEN** and if $n \in \text{Goal node}$, then terminate with success, **else** we generate the successor of n (using operator O) and insert them in **OPEN**. In this way, we repeat the process till search is successful or unsuccessful.

Search strategies differ mainly in how to select an OPEN node for expansion at each step of the search.

A general search algorithm

1. **Initialize:** Set = {s}, where s is a start state.
2. **Fail:** If $OPEN = \{\}$, terminate with failure.
3. **Select:** Select a state, n, from OPEN
4. **Terminate:** If $n \in \text{Goal node}$, terminate with success
5. **Expand:** Generate the successor of n using operator O and insert them in OPEN.
6. **LOOP:** Goto Step 2

However, the problem with the above search algorithm is that it does not specify that a node should not be revisited once it has already been visited. That is “**how we can maintain a part of the state space that is already visited**”. So, we have an extension of the same algorithm, where we can save the explicit state space. To save the explicit space, we maintain another list called **CLOSED**.

Thus, to implement any Uninformed search algorithm efficiently, two lists, **OPEN** and **CLOSED**, are used.

Now we can select a node from OPEN and save it in CLOSED. The CLOSED list keeps a record of nodes that are Opened. The major difference between this algorithm and the previous one is that when we generate a successor node from CLOSED, we check whether it is already in $(OPEN \cup CLOSED)$. If it is already in $(OPEN \cup CLOSED)$, it is not inserted into OPEN again; otherwise, it is added to OPEN. The following modified algorithm is used to save the explicit space using the list CLOSED.

Modified search algorithm to save the explicit space

1. **Initialize:** Set $OPEN = \{s\}$, $CLOSED = \{\}$.
2. **Fail:** If $OPEN = \{\}$, terminate with failure.
3. **Select:** Select a state, n, from OPEN and save n in CLOSED
4. **Terminate:** If $n \in \text{Goal node}$, terminate with success
5. **Expand:** Generate the successor of n using operator O
For each successor, m, insert m in OPEN, only if $m \notin (OPEN \cup CLOSED)$
6. **LOOP:** Goto Step 2.

Here, the OPEN and CLOSED lists are used as follows:

OPEN: Nodes are yet to be visited.

CLOSED: Keeps track of all the nodes visited already

Note that initially OPEN list initializes with start state of G (e.g., **OPEN** = {*s*}) and CLOSED list as empty (e.g., **CLOSED** = {}).

Uninformed & Informed
Search

Insertion or removal of any node in OPEN depends on the specific search strategy.

3.2.1 Evaluation of Search Algorithms:

In any search algorithm, we select a node and generate its successor. Search strategies differ mainly in how an OPEN node is selected for expansion at each step of the search. Also, the insertion and deletion of nodes from the OPEN list depend on the specific search strategy. Any search algorithm is commonly evaluated according to the following four criteria, which are used to measure its performance:

- **Completeness:** Guarantees finding a solution whenever one exists.
- **Time Complexity:** How long (worst or average case) does it take to find a solution? Usually measured in terms of the number of nodes expanded.
- **Space Complexity:** How much space is used by the algorithm? Usually measured in terms of the maximum size that the "OPEN" list becomes during the search. Time and Space complexity are measured in terms of *b*: the branching factor or maximum number of successors of any node, *d*: the depth of the shallowest goal node (depth of the least cost solution), and *m*: the maximum depth (length) of any path in the state space (may be infinite).
- **Optimality/Admissibility:** If a solution is found, is it guaranteed to be an optimal one? For example, is it the one with the minimum cost?

The search process constructs a search tree, where **the root** is the initial state *S*, and the **leaf nodes** are nodes that have not yet been expanded (i.e., they are in OPEN List) or having no successors (i.e., they're dead ends). The search tree may be infinite because of loops, even if the state space is small. Search strategies mainly differ in how they select nodes from OPEN. Each node represents a partial solution path and its associated cost from the start node to the current node. In general, there are many possible paths from a given node and, therefore, many possible solutions that share this partial path as a prefix.

All search algorithms are distinguished by the order in which nodes are expanded. There are two broad classes of search methods: Uninformed search and Heuristic Search. Let us first discuss the Uninformed search.

3.3 UNINFORMED SEARCH

The uninformed search does not contain any domain knowledge, such as closeness or the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf

and goal nodes. Uninformed search explores the search tree without using any additional information about the search space, such as heuristic knowledge. It relies only on the initial state, the available operators, and the goal test; therefore, it is also called blind search. It examines each node of the tree until it achieves the goal node. Sometimes we may not get much relevant information to solve a problem.

For example, suppose we lost our car key, and we are not able to recall where we left it, we have to search for the key with some information, such as in which places we used to place it. It may be in our pants pocket or maybe in a table drawer. If it is not there, then we must search the whole house to get it. A reasonable approach would be to search the most likely locations first, such as the table drawer and then the wardrobe. In this situation, we must search with very few clues, making it an example of uninformed (blind) search.

Based on the order in which nodes are expanded, we have the following types of uninformed search algorithms:

- Breadth-first search
- Depth-first search
- Uniform cost search
- Iterative deepening depth-first search
- Bidirectional Search

3.3.1 Breadth-first search (BFS):

It is the simplest form of blind search. In this technique, the root node is expanded first, then all its successors are expanded, and then their successors, and so on. **In general, in BFS, all nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.** It means that all immediate children of nodes are explored before any of the children's children are considered. The search tree generated by BFS is shown below in Fig 2.

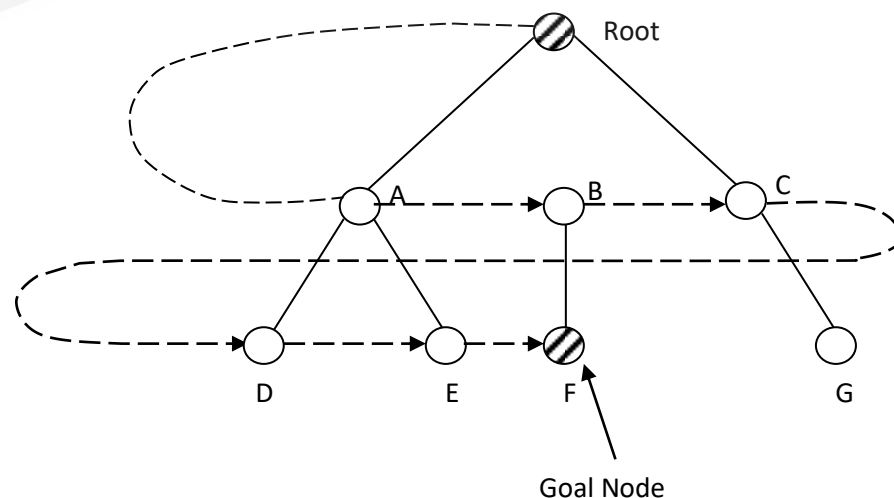


Fig 2 Search tree for BFS

Note that BFS is a brute-force search, so it generates all the nodes for identifying the goal, and note that we are using the convention that the alternatives are tried in the left-to-right order.

A BFS algorithm uses a data structure, a queue, that works on the FIFO principle. This queue will hold all generated but still unexplored nodes. **Please remember that the order in which nodes are placed on the queue, or removal and exploration, determines the type of search.**

We can implement it by using two lists called OPEN and CLOSED. The OPEN list contains those states that are to be expanded, and the CLOSED list keeps track of states already expanded. Here, the OPEN list is used as a **queue**.

BFS is effective when the search tree has a low branching factor.

Breath-First Search (BFS) Algorithm

- 1. Initialize:** Set = {s} , where s is a start state.
- 2. Fail:** If *OPEN* = { }, terminate with failure.
- 3. Select:** Remove the leftmost state (say *a*) from OPEN.
- 4. Terminate:** If *a* ∈ *Goal node*, terminate with success; else
- 5. Expand:** Generate the successor of node *a*, discard the successors of *a* if it's already in OPEN, insert only remaining successors on the right end of OPEN [i.e., QUEUE]
- 6. LOOP:** Goto Step 2

Let us take an example to see how this algorithm works.

Example 1: Consider the following graph in Figure 1 and its corresponding state space tree representation in Figure 2. Note that A is a start state and G is a goal state.

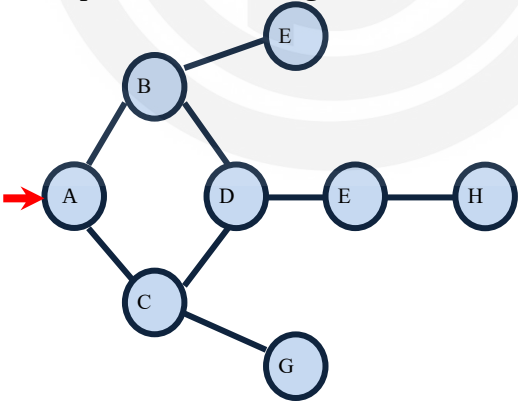


Fig-1 State space graph

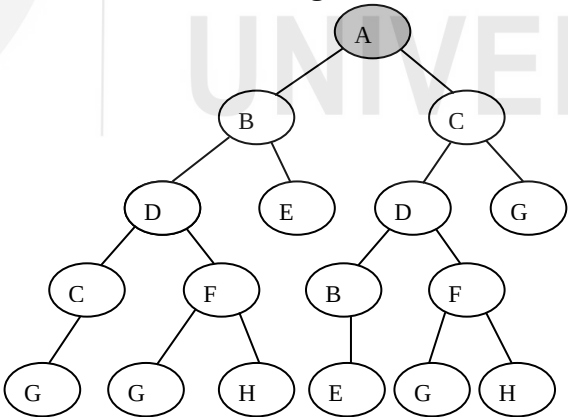
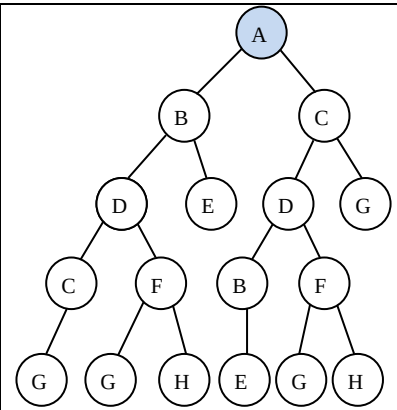
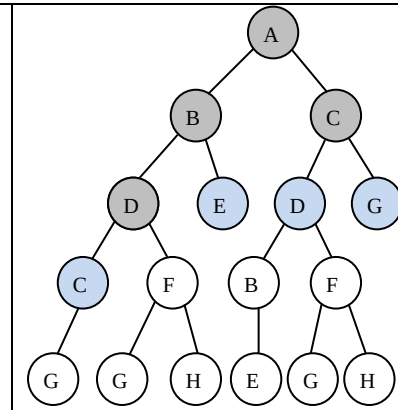
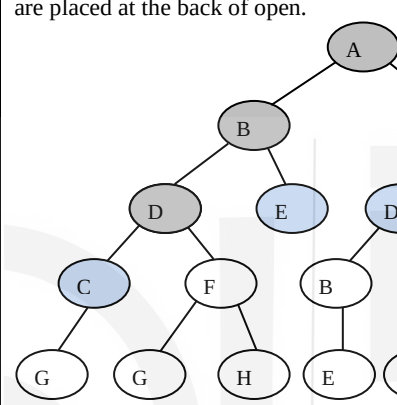
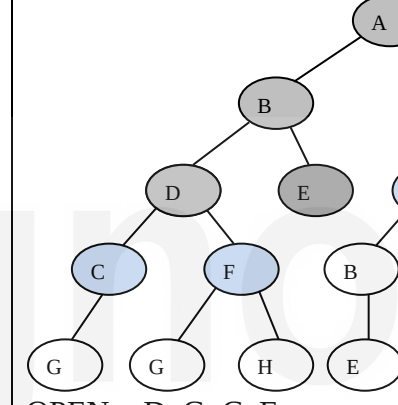
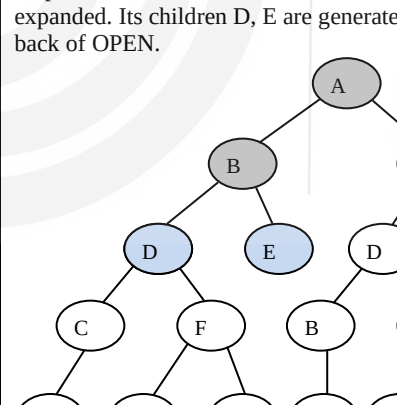
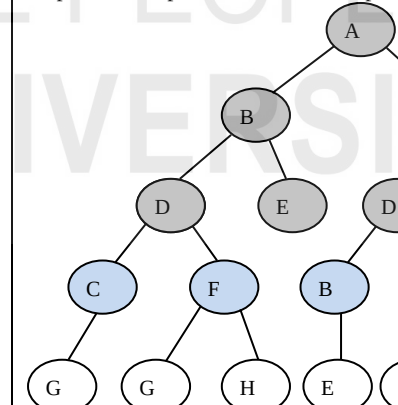
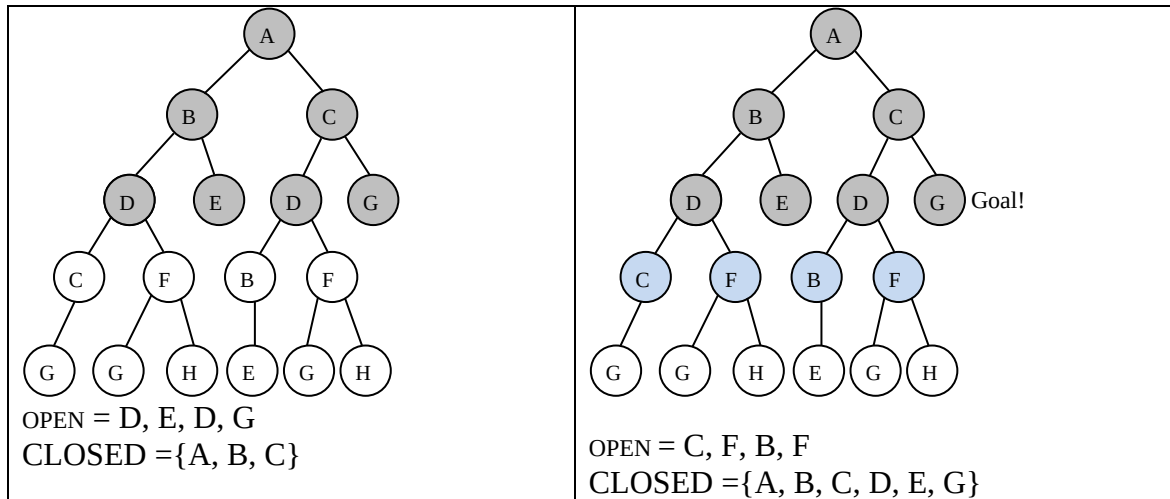


Fig-2 State space tree

Step1: Initially OPEN contains only one node corresponding to the source state A.	Step 5: Node D is removed from open. Its children C and F are generated and added to the back of OPEN.
--	---

 OPEN= A; CLOSED = { }	 OPEN = E, D, G, C, F CLOSED = A, B, C, D
Step2: A is removed from OPEN. The node is expanded, and its children B and C are generated. They are placed at the back of open.  OPEN = B, C CLOSED = {A }	Step 6: Node E is removed from OPEN. It has no children  OPEN = D, G, C, F CLOSED = A, B, C, D, E
Step: 3: Node B is removed from OPEN and is expanded. Its children D, E are generated and put at the back of OPEN.  OPEN = C, D, E ; CLOSED = {A, B}	Step 7: D is expanded, B and F are put in OPEN.  OPEN = G, C, F, B, F CLOSED = A, B, C, D, E, D
Step 4: Node C is removed from OPEN and is expanded. Its children D and G are added to the back of OPEN.	Step 8: G is selected for expansion. It is found to be a goal node. So, the algorithm returns the path ACG by following the parent pointers of the node corresponding to G. The algorithm terminates.



3.3.2 Time and Space complexity of BFS:

Consider a complete search tree of depth d , where each non-leaf node has b children (i.e., branching factor), and has a total of

$$1 + b + b^2 + b^3 + \dots + b^d = \frac{1(b^{d+1}-1)}{b-1} \text{ nodes.}$$

Time complexity is the number of nodes generated, so the time complexity of the BFS algorithm is $O(b^d)$

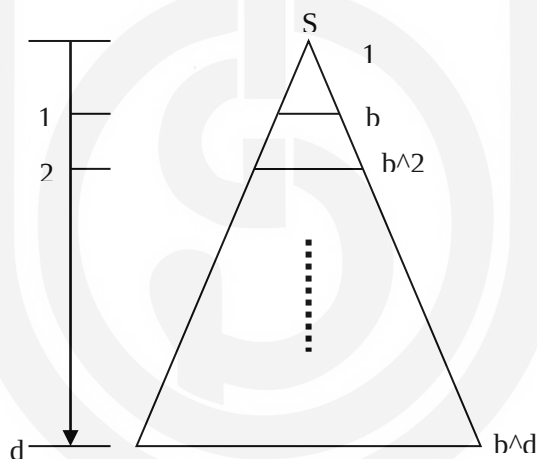


Fig 3: Search tree with b branching factor

For example, consider a complete search tree of depth 12, where every node at depths $0, 1, \dots, 11$ has 10 children (branching factor $b=10$) and every node at depth 12 has 0 children, then there are

$$1 + 10 + 10^2 + 10^3 + \dots + 10^{12} = \frac{1 \cdot (10^{13} - 1)}{10 - 1}$$

$$= \frac{10^{13} - 1}{9} = O(10^{12}) \text{ nodes in the complete search tree.}$$

- BFS is suitable for problems with shallow solutions

Space complexity:

BFS has to remember each and every node it has generated. Space complexity (maximum length of OPEN list):

So, space complexity is given by:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d).$$

Performance of BFS:

- **Time** required for BFS for a tree of b branching factor and d depth is $O(b^d)$.
- **Space** (memory) requirement for a tree with b branching factor and d depth is also $O(b^d)$
- BFS algorithm is **Complete** (if b is finite).
- BFS algorithm is **Optimal** (if cost = 1 per step)

Space is the bigger problem in BFS as compared to DFS.

3.3.3 Advantages and disadvantages of BFS:

BFS has some advantages, which are given below

1. BFS will never get trapped exploring bund alley.
2. It is guaranteed to find a solution if one exists.

BFS has certain disadvantages, too. They are given below-

1. Time complexity and Space complexity are both $O(b^d)$, i.e., exponential type. This is a very big hurdle.
2. All nodes are to be generated in BFS. So, even unwanted nodes are to be remembered (stored in the queue), which is of no practical use in the search.

3.3.4 Depth First Search

A Depth-First Search (DFS) explores a path all the way to a leaf before backtracking and exploring another path. That is, expand the deepest unexpanded node (expand the most recently generated deepest node first).

The search tree generated by the DFS is shown in the figure below:

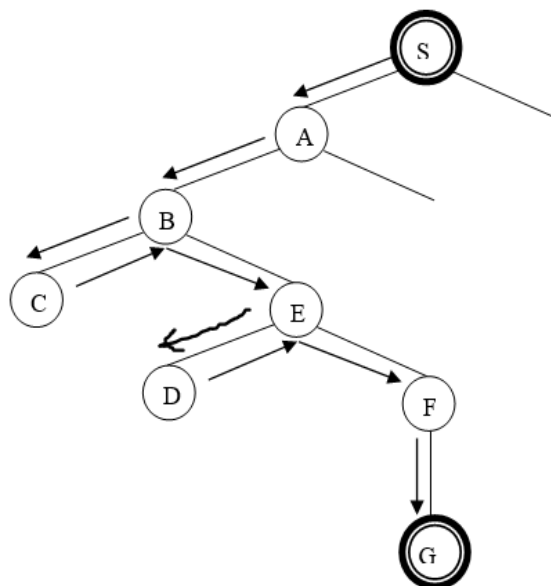


Fig. 4: Depth first search (DFS) tree

In depth-first search, we go as far down as possible into the search tree/graph before backing up and trying alternatives. It works by always generating a descendant of the most recently expanded node until some depth cut-off is reached, and then backtracks to the next most recently expanded node and generates one of its descendants. So, only the path of nodes from the initial node to the current node is stored, in order to execute the algorithm. For example, consider the following tree and see how the nodes are expanded using the DFS algorithm.

Example 1:

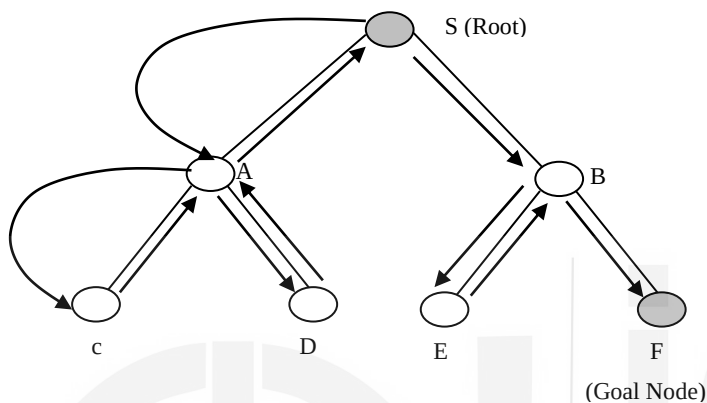


Fig. 5: Search tree for DFS

After searching root node S, then A and C, the search backtracks and tries another path from A. Nodes are explored in the order S, A, C, D, B, E, F .

Here again, we use the list OPEN as a STACK to implement DFS. If we found that the first element of OPEN is the goal state, then the search terminates successfully.

Depth-First Search (DFS) Algorithm

1. **Initialize:** Set = $\{s\}$, where s is a start state.
2. **Fail:** If $OPEN = \{\}$, terminate with failure.
3. **Select:** Remove the leftmost state (say a) from OPEN.
4. **Terminate:** If $a \in \text{Goal node}$, terminate with success, else
5. **Expand:** Generate the successor of the node a , discard the successors of a if it's already in OPEN, insert remaining successors on the **left** end of OPEN [i.e., STACK]
6. **LOOP:** Goto Step 2

Note: The only difference between BFS and DFS is in **Expand** (step 5). In BFS, we always insert the generated successors at the right end of the OPEN list, whereas in DFS at the left end of the OPEN list.

Properties of DFS Algorithm:

Suppose b (branching factor) is the maximum number of successors of any node, and m is the maximum depth of a leaf node, then

Number of nodes generated (in worst case): $1 + b + b^2 + \dots + b^m = O(b^m)$

3.3.5 Performance of DFS:

- **Time** required for DFS for a tree of b branching factor and m depth (of shallowest goal node) is $O(b^m)$.
- **Space** (memory) requirement for a tree with b branching factor and m depth (of shallowest goal node) is also $O(bm)$
- BFS algorithm is **Complete** (if b is finite).
- The BFS algorithm is not optimal.

Space is the advantage of DFS as compared to BFS.

3.3.6 Advantages and disadvantages of DFS:

Advantages:

- If depth-first search finds a solution without exploring much in a path, then the time and space it takes will be very less.
- The advantage of depth-first Search is that the memory requirement is only linear with respect to the search graph. This is in contrast with breadth-first search, which requires more space.

Example 1: Consider the following graph in Figure 1 and its corresponding state space tree representation in Figure 2. Note that A is a start state and G is a Goal state.

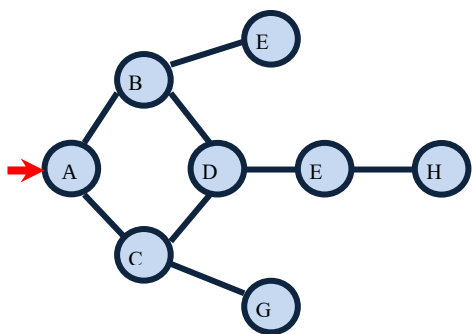


Fig. 1 State space graph

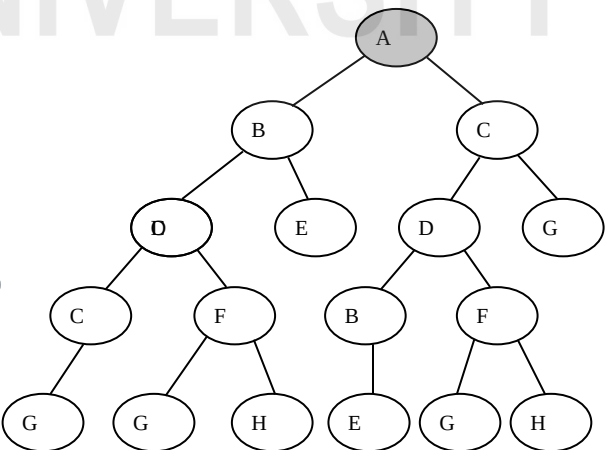
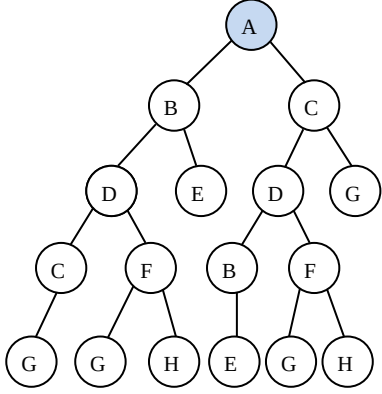
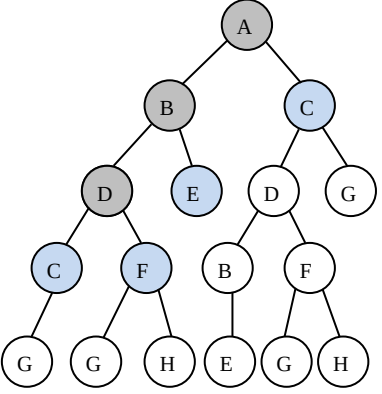
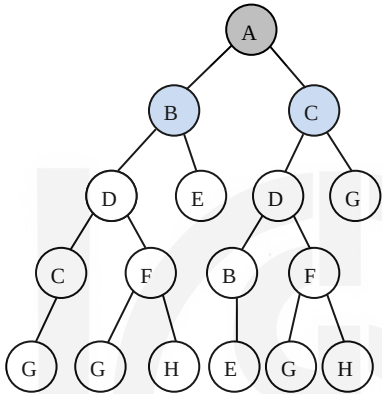
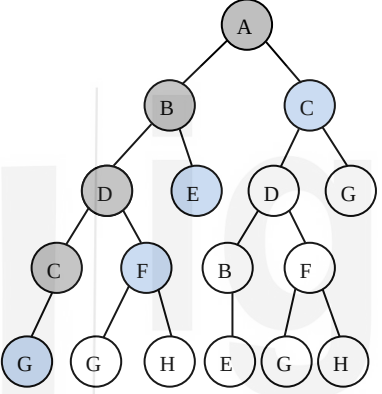
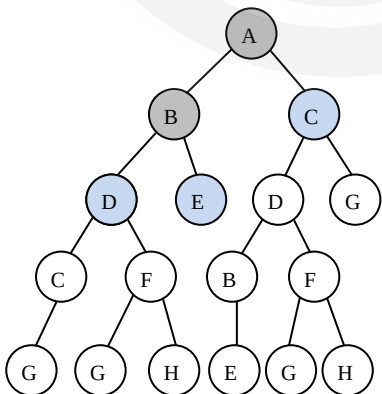
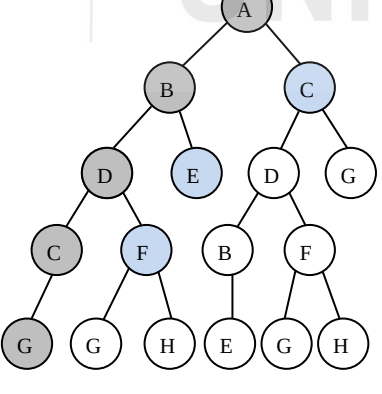


Fig. 2 State space tree

Step 1: Initially, the OPEN set contains only one node corresponding to the source state A.

Step 4: Node D is removed from OPEN and is expanded. Its children C and F are added to the front of the OPEN.

 OPEN = A CLOSED = { }	 OPEN = C, F, E, C CLOSED = A, B, D
Step 2: A is removed from OPEN. The node A is expanded, and its children B and C are generated. They are placed at the front of OPEN.  OPEN = B, C CLOSED = {A}	Step 5: Node C is removed from OPEN. Its children, G and F, are added to the front of the OPEN.  OPEN = G, F, E, C CLOSED = A, B, D, C
Step 3: Node B is removed from OPEN and is expanded. Its children D, E are generated and put at the front OPEN.  OPEN = D, E, C CLOSED = {A, B}	Step 6: Node G is expanded and found to be a goal node. The solution path A-B-D-C-G is returned, and the algorithm terminates.  OPEN = F, E, C CLOSED = A, B, D, C, G

3.3.7 Comparison of BFS and DFS

BFS goes level wise, but requires more space as compared to DFS. The space required by DFS is $O(d)$, where d is the depth of the tree, but the space required by BFS is $O(b^d)$.

DFS: The problem with this approach is that if there is a node close to the root, but not in the first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find the shortest path to a node (in terms of number of edges)

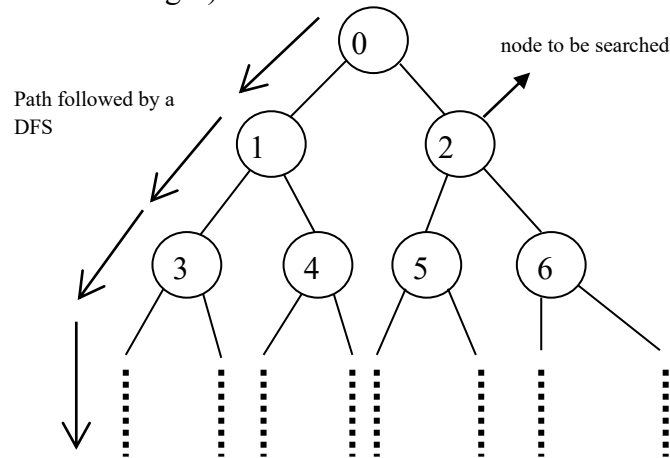


Fig 6: Path sequence in DFS

Suppose, we want to find node- '2' of the given infinite undirected graph/tree (Figure 6). A DFS starting from node-0 will dive left, towards node 1 and so on.

Whereas, node 2 is just adjacent to node 1.

Hence, a DFS wastes a lot of time coming back to node 2.

An **iterative deepening depth-first search** overcomes this and quickly finds the required node.

3.4 ITERATIVE DEEPENING DEPTH FIRST SEARCH (IDDFS)

Iterative Deepening Depth First Search (IDDFS) neither suffers the drawbacks of BFS nor DFS on trees. It takes advantage of both strategies.

It begins by performing DFS to a depth of zero, then to a depth of one, a depth of two, and so on until a solution is found or some maximum depth is reached.

It is like BFS in that it explores a complete layer of new nodes at each iteration before going to the next layer. It is like DFS for a single iteration.

It is preferred when there is a large search space, and the depth of a solution is not known. But it performs the wasted computation before reaching the goal depth. Since IDDFS expands all nodes at a given depth before expanding any nodes at greater depth, it is guaranteed to find a shortest-length (path) solution from the initial state to the goal state.

At any given time, it is performing a DFS and never searches deeper than depth 'd'. Hence, it uses the same space as DFS.

The disadvantage of IDDFS is that it performs wasted computation prior to reaching the goal depth.

Algorithm (IDDFS)

Initialized $d = 1$ /* depth of search tree */ , found = false

While (Found = False)

DO{

 perform a depth first search from start to depth d .

 if goal state is obtained

 then Found = true

else

discard the nodes generated in the search after depth d
(i.e. $d + 1$ onwards till last of tree)

}/* end while */

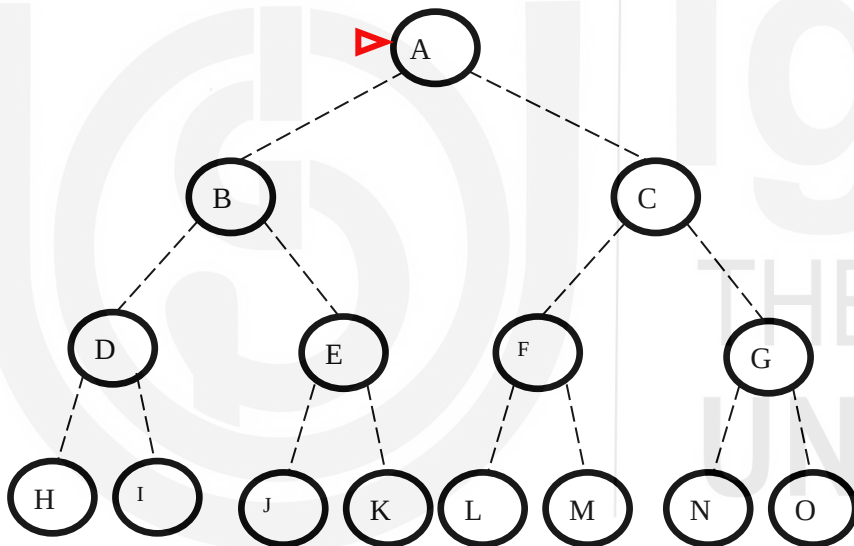
Report the solution, if obtained

Stop

Uninformed & Informed
Search

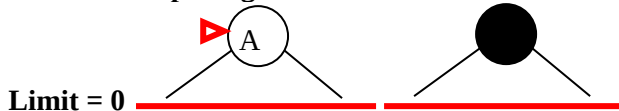
Let us consider the following example to understand the IDDFS:

- Here, the initial state is A and the goal state is M.

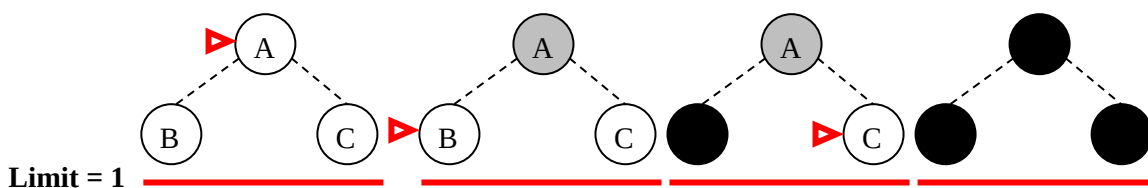


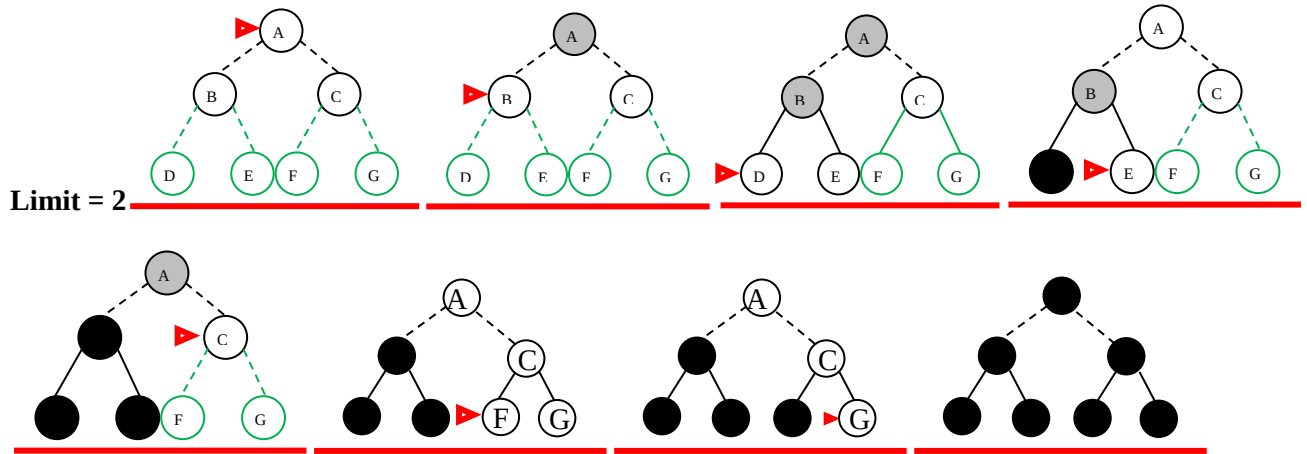
The Iterative deepening search proceeds as follows:

Iterative Deepening search $L = 0$

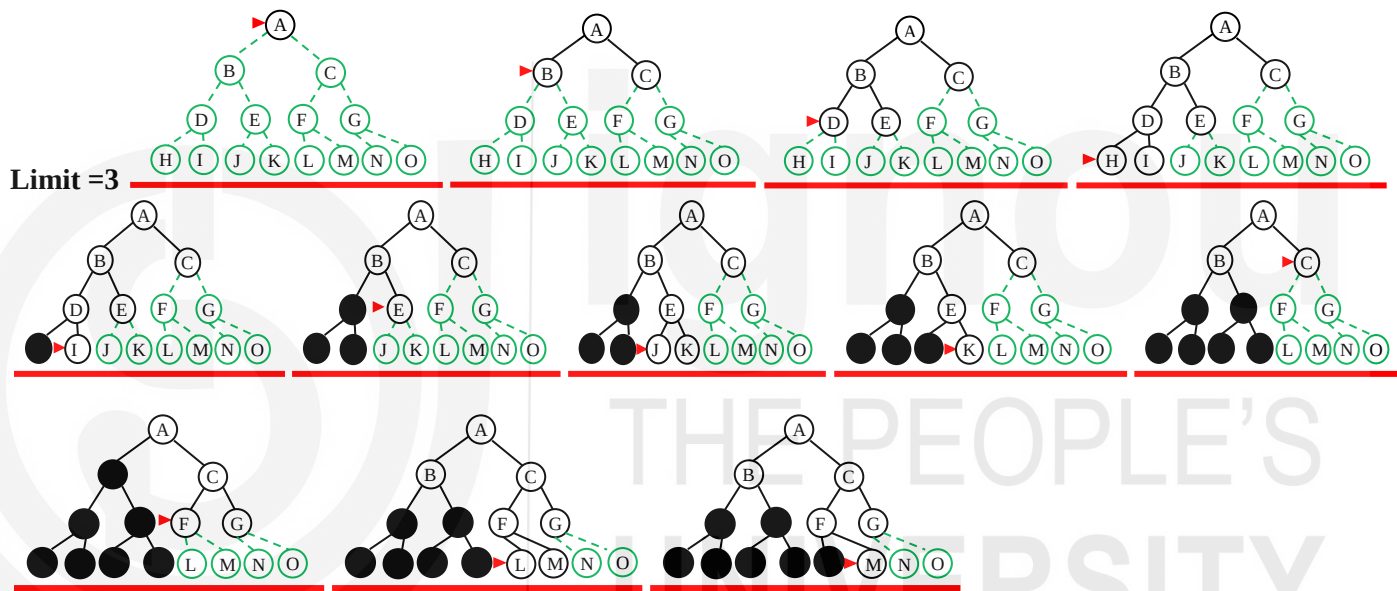


Iterative Deepening search $L = 1$





Iterative Deepening search $L = 3$



3.4.1 Time and space complexities of IDDFS

The time and space complexities of the IDDFS algorithm are $O(b^d)$ and $O(d)$, respectively.

It can be shown that depth-first iterative deepening is **asymptotically optimal**, among brute-force tree searches, in terms of time, space, and length of the solution. In fact, it is linear in its space complexity like DFS, and is asymptotically optimal to BFS in terms of the number of nodes expanded.

Please note that in general, iterative deepening is preferred over an uninformed search method when there is a large search space, and the depth of the solution is unknown. Also note that iterative deepening search is analogous to BFS in that it explores a complete layer, going to the next layer.

3.4.2 Advantages and Disadvantages of IDDFS:

Advantages:

1. It combines the benefits of BFS and DFS search algorithms in terms of fast search and memory efficiency.
2. It is guaranteed to find the shortest path solution.
3. It is a preferred uniformed search method when the search space is large and the depth of the solution is not known.

Disadvantages

1. The main drawback of IDDFS is that it repeats all the work from the previous phase, that is, it performs wasted computations before reaching the goal depth.
2. The time complexity is $O(b^d)$ i.e., exponential type only.

The following summarizes when to use which algorithm:

DFS	BFS	IDDFS
Many solutions exist. Know (or have a good estimate of) the depth of solution.	Some solutions are known to be shallow	Space is limited and the shortest solution path is required

3.5 BIDIRECTIONAL SEARCH

This search technique expands nodes from the start and goal state simultaneously. Check at each stage if the nodes of one have been generated by the other. If so, the path concatenation is the solution.

- This search is used when a problem has a single goal state that is given explicitly, and all the node generation operators have inverses.
- So, it is used to find the shortest path from an initial node to a goal node, instead of finding the goal itself along with the path.
- It works by searching forward from the initial node and backward from the goal node simultaneously, by hoping that the two searches meet in the middle.
- Check at each stage if the nodes of one have been generated by the other, i.e., they meet in the middle.
- If so, the path concatenation is the solution.

Thus, the BS Algorithm is applicable when generating predecessors is easy in both forward and backward directions, and there exist only one or fewer goal states. The following figure illustrates how the Bidirectional search is executed.

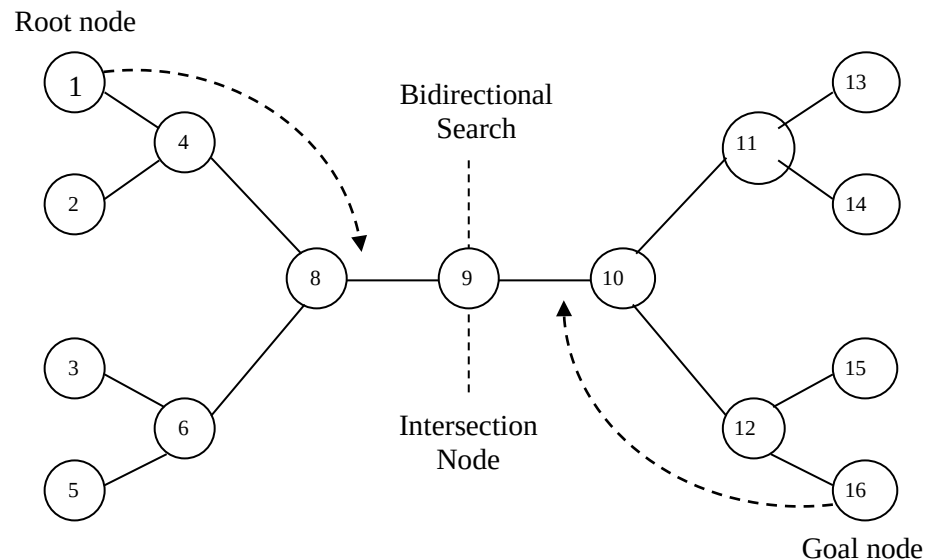


Fig. 7: Bidirectional search

We have node 1 as the start/root node and node 16 as the goal node. The algorithm divides the search tree into two sub-trees. So, from start node 1, we do a forward search, and at the same time, we do a backward search from goal node 16. The forward search traverses nodes 1, 4, 8, and 9, whereas the backward search traverses through nodes 16, 12, 10, and 9. We see that both the forward and backward searches meet at node 9, called the intersection node. So, the total path traced by the forward search and the path traced by the backward search is the optimal solution. This is how the BS Algorithm is implemented.

Advantages:

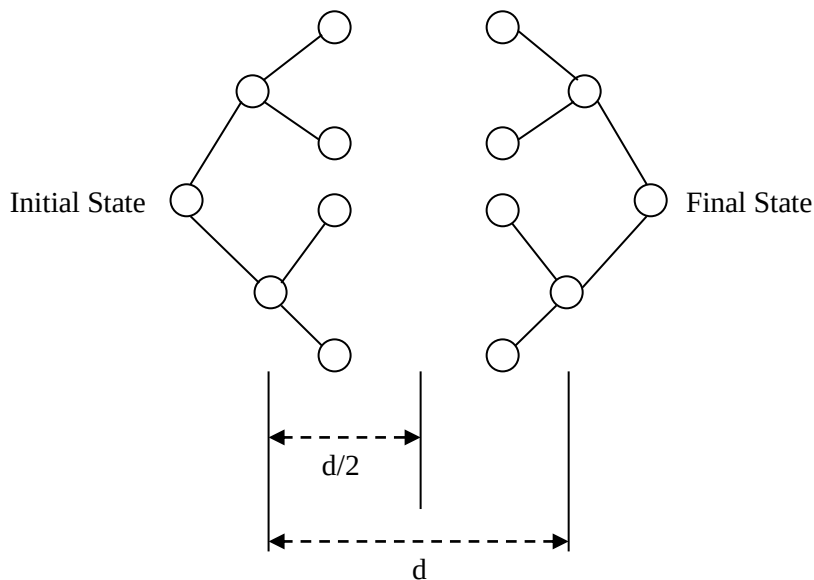
- Since BS uses various techniques like DFS, BFS, Depth-limited search (DLS), etc., it is efficient and requires less memory.

Disadvantages:

- The implementation of the bidirectional search tree is difficult.
- In bidirectional search, one should know the goal state in advance.
- It is practically inefficient due to the additional overhead of performing the insertion operation at each point of the search.

Time complexity:

The total number of nodes expanded in bidirectional search is $= 2b^{d/2} = O(b^{d/2})$, where b is a branching factor, and d is the depth of the shallowest goal node.



Finally, the Bidirectional search is:

- Complete? Yes
- Time Complexity: $O(b^{d/2})$
- Space complexity: $O(b^{d/2})$
- Optimal: Yes (if step cost is uniform in both forward and backward directions)

3.6 COMPARISON OF UNINFORMED SEARCH STRATEGIES

The following Table 1 compares the efficiency of uninformed search algorithms. These are the measures to evaluate the performance of the search algorithms:

Table -1 Performance of uninformed search algorithm

	BFS	DFS	IDDFS	Bidirectional Search (if applicable)
Time	b^d	b^d	b^d	$b^{d/2}$
Space	b^d	bm	bd	$b^{d/2}$
Optimum?	Yes	No	Yes	Yes
Complete?	Yes	No	Yes	Yes

Where

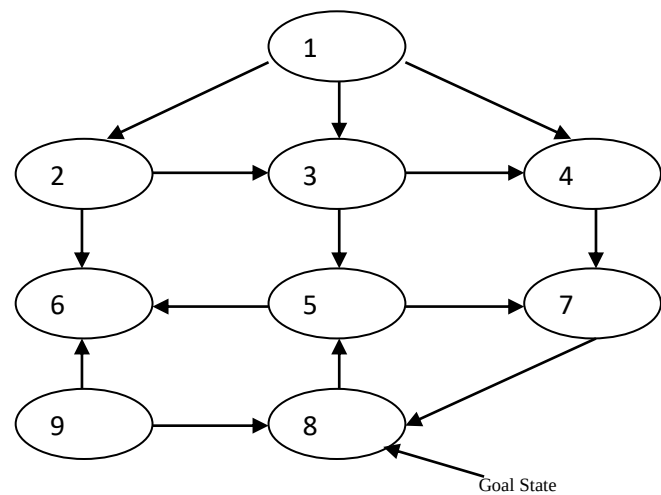
b =branching factor

d =depth of shallowest goal state

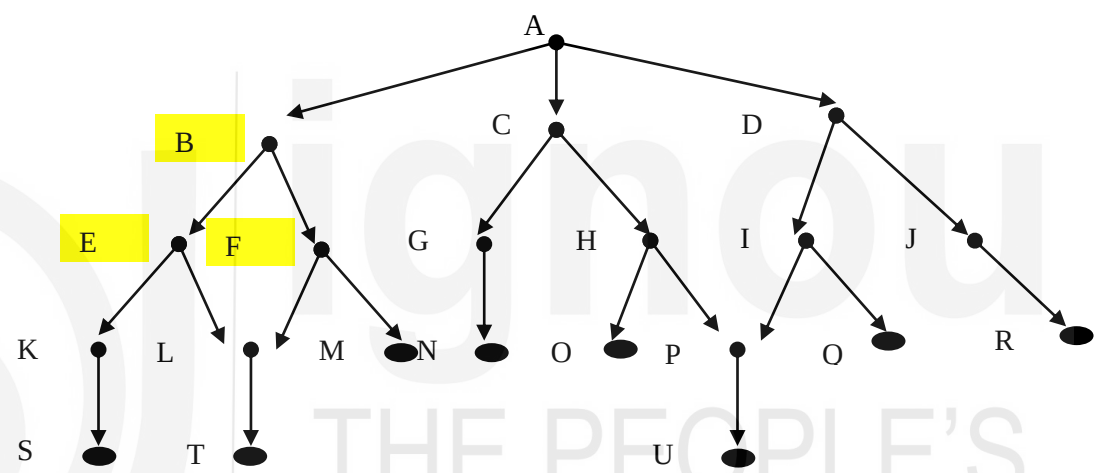
m =Maximum depth of the search space

☞ Check Your Progress 1

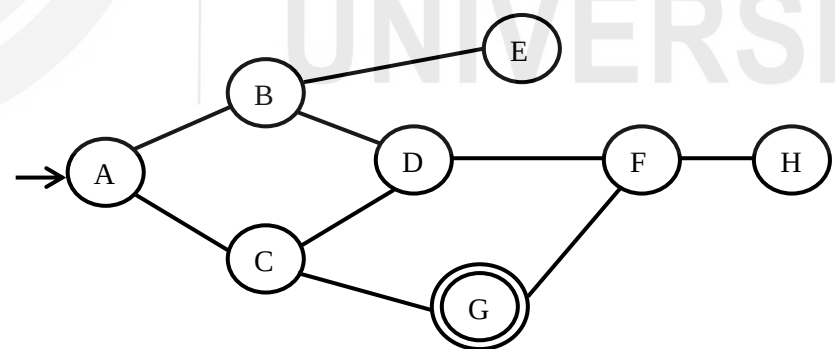
Q.1 Apply BFS and DFS algorithms on the following graph, clearly showing the contents of OPEN and CLOSE lists.



Q.2 Apply BFS algorithm on the following tree (M is the goal node)



Q.3 Apply BFS algorithm on the following graph.



Let A be the state, and G be the final or goal state to be searched.

Q.4 Compare the Uninformed search algorithm with respect to Time, Space, Optimal, and Complete.

3.7 INFORMED (HEURISTIC) SEARCH

Uninformed (blind) search is inefficient in most cases because they do not have any domain-specific knowledge about the goal state. Heuristic Search

uses domain-dependent (heuristic) information beyond the definition of the problem itself in order to search the space more efficiently.

Uninformed & Informed
Search

The following are some ways of using heuristic information:

- Deciding which node to expand next, instead of doing the expansion in a strictly breadth-first or depth-first order;
- In the course of expanding a node, deciding which successor or successors to generate, instead of blindly generating all possible successors at one time;
- Deciding that certain nodes should be discarded, or pruned, from the search space.

Informed search algorithms use domain knowledge. In an informed search, problem information is available, which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a **Heuristic search**.

Heuristics are a guesswork, or additional information about the problem. It may miss the solution if wrong heuristics are supplied. However, in almost all problems with correct heuristic information, it provides a good solution in a reasonable time

Informed search can solve many complex problems that could not be solved in any other way.

We have the following informed search algorithm:

1. Best-First Search
2. A* algorithm
3. Iterative Deepening A*

3.7.1 Strategies for providing heuristics information:

The informed search algorithm is more useful for a large search space. An informed search algorithm uses the idea of a heuristic, so it is also called a heuristic search.

“Heuristics are criteria, methods, or principles for deciding which among several alternative courses of action promises to be the most effective to achieve some goal.”

Heuristic Function:

Heuristic information is provided in the form of a function called **the heuristic function**.

- ❖ A heuristic is a function that is used in Informed Search, and it finds the most promising path.
- ❖ It takes the current state of the agent as its input and produces the estimation of how close the agent is to the goal.
- ❖ Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states.
- ❖ The heuristic method, however, might not always give the best solution,

but it is guaranteed to find a good solution in a reasonable time.

- ❖ This technique is always used to find a solution quickly.
- ❖ Informed Search defines a heuristic function $h(n)$ that estimates the "goodness" of a node n .
- ❖ The heuristic function is an estimate, based on domain-specific information that is computable from the current state description of how close we are to a goal.
- ❖ Specifically, $h(n)$ = estimated cost (or distance) of the minimal cost path from state 'n' to a goal state.

A heuristic function at a node n is an estimate of the optimum cost from the current node to a goal, denoted by $h(n)$

$h(n)$ = estimated cost of the cheapest path from node n to a goal node

For example, suppose you want to find the shortest path from Kolkata to Guwahati, then the heuristic for Guwahati may be the straight-line distance between Kolkata and Guwahati, that is:

$$h(\text{Kolkata}) = \text{euclideanDistance}(\text{Kolkata}, \text{Guwahati})$$

3.7.2 Formulation of informed (heuristic) search problem as State Space

Informed search problems can be represented as state space. The state space of a problem includes: an Initial state, one or more goal states, a set of state transition operators O (a set of rules), used to change the current state to another state, and a heuristic function h .

In general, a state space is represented by 5-tuples as follows: $S: [S, S_0, O, G, h]$

Where S : (Implicitly specified) Set of all possible states (possibly infinite).

S_0 : start state of the problem, $S_0 \in S$.

O : Set of state transition operators, each having the same cost. This is used to change the state from one state to another. It is the set of arcs (or links) between nodes

G : Set of Goal states, $G \subseteq S$.

$h()$: A heuristic function, estimating the distance to a goal node.

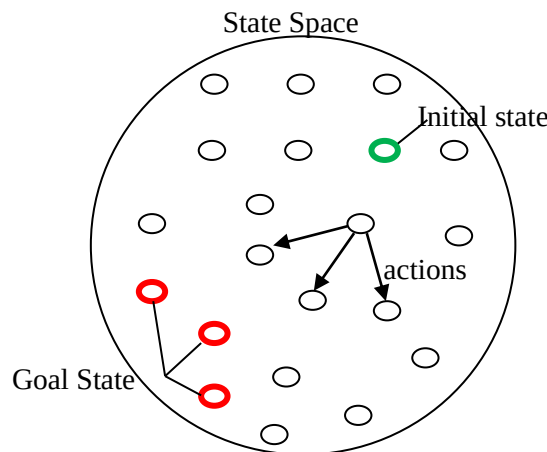


Fig. 8: State space with initial and goal node

We need to find a sequence of actions that transforms the agent from the initial state S_0 to Goal state G . State space is commonly defined as a directed graph or as a tree in which each node is a state, and each arc represents the application of an operator transforming a state to a successor state.

Thus, the problem is solved by using the rules (operators), in combination with an appropriate control strategy, to move through the problem space until a path from the **initial state to a goal state** is found. This process is known as **search**. A solution path is a path in the state space from S_0 (initial state) to G (Goal state).

We have already seen that an OPEN list is used to implement an uninformed (Blind) search (section 3.2). But the problem with using only one list OPEN is that it is not possible to keep track of the node that has already been visited. That is “**how we can maintain a part of the state space that is already visited**”. To save the explicit space, we maintain another list called **CLOSED**. Now we can select a node from OPEN and save it in **CLOSED**. Now, when we generate a successor node from CLOSED, we check whether it is already in $(OPEN \cup CLOSED)$. If it is already in $(OPEN \cup CLOSED)$, we will not insert it into OPEN again, otherwise insert it.

3.7.3 Best-First Search

Best first search uses an evaluation function $f(n)$ that gives an indication of which node to expand next for each node. Every node in a search space has an evaluation function (heuristic function) associated with it. A heuristic function value $h(n)$ on each node indicates how the node is from the goal node. Note that the evaluation function = heuristic cost function (in case of minimization problem) OR objective function (in case of maximization). The decision of which node to be expanded depends on the value of the evaluation function. Evaluation value = cost/distance of current node from goal node, and for the goal node evaluation function value=0

Based on the evaluation function, $f(n)$, Best-first search can be categorized into the following categories:

- 1) Greedy Best first search
- 2) A* search

The following 2 list (**OPEN** and **CLOSED**) are maintained to implement these two algorithms.

1. **OPEN** – all those nodes that have been generated & have had a heuristic function applied to them but have not yet been examined.
2. **CLOSED**- contains all nodes that have already been examined.

3.7.4 Greedy Best-First Search:

The greedy best-first search algorithm always selects the path that appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first

search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node that is closest to the goal node, and the closest cost is estimated by the heuristic function, i.e.

$$f(n) = g(n).$$

Where $h(n)$ = estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the **priority queue** (or to store the heuristic function value).

Best first search algorithm:

1. Initialize: Set $OPEN = \{s\}$, $CLOSED = \{ \}$;

$$f(s) = h(s)$$

2. Fail: If $OPEN = \{ \}$, terminate with failure.

3. Select: Select the minimum cost state n from $OPEN$, save n in $CLOSED$.

4. Terminate: If $n \in Goal\ node$, terminate with success and return $f(n)$, else

5. Expand: For each successor, m of n ,

If $m \notin [OPEN \cup CLOSED]$

Set $f(m) = h(m)$ and Insert m in $OPEN$

.....

6. LOOP: Goto Step 2

OR Pseudocode (for Best-First Search algorithm)

Step 1: Place the starting node into the $OPEN$ list.

Step 2: If the $OPEN$ list is empty, stop and return failure.

Step 3: Remove the node n from the $OPEN$ list that has the lowest value of $h(n)$, and place it in the $CLOSED$ list.

Step 4: Expand the node n , and generate the successors of node n .

Step 5: Check each successor of node n , and find whether any node is a goal node or not. If any successor node is a goal node, then return success and terminate the search; else, proceed to Step 6.

Step 6: For each successor node, the algorithm checks for the evaluation function $f(n)$, and then checks if the node has been in either $OPEN$ or the $CLOSED$ list. If the node has not been in either list, then add it to the $OPEN$ list.

Step 7: Return to Step 2.

Advantages of Best-First search:

- The best first search can switch between BFS and DFS, thus gaining the advantages of both algorithms.
- This algorithm is more efficient than the BFS and DFS algorithms.

Disadvantages of Best-First search:

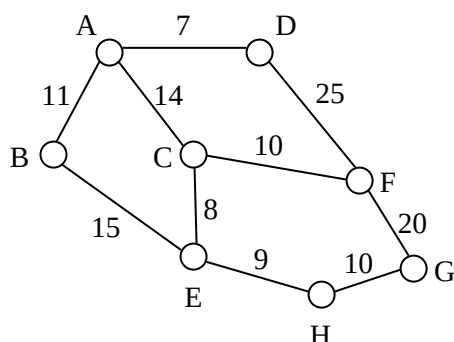
- Chances of getting stuck in a loop are higher.
- It can behave as an unguided depth-first search in the worst-case scenario.

Consider the following example for a better understanding of the greedy Best-First search algorithm.

Example 1: Consider the following **example** (graph) with heuristic function value $h(n)$, which illustrates the greedy Best-first search. Note that in the following example, the heuristic function is defined as

Uninformed & Informed Search

$$h_{SLD} = \text{straight line distance from } n \text{ to goal}$$



Let the heuristic function value $h(n)$ for each node n to the goal node G be defined as:

Straight line distance

$$A \rightarrow G = h(A) = 40$$

$$B \rightarrow G = h(B) = 32$$

$$C \rightarrow G = h(C) = 25$$

$$D \rightarrow G = h(D) = 35$$

$$E \rightarrow G = h(E) = 19$$

$$F \rightarrow G = h(F) = 17$$

$$H \rightarrow G = h(H) = 10$$

$$G \rightarrow G = h(G) = 0$$

$h(n)$ = straight line distance from node n to G

Note that $h(G)=0$

The nodes added/deleted from OPEN and CLOSED lists using Best-First Search algorithm are shown below.

OPEN	CLOSED
[A]	[]
[C, B, D]	[A]
B, D	A, C
F, E, B, D	A, C
G, E, B, D	A, C, F
E, B, D	A, C, F, G

Explanations are as follows:

Step 1: Initially, OPEN list starts with start state 'A', and CLOSED list is empty.

Step2: Children of A={C[25], B[32], D[35]}, so

OPEN={C[25], B[32], D[35]} therefore Best = C, so expand C node next.

Step3: Children of C={E[19], F[17]}, so

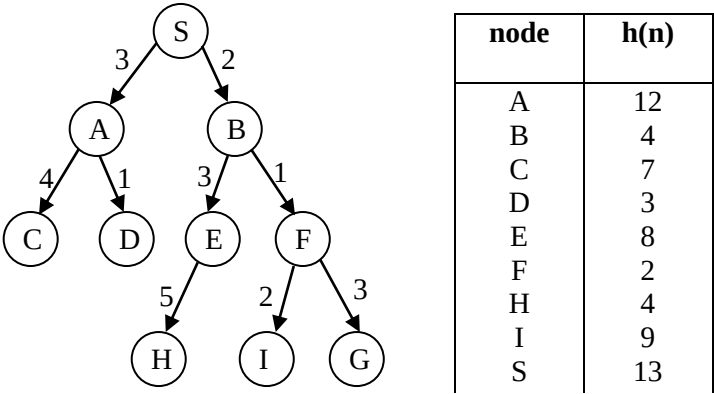
OPEN={F[17], E[19], B[32], D[35]} therefore Best = F, so expand node F next.

Step4: Children of F ={G[0]}, therefore OPEN= {G[0], E[19], B[32], D[35]}

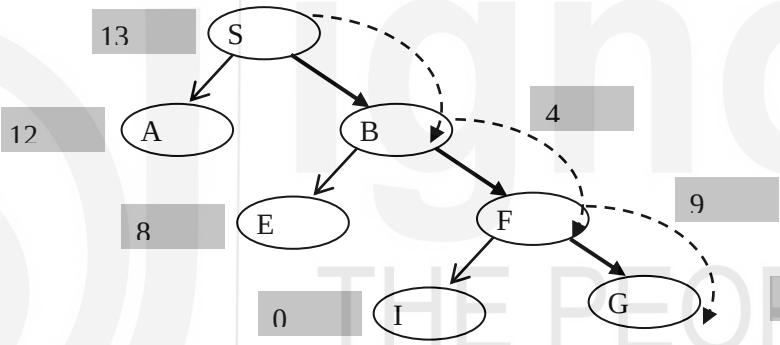
Best=G, this is a goal node so Stop.

Finally, we got the shortest path: $A \rightarrow C \rightarrow F \rightarrow G$, and the cost is 44.

Example 2: Consider the search problem below, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using the evaluation function $f(n)=h(n)$, which is given in the table below.



Here, we are using two lists, which are the **OPEN** and **CLOSED** Lists. The following are the iterations for traversing the above example.



Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration2: Open [E, F, A], Closed [S, B]

: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]

: Open [I, E, A], Closed [S, B, F, G]

Hence, the final solution path will be $S \rightarrow B \rightarrow F \rightarrow G$

Evaluation of Best-First Search algorithm:

Time Complexity:

The worst-case time complexity of Greedy best-first search is $O(b^m)$.

Space Complexity:

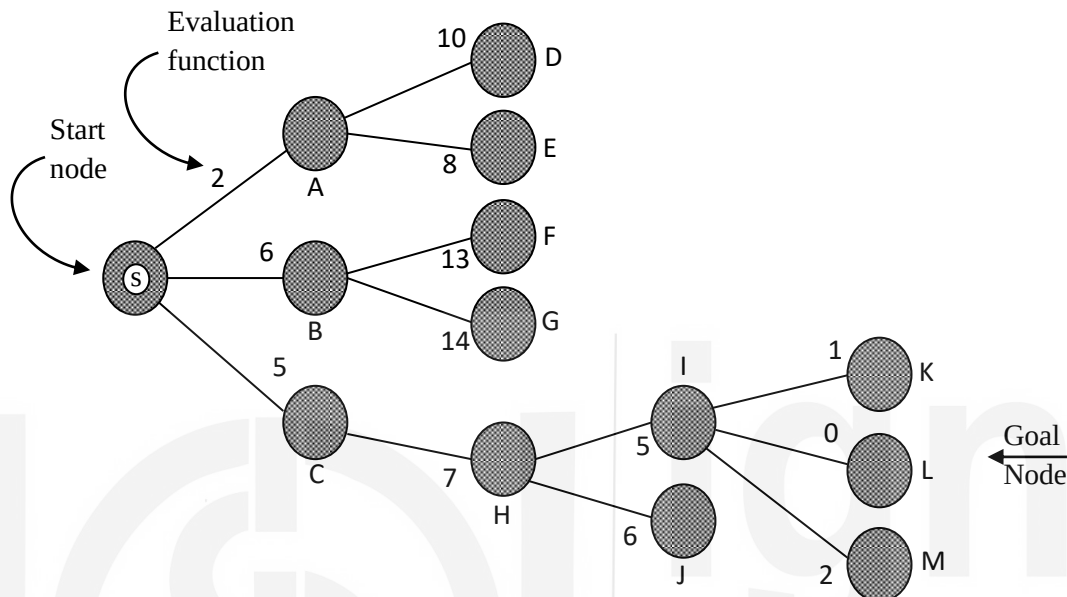
The worst-case space complexity of Greedy best-first search is $O(b^m)$. Where m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Uninformed & Informed Search

Optimal: Greedy best first search algorithm is not optimal.

Example 3: Apply the Greedy Best-First Search algorithm on the following graph (L is a goal node).



Working: We start with a start node, S. Now, S has three children, i.e., A, B, and C with their Heuristic function values 2, 6, and 5, respectively. These weights show approximately how far they are from the goal node. So, we write, children of S are: (A: 2), (B: 6), (C: 5)

Out of these, the node with the minimum value is (A: 2). So, we select A, and its children are explored (or generated).

Its children are (D: 10) and (E: 8)

The search process now has four nodes to search for, namely-

(B: 6), (C: 5), (D: 10), and (E: 8)

Out of these, node-C has the minimal value of 5. So, we select it and expand.

So, we get (H: 7) as its child. Now, the nodes to search are as follows-

(B:6), (D: 10), (E: 8), (H: 7), and so on.

Working of the algorithm can be represented in a tabular form as follows:

Step	Node being expanded	Children (on expansion)	Available nodes (to search)	Node Chosen
1.	S	(A:2), (B:6), (C:5)	(A:2), (B:6), (C:5)	(A:2)
2.	A	(D:10), (E:8)	(B:6), (C:5), (D:10), (E:8)	(C:5)
3.	C	(H:7)	(B:6), (D:10), (E:8), (H:7)	(B:6)
4.	B	(F:13), (G:14)	(D:10), (E:8), (H:7), (F:13), (G:14)	(H:7)
5.	H	(I:5), (J:6)	(D:10), (E:8), (F:13), (G:14), (I:5), (J:6)	(I:5)

6.	I	(K:1), (L:0), (M:2)	(D:10), (E:8), (H:7), (F:13), (G:14), (J:6), (K:1), (L:0), (M:2)	Goal node is found. So, search stops now
----	---	---------------------	---	--

3.8 A* ALGORITHM

A* search is the most commonly known form of best-first search. It uses a heuristic function $h(n)$ and the cost to reach node n from the start state $g(n)$. It has combined features of uniform cost search (UCS) and greedy best-first search, by which it solves the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands a smaller search tree and provides an optimal result faster.

In A* search algorithm, we use a search heuristic as well as the cost to reach the node. Hence, we can combine both costs as follows, and this sum is called the **fitness number (Evaluation Function)**.

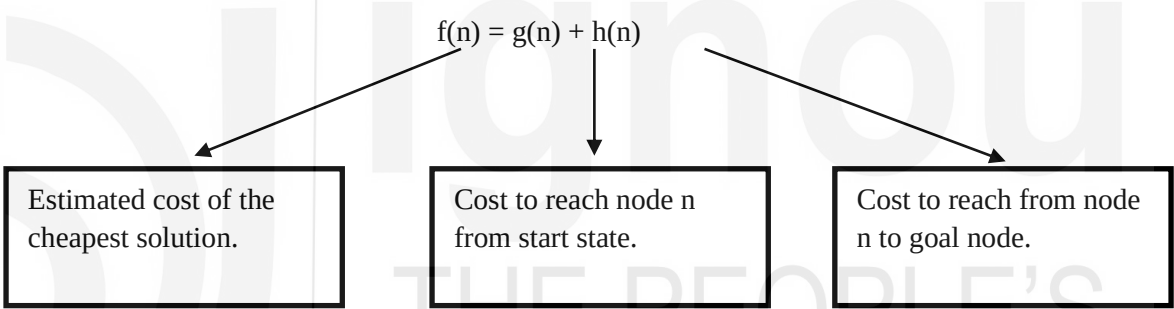


Fig 9: Evaluation function $f(n)$ in A* Algorithm

A* algorithm **evaluation function $f(n)$** is defined as **$f(n) = g(n) + h(n)$**
Where **$g(n)$** =sum of edge costs from the start state to n
And **$h(n)$** = An estimate of the lowest cost path from node $n \rightarrow$ goal node.

If $h(n)$ is **admissible**, then search will find an optimal solution. Admissible means underestimates the cost of any solution that can be reached from a node. In other words, a heuristic is called admissible if it always **underestimates**, that is, we always have $h(n) \leq h^*(n)$, where $h^*(n)$ denotes the minimum distance to a goal state from state n .

A* search begins at root node, and then the search continues by visiting the next node which has the least evaluation value $f(n)$.

It evaluates nodes by using the following evaluation function
 $f(n) = h(n) + g(n)$ = estimated cost of the cheapest solution through n .

Where,

- $g(n)$** : the actual shortest distance traveled from the initial node to the current node, which helps to avoid expanding paths that are already expensive
 - $h(n)$** : the estimated (or “heuristic”) distance from the current node to the goal, it estimates which node is closest to the goal node.
- Nodes are visited in this manner until a goal is reached.

Suppose s is a start state, then the calculation of the evaluation function $f(n)$ for any node n is shown in the following figure 10.

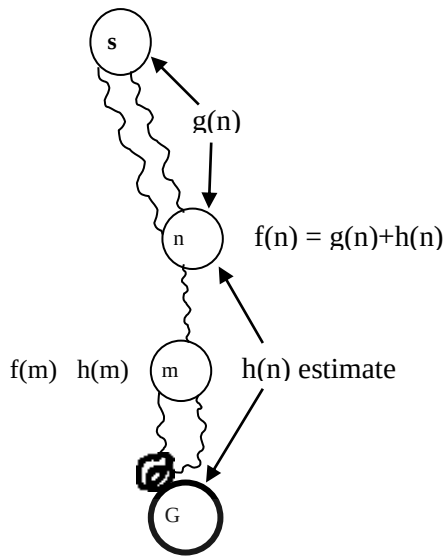


Fig 10 Calculation of evaluation function $f(n)$.

Algorithm A*

1. **Initialize:** Set $OPEN = (s)$, $CLOSED = \{\}$, $g(s) = 0$, $f(s) = h(s)$
2. **Fail:** If $OPEN = \{\}$, Terminate & fail
3. **Select :** Select the minimum cost state, n , from $OPEN$, Save n in $CLOSED$.
4. **Terminate:** If $n \in G$, terminate with success, and return $f(n)$
5. **Expand:** For each successor, m , of n
 - If $m \notin [OPEN \cup CLOSED]$
 - Set $g(m) = g(n) + C(n, m)$
 - Set $f(m) = g(m) + h(m)$
 - Insert m in $OPEN$
 - If $m \in [OPEN \cup CLOSED]$
 - Set $g(m) = \min \{ \min \{ g(m), g(n) + C(n, m) \} \}$
 - Set $f(m) = g(m) + h(m)$
 - If $f(m)$ has decreased and $m \in CLOSED$, move m to $OPEN$

In step 5, we generate a successor of n (say m), and for each successor m , if it does not belong to $OPEN$ or $CLOSED$, that is $m \notin [OPEN \cup CLOSED]$, then we insert it into $OPEN$ with the cost $g(n) + C(n, m)$, i.e., cost up to n and additional cost from $n \rightarrow m$.

If $m \in [OPEN \cup CLOSED]$ then we set $g(m)$ with the original cost and the new cost $[g(n) + C(n, m)]$. If we arrive at some state with another path that has less cost than the original one, then we replace the existing cost with this minimum cost.

If we find that $f(m)$ is decreased (if larger then ignore) and $m \in CLOSED$ then move m from $CLOSED$ to $OPEN$.

Note that the implementation of A* Algorithm involves maintaining two lists- **OPEN** and **CLOSED**. The list **OPEN** contains those nodes that have been evaluated by the heuristic function but have not expanded into successors yet and the list **CLOSED** contains those nodes that have already been visited. See the following steps for working of A* algorithm:

Step-1: Define a list OPEN. Initially, OPEN consists of a single node, the start node S.

Step-2: If the list is empty, return failure and exit.

Step-3: Remove node n with the smallest value of $f(n)$ from OPEN and move it to list CLOSED.

If node n is a goal state, return success and exit.

Step-4: Expand node n.

Step-5: If any successor to n is the goal node, return success and the solution by tracing the path from the goal node to S.

Otherwise, go to Step 6.

Step-6: For each successor node,

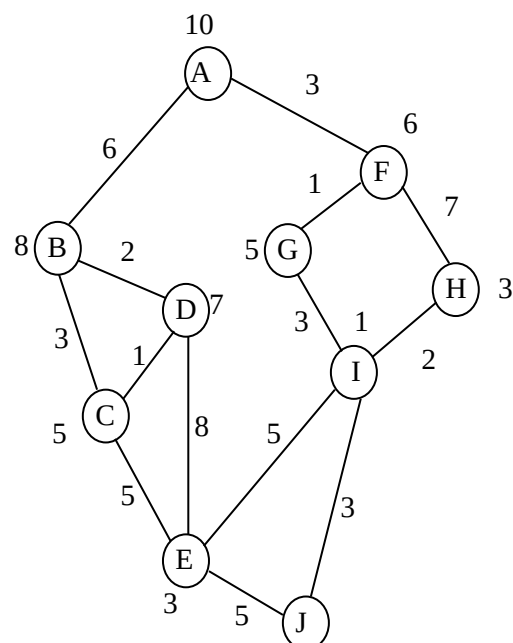
Apply the evaluation function f to the node.

If the node has not been in either list, add it to OPEN.

Step-7: Go back to Step-2.

3.8.1 Working of A* algorithm

Example 1: Let's us consider the following graph to understand the working of A* algorithm. The numbers written on edges represent the distance between the nodes. The numbers written on nodes represent the heuristic value. Find the most cost-effective path to reach from start state **A** to **final** state **J** using A* Algorithm.



Step-1:

We start with node A. Node B and Node F can be reached from node A. A* Algorithm calculates $f(B)$ and $f(F)$. Estimated Cost $f(n) = g(n) + h(n)$ for Node B and Node F is:

$$f(B) = 6+8=14$$

$$f(F) = 3+6=9$$

Since $f(F) < f(B)$, it decides to go to node F.

➔ Closed list (F)

Path- $A \rightarrow F$

Step-2: Node G and Node H can be reached from Node F.

A* Algorithm calculates $f(G)$ and $f(H)$.

$$f(G) = (3+1)=5=9$$

$$f(H) = (3+7) + 5 = 13$$

Since $f(G) < f(H)$, it decides to go to node G.

➔ Closed list (G)

Path- $A \rightarrow F \rightarrow G$

Step-3: Node I can be reached from node G.

A* Algorithm calculates $f(I)$.

$$f(I) = (3+1+3)+1=8; \text{ It decides to go to node I.}$$

➔ Closed list (I).

Path- $A \rightarrow F \rightarrow G \rightarrow I$

Step-4: Node E, Node H, and Node J can be reached from Node I.

A* Algorithm calculates $f(E)$, $f(H)$, $f(J)$.

$$f(E) = (3+1+3+5) + 3 = 15$$

$$f(H) = (3+1+3+2) + 3 = 12$$

$$f(J) = (3+1+3+3) + 0 = 10$$

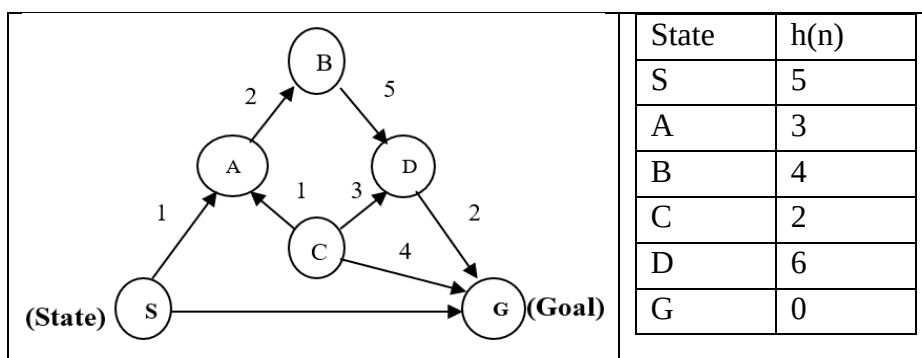
Since $f(J)$ is least, so it decides to go to node J.

➔ Closed list (J)

Shortest Path - $A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$

Path Cost is $3+1+3+3=10$

Example 2: Consider the following graph and apply A* algorithm and find the most cost-effective path to reach from start state **S** to **final** state G. The heuristic function value of each node n is defined in the table given.



Solution:

$$S \rightarrow A = 1+3 = 4$$

$$S \rightarrow G = 10+0 = 10$$

$$S \rightarrow A \rightarrow B = 1+2+4 = 7$$

$$S \rightarrow A \rightarrow C = 1+1+2 = 4$$

$$S \rightarrow A \rightarrow C \rightarrow D = 1+1+3+6 = 11$$

$$S \rightarrow A \rightarrow C \rightarrow G = 1+1+4 = 6$$

$$S \rightarrow A \rightarrow B \rightarrow D = 1+2+5+6 = 14$$

$$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 1+1+3+2 = 7$$

$$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 1+2+5+2 = 10$$

3.8.2 Advantages and disadvantages of A* algorithm

Advantages:

- A* search algorithm is the best algorithm among other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

- It does not always produce the shortest path as it is mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is the memory requirement, as it keeps all generated nodes in memory, so it is not practical for various large-scale problems.

3.8.3 Admissibility Properties of A* algorithm

A heuristic is called admissible if it always **underestimates**, that is, we always have $h(n) \leq h^*(n)$, where $h^*(n)$ denotes the minimum distance to a goal state from state n . For finite state spaces, A* always terminates.

In other words, **if the heuristic function h always underestimates, then the true cost h^* (that is, heuristic function cost h) is smaller than true cost $h^*(n)$, then A* is guaranteed to find an optimal solution.**

If there is a path from s to a goal state, A* terminates (even when the state space is infinite). Algorithm A* is admissible, that is, if there is a path from s to a goal state, A* terminates by finding an optimal path. If we are given two or more admissible heuristics, we can take their max to get a stronger admissible heuristic.

Admissibility Condition:

By admissible algorithm, we mean that the algorithm is sure to find an optimal solution if one exists. Please note that this is possible only when the evaluation function value never overestimates the distance of the node to the goal. Also note that if the evaluation function value, which is a heuristic one, is exactly

the same as the distance of the node to the goal, then this algorithm will immediately give the solution.

For example, the A* algorithm discussed above is admissible. There are three conditions to be satisfied for A to be admissible.

They are as follows-

1. Each node in the graph has a finite number of successors (or O).
2. All arcs in the graph have costs greater than some positive amount (say C).
3. For each node in the graph, n , $h(n) \leq h'(n)$.

This implies that the heuristic guess of the cost of getting from node n to the goal is never an overestimate. This is known as a **heuristic condition**. Only if these three conditions are satisfied, A* is guaranteed to find an optimal (least) cost path. **Please note that the A* algorithm is admissible for any node n if on such a path, $h'(n)$ is always less than or equal to $h(n)$.** This is possible only when the evaluation function value never overestimates the distance of the node to the goal. Although the admissibility condition requires $h'(n)$ to be a lower bound on $h(n)$, it is expected that the more closely $h'(n)$ approaches $h(n)$, the better the performance of the algorithm.

If $h(n) = h'(n)$, an optimal solution path would be found without over-expanding a node of the path. We assume that one optimal solution exists. If $h'(n)=0$, then A* reduces to the blind uniform cost algorithm or breadth-first algorithm.

Please note that the admissible heuristics are by nature optimistic because they think that the cost of solving the problem is less than it actually is because $g(n)$ is the exact cost for each n . Also note that $f(n)$ should never overestimate the true cost of a solution through n .

For example, consider a network of roads and cities with roads connecting these cities. Our problems to find a path between two cities such that the mileage/fuel cost is minimal. Then an admissible heuristic would be to use distance to estimate the costs from a given city to the goal city. Naturally, the air distance will be either equal to the real distance or will underestimate it, i.e., $h(n) \leq h'(n)$.

3.8.4 Properties of the Heuristic Algorithm:

1. **Admissibility condition:** Algorithm A is admissible if it guarantees to return an optimal solution when one exists. A heuristic function h is called admissible if, in general, we always have $h(n) \leq h'(n)$.*
2. **Completeness condition:** Algorithm A is complete if it always terminates with a solution when one exists.
3. **Dominance property:** If A_1 and A_2 are two admissible versions of Algorithm A such that A is more informed than A_2 , $h_1(n) > h_2(n)$.
4. **Optimal Property:** Algorithm A is optimal over a class of algorithms if it dominates all members of the class.

3.8.5 Results on A* Algorithm

1. **A* is admissible: Algorithm A* is admissible**, that is, if there is a path from S to the goal state, A* terminates by finding an optimal solution.
2. **A* is complete:** If there is a path from S to the goal state, **A terminates** (Even when the state space is ∞).
3. **Dominance property:** If A_1 and A_2 are two admissible versions of A* such that A_1 is more informed than A_2 , then A_2 expands at least as many states as does A_1 . (So A_1 dominates A_2 here, because it is a better heuristic than A_2)

If we are given two or more admissible heuristics, we can take their max to get a stronger admissible heuristic.

3.9 PROBLEM REDUCTION SEARCH

Problem reduction search is broadly defined as planning how best to solve a problem that can be recursively decomposed into subproblems in multiple ways. There are many ways to decompose a problem; we must find the best decomposition, one that provides the highest search efficiency or incurs the minimum cost.

We already know about the divide and conquer strategy; a solution to a problem can be obtained by decomposing it into smaller sub-problems. Each of these sub-problems can then be solved to get its sub-solution. These sub-solutions can then be recombined to get a solution as a whole. That is called **Problem Reduction**. This method generates an arc, which is called an AND arc. One AND arc may point to any number of successor nodes, all of which must be solved for an arc to point to a solution.

When a problem can be divided into a set of sub-problems, where each sub-problem can be solved separately, and a combination of these will be a solution, AND-OR graphs or AND - OR trees are used for representing the solution. The decomposition of the problem or problem reduction generates AND arcs. Consider the following example to understand the AND-OR graph (figure-11).

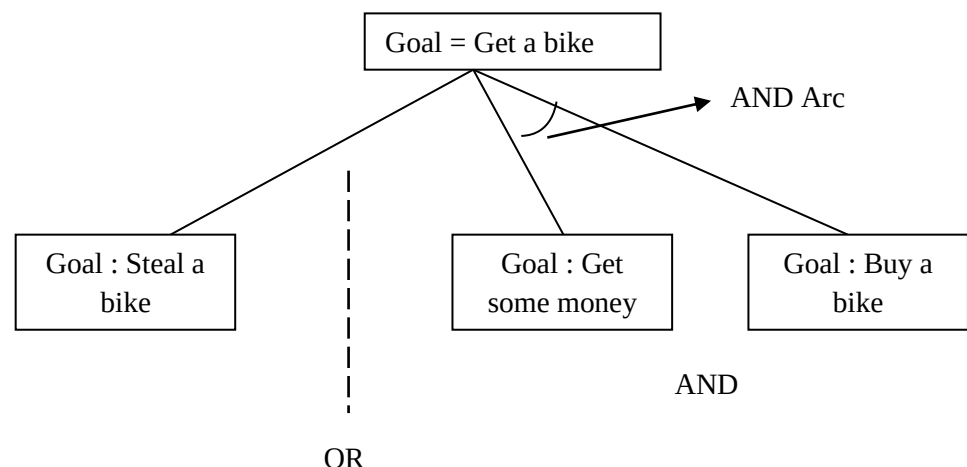


Fig 11: AND-OR graph

Figure 11 shows an AND-OR graph. In an AND-OR graph, an OR node represents a choice between possible decompositions, and an AND node represents a given decomposition. For example, to get a bike, we have two options:

1. (Steal a bike)

OR

2. Get some money **AND** buy a bike.

In this graph, we are given two choices: first, steal a bike or get some money **AND** buy a bike.

When we have more than one choice, and we have to pick one, we apply the OR condition to choose one. That's what we did here.

Basically, the ARC here denotes an AND condition.

Here we have replicated the arc between the get some money and buy a bike, because by getting some money, the possibility of buying a bike is greater than that of stealing one.

AO* search algorithm is based on an AND-OR graph, so it is called AO* search algorithm. AO* Algorithm is basically based on problem decomposition (breaking down the problem into small pieces).

The main difference between the A*(A star) and AO*(AO star) algorithms is that the A* algorithm represents an OR graph algorithm that is used to find a single solution (either this or that). But an **AO* algorithm** represents an AND-OR graph algorithm that is used to find more than one solution by ANDing more than one branch.

A* algorithm guarantees to give **an optimal solution**, while AO* doesn't since AO* doesn't explore all other solutions once it gets a solution.

3.9.1 Problem definition in AND-OR graph:

Given $[G, s, T]$

Where **G**: Implicitly specified AND/OR graph

s: Start node of the AND/OR graph

T: Set of terminal nodes (called SOLVED)

$h(n)$: Heuristic function estimating the cost of solving the sub-problem at n .

Example 1:

Let us see one example with the presence of heuristic value at every node (see Fig 12). The estimated heuristic value is given at each node. The heuristic value $h(n)$ at any node indicates "from this node at least $h(n)$ value (or cost) is required to find a solution". Here, we assume the edge cost value (i.e., $g(n)$ value) for each edge is 1. Remember, in the OR node, we always mark that successor node which indicates the best path for the solution.

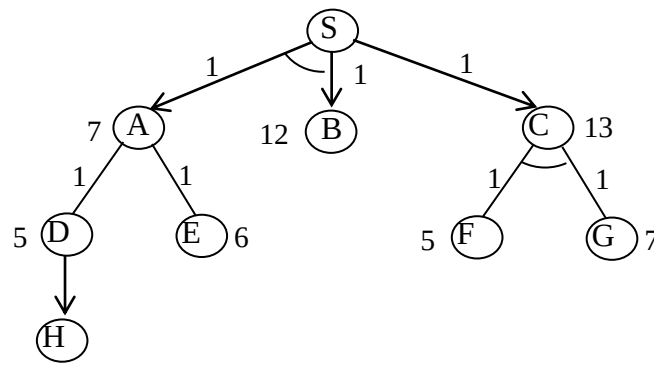


Fig 12: AND-OR graph with heuristic value at each node

Note that, the graph given in Fig 12, there are two paths for the solution from start state S: either S-A-B or S-C. To calculate the cost of the path, we use the formula $f(n) = g(n) + h(n)$ [note that here the $g(n)$ value is 1 for every edge].

Path1: $f(S-A-B) = 1+1+7+12=21$

Path2: $f(S-C) = 1+13=14$

Since $\min(21, 14) = 14$; so, we select successor node C, as its cost is minimum, which indicates the best path for the solution.

Note that C is an AND node; so, we consider both the successor nodes of C. The cost of node C is $f(C-F-G) = 1+1+5+7=14$; so, the revised cost of node C is 14, and now the revised cost of node S is $f(S-C) = 1+14=15$ (revised).

Note that once the cost (that is f value) of any node is revised, we propagate this change backward through the graph to decide the current best path.

Now, let us explore another path and check whether we are getting a lower cost than this cost or not.

$f(A-D)=1+5=6$ and $f(A-E)=1+6=7$; since A is an OR node so best successor node is D since $\min(6,7)=6$. So the revised cost of Node A will be 6 instead of 7, that is $f(A)=6$ (revised). Now next selected node is D, and D has only one node H so $f(D-H)=1+2=3$, so the revised cost of node D is 3, so now, the revised cost of node A is $f(A-D-H)=4$. This path is better than $f(A-E)=7$. So, the final revised cost of node A is 4. Now the final revised cost of $f(S-A-B)=1+1+4+12=18$ (revised).

Thus, the final revised cost for

Path1: $f(S-A-B)=18$ and

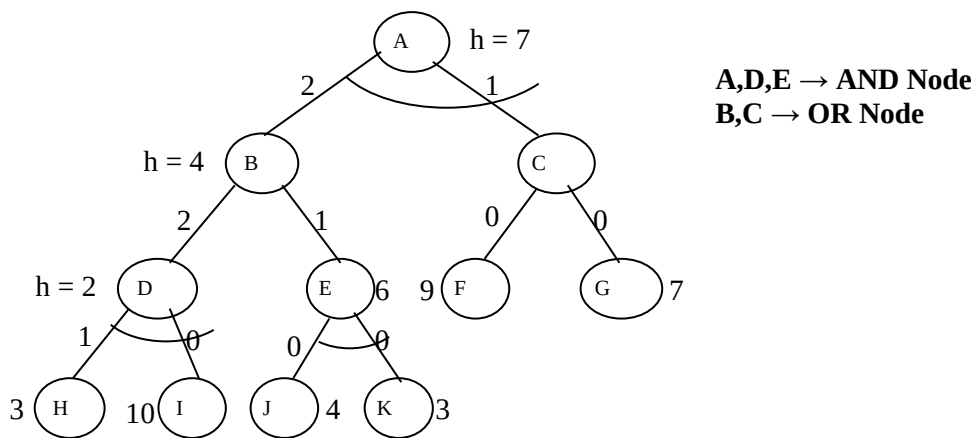
Path2: $f(S-C)=15$

So the optimal cost is 15.

Example 2:

Consider the following AND-OR Graph with estimated heuristic cost at every node. Note that A, D, E are AND nodes and B, C are OR nodes. Edge cost (i.e., $g(n)$ value) is also given.

Apply the AO* algorithm and find the optimal cost path using the AO* algorithm.

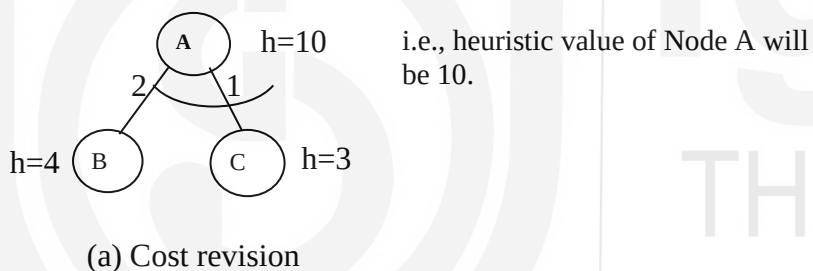


AND-OR graph with heuristic function value.

Heuristic (Estimated) cost at every node is given. For example, heuristic cost at node A is $h=7$, which means at least a 7-unit cost is required to find a solution. Since A is an AND Node, we have to solve both of its successor Nodes, B and C.

Cost of Node A i.e., $f(A-B-C) = (2+4) + (1+3) = 10$

We perform cost revision in a bottom-up fashion.

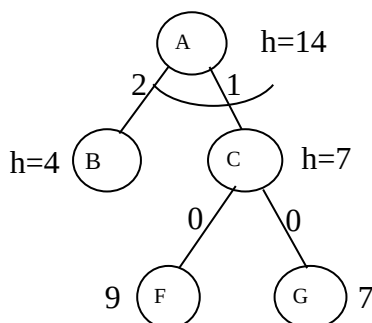


Now, let's see R.H.S first. Node C is an OR Node.

So, cost $f(C-F) = 0+9=9$

& $f(C-G) = 0+7=7$

So, the best successor of C is G, so we perform cost revision in Bottom-up fashion as follows:



Now, the revision cost of node A is 14; So, till now, we can say that the best path to solve the problem is: G-C-A. Now expand the left node of the root, i.e., node B.

Since B is an OR Node; So, we have to see which successor is best, i.e., D or E.

The revised cost of node D is $= (3+1) + (10+0) = 14$

And the revised cost of node E $= (4+0) + (3+0) = 7$

So, the best promising successor node is E.

Now, perform cost revision in a bottom-up fashion.

Note that all the leaf node in the marked tree is solved. So, the best way to solve the problem is to follow the marked tree and solve those marked problems. The best cost to solve the problem is **18**.

Note that for the **AND Node**: if both successor (unit problem) is solved, then we declare SOLVED, and for **the OR Node**: if any one best successor is SOLVED, then we declare SOLVED.

3.9.2 AO* algorithm

Our real-life situations cannot be exactly decomposed into either an AND tree or an OR tree, but are always a combination of both. So, we need an AO* algorithm where O stands for 'ordered'. Instead of two lists OPEN and CLOSED of A* algorithm, we use a single structure GRAPH in AO* algorithm. It represents a part of the search graph that has been explicitly generated so far. Please note that each node in the graph will point both down to its immediate successors and to its immediate predecessors. Also note that each node will have some $h'(n)$ value associated with it. But unlike A* search, $g(n)$ is not stored. It is not possible to compute a single value of $g(n)$ due to many paths to the same state. It is not required, also, as we are traversing top-down along the best-known path.

This guarantees that only those nodes that are on the best path are considered for expansion

Hence, $h'(n)$ will only serve as the estimate of the goodness of a node.

Next, we develop an AO* algorithm.

Algorithm AO*

1. Initialize: Set $G^* = \{s\}$, $f(s) = h(s)$
If $s \in T$, label s as SOLVED
2. Terminate: If s is SOLVED, then Terminate
3. Select: Select a non-terminal leaf node n from the marked sub-graph
4. Expand: Make explicit the successors of n for each new successors, m :
Set $f(m) = h(m)$
If m is terminal, label m SOLVED
5. Cost Revision: Call **Cost-Revise (n)**
6. Loop: Go to Step 2.

Cost Revision in AO*: Cost-Revise(n)

1. Create $Z = \{n\}$
2. If $Z = \{\}$ return
3. Select a node m from Z such that m has no descendants in Z
4. If m is an AND node with successors $r_1, r_2, \dots, r_k$:

Set $f(m) = \sum [f(r_i) + c(m, r_i)]$

Mark the edge to each successor of m

If each successor is labelled SOLVED,
then label m as SOLVED.

5. If m is an OR node with successor $r_1, r_2, \dots, r_k$:

Set $f(m) = \min \{f(r_i) + c(m, r_i)\}$

Mark the edge to the best successor of m

If the marked successor is labelled SOLVED, label m as SOLVED

6. If the cost or label of m has changes, then insert those parents
of m into Z for which m is a marked successor

3.9.3 Advantage of AO* algorithm

Note that **AO*** will always find a minimum cost solution if one exists if $h'(n) < h(n)$ and that all arc costs are positive. The efficiency of this algorithm will depend on how closely $h'(n)$ approximates $h(n)$. Also note that **AO*** is guaranteed to terminate even on graphs that have cycles.

Note: When the graph has only OR node, then the AO* algorithm works just like A* algorithm.

3.10 MEMORY BOUND HEURISTIC SEARCH

The following are the commonly used memory-bound heuristics search:

1. Iterative deepening A* (IDA*)
2. Recursive Best-First search (RBFS)
3. Memory-bound A* (MBA*)

3.10.1: Iterative Deepening A* (IDA*)

IDA* is a variant of the A* search algorithm, which uses iterative deepening to keep the memory usage lower than in A*. It is an informed search based on the idea of the uniformed iterative deepening search. Iterative deepening A* or IDA* is similar to iterative-deepening depth-first, but with the following modifications:

The depth bound is modified to be an f-limit

1. Start with limit = $f(\text{start})$
2. Prune any node if $f(\text{node}) > f\text{-limit}$
3. Next f-limit = minimum cost of any node pruned

Iterative Deepening is a kind of uniformed search strategy. It combines the benefits of depth-first and breadth-first search.

Advantage of IDA* is: It is optimal and complete like breadth-first search, and has a modest memory requirement like depth-first search.

IDA* algorithm

1. Set $C = f(s)$
2. Perform DFBB with cut-off C
Expand a state, n , only if its f -value is less than or equal to C
If a goal is selected for expansion, then return C and terminate
3. Update C to the minimum f -value which exceeded C among states that were examined, and Go TO Step 2.

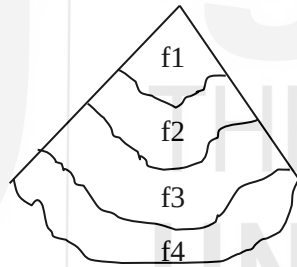
3.10.2 Working of IDA*

- Perform depth-first search LIMITED to some f -bound.
- If the goal is found, then ok.
- Else: increase the f -bound and restart.

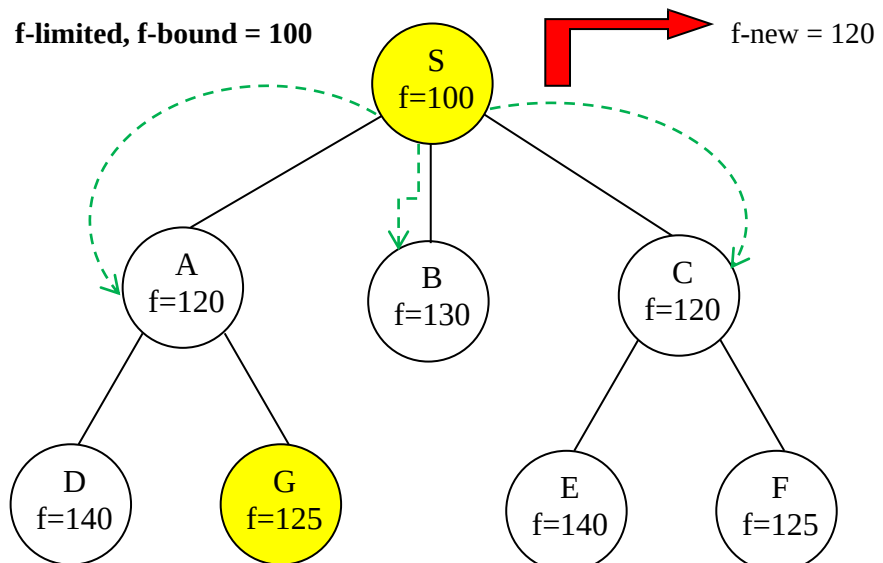
How to establish the f -bound?

Initially: $f(S)$

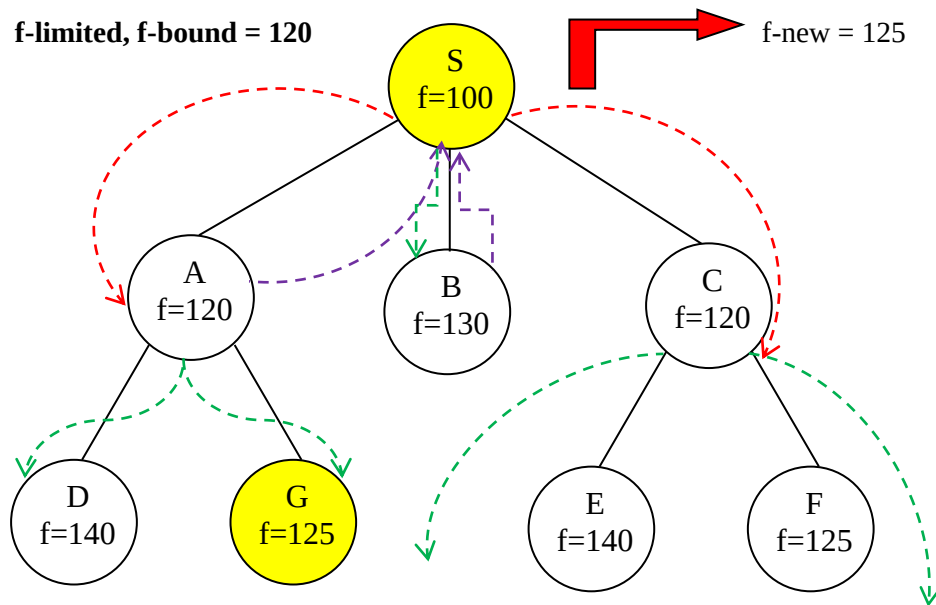
- Generate all successors
- Record the minimal $f(\text{succ}) > f(S)$
- Continue with minimal $f(\text{succ})$ instead of $f(S)$



Consider the following example to understand the IDA*

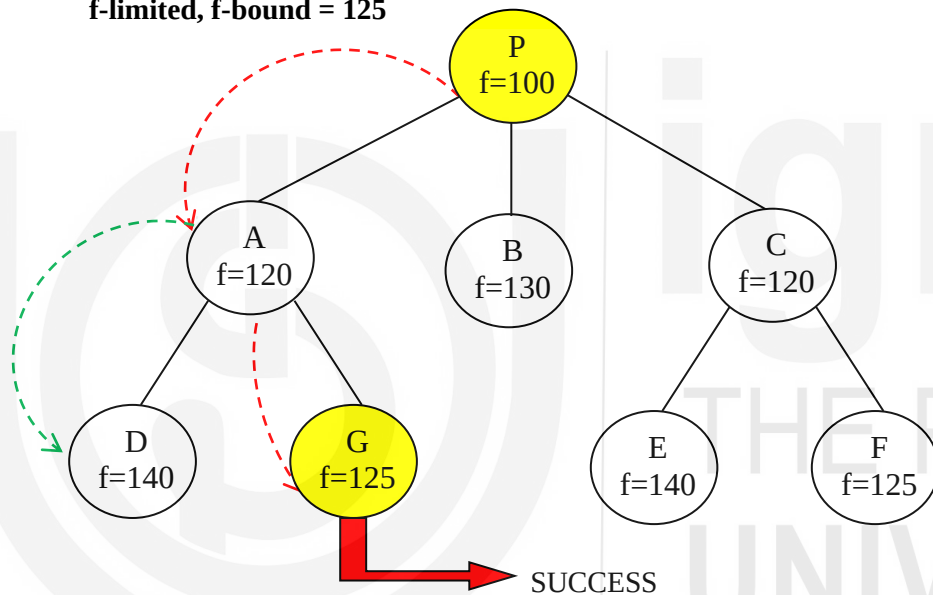


f-limited, f-bound = 120



Uninformed & Informed Search

f-limited, f-bound = 125



3.10.3 Analysis of IDA*

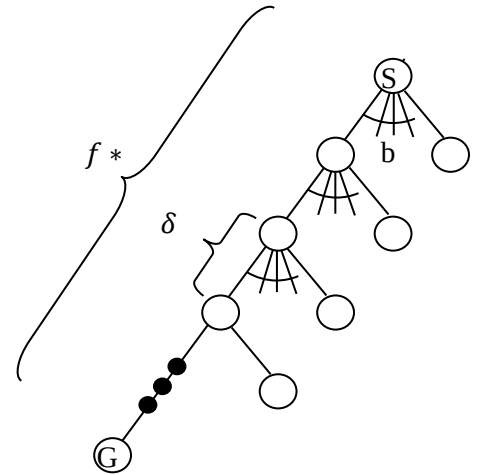
IDA* is complete, optimal, and optimally efficient (assuming a consistent, admissible heuristic), and requires only a polynomial amount of storage in the worst case:

F^* = optimal path cost to a goal

B = branching factor

δ = minimum operator step cost

$\frac{bf^*}{\delta}$ nodes of storage required



Note that IDA* is complete and optimal. Space usage is linear in the depth of the solution. Each iteration is a depth-first search, and thus it does not require a priority queue.

3.10.4 Comparison of A* and IDA* algorithm:

- Iterative Deepening Search (IDS) is nothing but BFS plus DFS for tree search.
- IDA* algorithm is “complete and optimal” algorithm.
- BFA and A* are good for optimality, but not memory.
- DFS: good for memory $O(bd)$, but not optimality
- In the worst case, only one new state is expanded in each iteration
- (IDA*). If A* expands N states, then IDA* can expand:

$$1+2+3+\dots+N=O(N^2)$$

3.11 RECURSIVE BEST FIRST SEARCH (RBFS)

The idea of recursive best first search is to simulate **A*** search with **$O(bd)$** memory, where b is the branching factor, and d is the solution depth.

It is a memory-bound, simple recursive algorithm that works like a standard best first search but only takes up linear space. There are some things that make it different from recursive DFS. It keeps track of f , the value of the best alternative path that can be found from any ancestor of the current node, instead of continuing indefinitely down the current path.

RBFS mimics the operation of the standard Best-First search algorithm. RBFS keeps track of the f -value of the best alternative path available from any ancestor of the current node. If the current node exceeds the limit, the recursion unwinds back to the alternative path. As the recursion unwinds, RBFS replaces the f -value of each node along the path with the best f -value of its children. In this way, RBFS remembers the f -value of the best leaf in the forgotten subtree and can therefore decide whether it's worth re-expanding the subtree at some later time.

RBFS is somewhat more efficient than IDA*, but still suffers from excessive node regeneration. A* and RBFS are optimal algorithms if the heuristic function $h(n)$ is admissible.

Uninformed & Informed Search

```

function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
  return RBFS(problem, MAKE-NODE(problem.INITIAL-STATE),  $\infty$ )

function RBFS(problem, node, f-limit) returns a solution, or failure and a new f-cost limit
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  successors  $\leftarrow$  []
  for each action in problem.ACTIONS(node.STATE) do
    add CHILD-NODE(problem, node, action) into successors
  if successors is empty then return failure,  $\infty$ 
  for each s in successors do /* update f with value from previous search, if any */
    s.f  $\leftarrow$  max(s.g + s.h, node.f)
  loop do
    best  $\leftarrow$  the lowest f-value node in successors
    if best.f > f-limit then return failure, best.f
    alternative  $\leftarrow$  the second-lowest f-value among successors
    result, best.f  $\leftarrow$  RBFS(problem, best, min(f-limit, alternative))
    if result  $\neq$  failure then return result

```

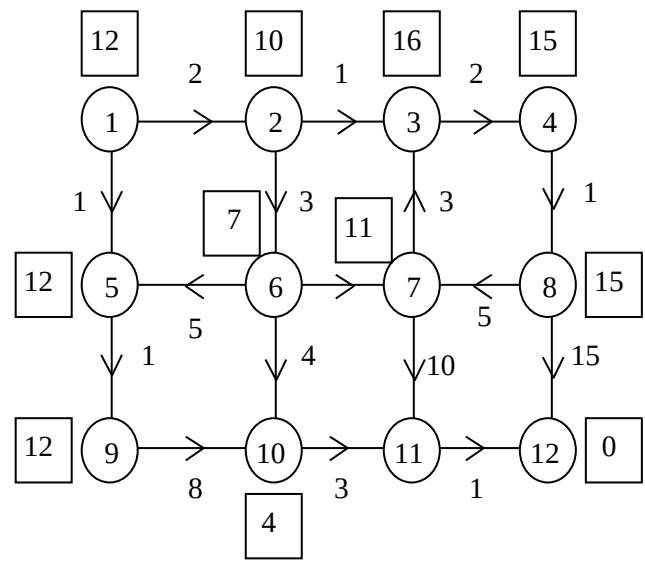
Algorithm for recursive Best First Search

3.11.1 Advantages and Disadvantages of RBFS:

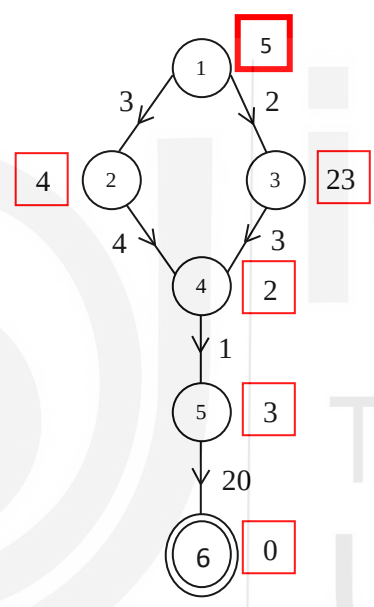
- More efficient than IDA* and still optimal.
- Best-first Search based on next best f -contour; fewer regenerations of nodes.
- Exploit the results of the search at a specific f -contour by saving the next f -contour associated with a node whose successors have been explored.
- Like IDA*, RBFS still suffers from excessive node regeneration.
- IDA* and RBFS are not good for graphs.
- Can't check for repeated states other than those on the current path.
- Both are hard to characterize in terms of expected time complexity.

☞ Check Your Progress 2

Q.1 Apply A* on the following graph: -

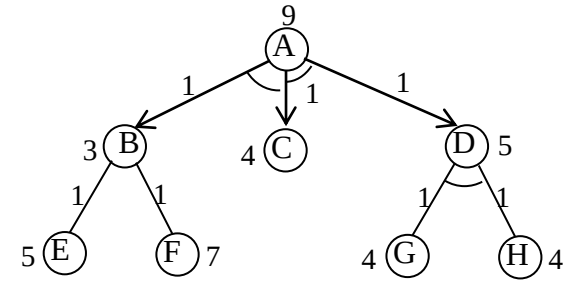


Q.2 Apply A* algorithm on the following graph

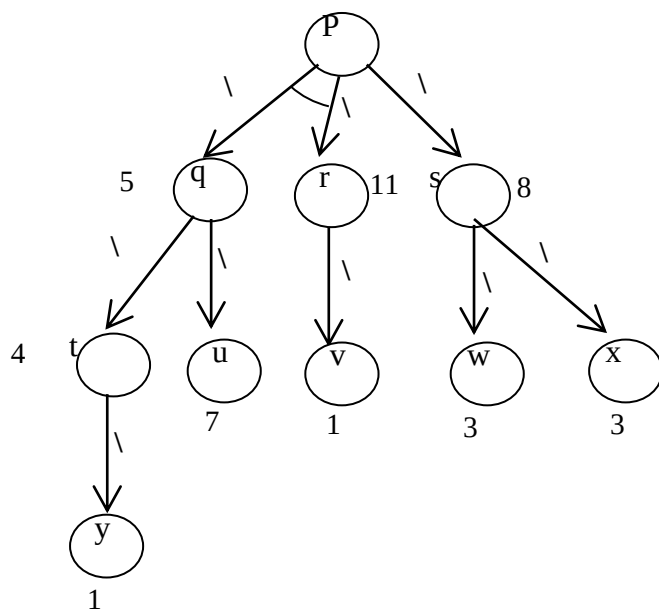


Q.3 Differentiate between the A* and AO* algorithms.

Q.4: Apply AO* algorithm on the following graph. Heuristic value is also given at every node, and assume the edge cost value of each node is 1.



Q.5 Apply AO* algorithm on the following graph. Heuristic value is also given at every node and assume the edge cost value of each node is 1.



Example 6: Given the 3 matrices A1, A2, A3 with their dimensions (3×4) , (4×10) , (10×1) , respectively. Consider the problem of solving this **chain matrix multiplication**. Apply the concept of AND-OR graph and find a minimum cost solution tree.

(Multiple choice Questions)

Q.6 A* algorithm always finds an optimal solution if

- A. h' is always 0
- B. g is always 1
- C. h' never overestimates h
- D. h' never underestimate h

Q.7 A* algorithm uses $f^* = g + h^*$ to estimate the cost of getting from the initial state to the goal state, where g is a measure of the cost of getting from the initial state to the current node, and the function h^* is an estimate of the cost of getting from the current node to the goal state. To find a path involving the fewest number of steps, we should test,

- A. $g=1$
- B. $g=0$
- C. $h^* = 0$
- D. $h^* = 1$

3.12 SUMMARY

- As the name ‘Uninformed Search’ means the machine blindly follows the algorithm regardless of whether right or wrong, efficient or inefficient.
- These algorithms are brute force operations, and they don’t have extra information about the search space; the only information they have is on how to traverse or visit the nodes in the tree. Thus, uninformed search algorithms are also called **blind search** algorithms.
- The search algorithm produces the search tree without using any domain knowledge, which is a brute force in nature. They don’t have

any background information on how to approach the goal or whatsoever. But these are the basics of search algorithms in AI.

- **The different types of uninformed search algorithms are as follows:**
 - Depth First Search
 - Breadth-First Search
 - Depth Limited Search
 - Uniform Cost Search
 - Iterative Deepening Depth First Search
 - Bidirectional Search (if applicable)
- The following terms are frequently used in any search algorithms:
 - **State:** It provides all the information about the environment.
 - **Goal State:** The desired resulting condition in a given problem and the kind of search algorithm we are looking for.
 - **Goal Test:** The test to determine whether a particular state is a goal state.
 - **Path/Step Cost:** These are integers that represent the cost to move from one node to another node.
- To evaluate and compare the efficiency of any search algorithm, the following 4 properties are used:
 - **Completeness:** A search algorithm is said to be complete if it guarantees to return a solution if exist.
 - **Optimality/Admissibility:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.
 - **Space Complexity:** A function describing the amount of space(memory) an algorithm takes in terms of input to the algorithm. That is how much space is used by the algorithm? Usually measured in terms of the maximum number of nodes in memory at a time.
 - **Time Complexity:** A function describing the amount of time the algorithm takes in terms of input to the algorithm. That is, how long (worst or average case) does it take to find a solution?
- Time and space complexity are measured in terms of: ‘b’ – maximum branching factor (the maximum number of successors (children) of any node) in a tree, ‘d’ – the depth of the shallowest goal node, and ‘m’ – maximum depth of the search tree (maybe infinity).
- The following table summarizes the 4 properties (**Completeness, Optimality/Admissibility, Space Complexity, Time Complexity**) of the search algorithm.

	BFS	DFS	IDDFS	Bidirectional Search (if Applicable)
Time	bd	bd	bd	bd/2
Space	bd	bm	bd	bd/2
Optimum?	Yes	No	Yes	Yes
Complete?	Yes	No	Yes	Yes

- The advantage of DFS is that it requires very little memory as compared to BFS, as it only needs to store a stack of the nodes on the path from the root node to the current node. The disadvantages of DFS are as follows: There is the possibility that many states keep recurring,

and there is no guarantee of finding the solution. The DFS algorithm goes for deep down searching, and sometimes it may go into an infinite loop.

- On the other hand, BFS gives a guarantee to get an optimal solution, if any solution exists (Completeness) and if there is more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps (Optimal), but one major drawback of BFS is that it requires lots of memory space since each level of the tree must be saved into memory to expand the next level.
- Iterative Deepening DFS (IDDFS) combines the benefits of both BFS and DFS search algorithms in terms of fast search and memory efficiency. It is better than DFS and needs less space than BFS. But the main drawback of IDDFS is that it repeats all the work from the previous phase.
- The advantage of Bidirectional search (BS) is that it uses various techniques like DFS, BFS, DLS, etc, so it is efficient and requires less memory. The Implementation of the bidirectional search tree is difficult, and in bidirectional search, one should know the goal state in advance.
- In Informed search, the domain-dependent (heuristic) information is used in order to search the space more efficiently.
- Informed searches include the following search:
 - **Best-first search:** Order agenda based on some measure of how 'good' each state is.
 - **Uniform-cost search:** Cost of getting to current state from initial state = $g(n)$
 - **Greedy search:** Estimated cost of reaching goal from current state – Heuristic evaluation functions, $h(n)$
 - **A* search:** $f(n) = g(n) + h(n)$
- Admissibility: $h(n)$ never overestimates the actual cost of getting to the goal state.
- Informed Ness: A search strategy that searches fewer of the states person orders to find a goal state is more informed.
- A* algorithm avoids expanding paths that are already expensive.
- In A*, the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ = cost so far to reach n , $h(n)$ = estimated cost to goal from n , and $f(n)$ = estimated total cost of path through n to goal.
- A* search uses an admissible heuristic function, i.e., $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost of the cheapest solution from n .
- A* search has a very good property: A* search is optimal! So, if there is any solution, A* search is guaranteed to find a least cost solution. Remember, this needs an admissible heuristic.
- The commonly used memory bound heuristics search are Iterative deepening A* (IDA*), Recursive Best-First search (RBFS), and Memory-bound A* (MBA*).
- Iterative Deepening Search (IDS) is nothing but BFS plus DFS for tree search. IDA* algorithm is "complete and Optimal" algorithm.
- The idea of recursive best first search is to simulate A* search with **O(bd)** memory, where b is the branching factor, and d is the solution depth.

3.13 SOLUTIONS/ANSWERS

Check your progress 1:

Answer 1:

Table 1: Open and closed list for BFS

OPEN LIST	CLOSED LIST
1	1
2,3,4	2
3,4,6	3
4,6,5	4
6,5,7	5
5,7	6
7,	7
8-Goal State	-

Table 2: Open and closed list for DFS

OPEN LIST	CLOSED LIST
1	1
4,3,2	2
4,3,6	6
4,3	3
4,5	5
4,7	7
4,8-Goal State	-

Answer 2

- open = [A]; closed = []**
- open = [B,C,D]; closed = [A]** B is not the goal.
Put his children onto the queue.
- open = [C,D,E,F]; closed = [B,A]** Put him in closed. He is done.
- open = [D,E,F,G,H]; closed =[C,B,A]** Items between red bard are siblings.
- open = [E,F,G,H,I,J]; closed = [D,C,B,A]**
- open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]**
- open = [G,H,I,J,K,L,M,N]** (as L is already on open);**closed = [G,F,E,D,C,B,A]**
- open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]**
- and so on until either goal is reached or open is empty.

Answer 3: Refer BFS section for solution.

Answer 4

	BFS	DFS	IDDFS	Bidirectional Search (if Applicable)

Time	b^d	b^d	b^d	$b^{d/2}$
Space	b^d	bm	bd	$b^{d/2}$
Optimum?	Yes	No	Yes	Yes
Complete?	Yes	No	Yes	Yes

Uninformed & Informed
Search

Check your progress 2

Answer 1:

The OPEN and CLOSED list are shown below. Node with their $f(n)$ values are inserted in OPEN list and that node will be expended next whose $f(n)$ value is minimum.

CLOSED

$1^{(12)}$	$2^{(12)}$	$6^{(12)}$	$5^{(13)}$	$10^{(13)}$	$11^{(13)}$	$12^{(13)}$
------------	------------	------------	------------	-------------	-------------	-------------

OPEN

$1^{(12)}$
$2^{(12)}$ $5^{(13)}$
$5^{(13)}$ $3^{(14)}$ $6^{(12)}$
$5^{(13)}$ $3^{(14)}$ $7^{(17)}$ $10^{(13)}$
$3^{(19)}$ $7^{(17)}$ $10^{(13)}$
$3^{(19)}$ $7^{(17)}$ $10^{(13)}$ $9^{(14)}$
$3^{(19)}$ $7^{(17)}$ $9^{(14)}$ $11^{(13)}$
$3^{(19)}$ $7^{(17)}$ $9^{(14)}$ $12^{(13)}$

Note that only 6 nodes are expended to reach to a goal node. Optimal cost to reach from start state (1) to goal node (12) is **13**.

Note: If all the edge cost is positive then **Uniform cost search** (UCS) algorithm is same as Dijkstra's algorithm. Dijkstra algorithm fails if graph is having a negative weight cycle. A* algorithm allows negative weight also. It means A* algorithm work for negative (-ve) edge cost also. If some edge cost is negative, then at any point of successive iteration, we cannot say till that node we have optimum cost (because of the negative cost). So, in this case (-ve edge cost), nodes come back from CLOSED to OPEN.

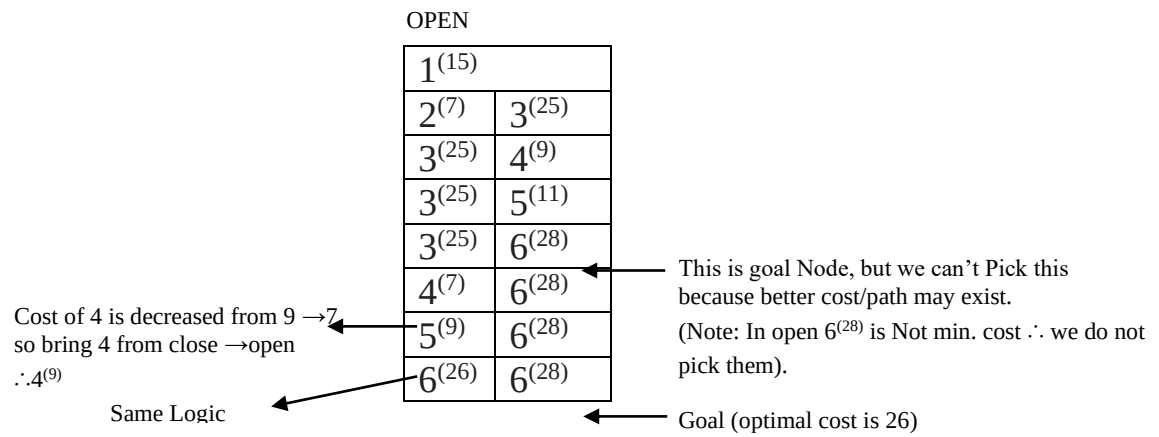
Let us see one example (Example 2) having negative edge cost and you can also see how nodes come back from CLOSED to OPEN.

Answer 2:

The OPEN and CLOSED list are shown below. Node with their $f(n)$ values are inserted in OPEN list and that node will be expended next whose $f(n)$ value is minimum.

CLOSED

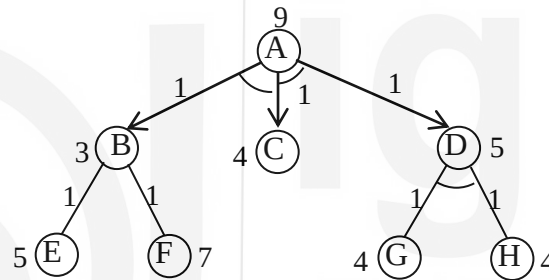
$1^{(15)}$	$2^{(7)}$	$4^{(9)}$	$5^{(11)}$	$3^{(25)}$	$4^{(7)}$	$5^{(9)}$
------------	-----------	---------------------------------	----------------------------------	------------	-----------	-----------



Optimal cost to reach from start state (1) to goal node (6) is **26**.

Answer 3: An A* algorithm represents an OR graph algorithm that is used to find a single solution (either this or that), but an AO* algorithm represents an AND-OR graph algorithm that is used to find more than one solution by ANDing more than one branch.

Answer 4 :



Path - 1: $f(A-B-C) = 1+1+3+4=9$

$f(B-E) = 1+5=6$

$f(B-F) = 1+7=8$

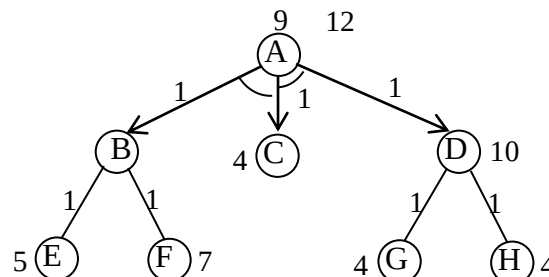
$f(A-B-C) = 1+1+6+4=12$

Path-2 : $f(A-C-D) = 1+1+4+5=11$

$f(D-G-H) = 1+1+4+4=10$

$f(A-C-D) = 1+1+4+10=16$

AND-OR graph with revised cost is shown in figure.



So, the optimal cost $f(A-B-C) = 12$

Note that AO* algorithm does not explore all the solution path once it finds a solution.

Answer 5: Similar to Q.4

Answer 6:

Given a chain of matrices: $A_1, A_2, \dots, A_n$, where each matrix A_i has a dimension $p_{i-1} \times p_i$. Problem is to determine the order of multiplication or the way of parenthesizing the product of matrices, so that the minimum number of required operations (i.e., multiplications) can be minimized.

There are only 2 ways to parenthesizing the given 3 matrices, A_1, A_2, A_3 :

$$A_1 \times (A_2 \times A_3)$$

or

$$(A_1 \times A_2) \times A_3$$

As we know, if matrix A is of dimension $(p \times q)$ and matrix B is of dimension $(q \times r)$, then the cost of multiplying A to B that is $(A \times B)$ is $(p \times q \times r)$ and the final dimension of the resultant matrix $(A \times B)$ is $(p \times r)$.

Let us see how AND-OR graph is used to get the solution of this problem.

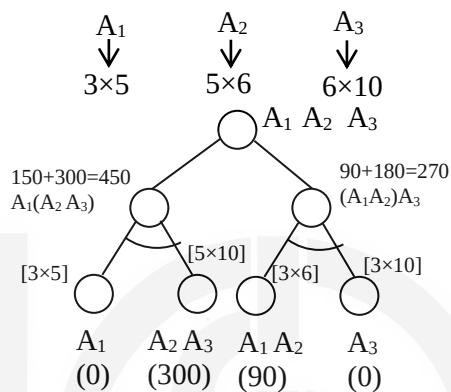


Figure-1 AND/OR graph for multiplying $A_1 \times A_2 \times A_3$

In this AND-OR graph, parent (root) node indicates the given problem for multiplying $A_1 A_2 A_3$. Next level of the tree (2 successors node) indicates the 2 choices (or ways) of multiplying (or parenthesizing) the $A_1 A_2 A_3$; first way is $A_1 \times (A_2 \times A_3)$ and another way is $(A_1 \times A_2) \times A_3$. Since out of these two choices, anyone will be the solution so there is a OR node for this. In an OR node, we always mark the current best successor node. Next level we have an AND node. For any AND node we must add the cost of both the successor node.

Cost of multiplying $(A_2 \times A_3) = 5 \times 6 \times 10 = 300$ and dimension of $A_2 \times A_3$ is (5×10) . Since the dimension of A_1 is (3×5) and the dimension of $A_2 \times A_3$ is (5×10) , so the cost of multiplying $A_1 \times (A_2 \times A_3) = 3 \times 5 \times 10 = 150$. Thus the total cost will be $300+150=450$. Similarly,

The cost of multiplying $(A_1 \times A_2) = 3 \times 5 \times 6 = 90$ and dimension of $A_1 \times A_2$ will be (3×6) . Since the dimension of $A_1 \times A_2$ is (3×6) and the dimension of A_3 is (6×10) , so the cost of multiplying $(A_1 \times A_2) \times A_3 = 3 \times 6 \times 10 = 180$. Thus the total cost will be $90+180=270$. So, the best way to multiplying $A_1 \times A_2 \times A_3$ is $(A_1 \times A_2) \times A_3$ and the minimum cost of multiplying $A_1 \times A_2 \times A_3$ is 270.

Multiple Choice Questions

Answer 6: Option C

3.14 FURTHER READINGS

1. Ela Kumar, “Artificial Intelligence”, IK International Publications
2. E. Rich and K. Knight, “Artificial intelligence”, Tata Mc Graw Hill Publications
3. N.J. Nilsson, “Principles of AI”, Narosa Publ. House Publications
4. John J. Craig, “Introduction to Robotics”, Addison Wesley publication
5. D.W. Patterson, “Introduction to AI and Expert Systems" Pearson publication

